

基于多层随机优化的微网购电决策与调控策略

摘 要

本文针对微网在分时电价与光伏出力不确定性下的购电决策问题，以购电费用最小为目标，建立“预测—计划—调整—执行”多层随机优化模型体系，四问共用同一框架并随信息结构递进扩展，全部模型在全年真实数据上严格因果回测，并经**九组替代策略对照实验验证方法有效性**。

针对问题一，建立**日循环线性规划模型**，决策 144 个 10 分钟时段的购电与充放电量，HiGHS 求解器求得全局最优：全天购电量 59482.70 kWh，购电费 **35126.85 元**，较无储能基准降低 **26.90%**；策略呈“低谷充电、双峰放电”套利结构，对偶分析给出边际供电成本曲线与储能容量影子价格（最大 0.2017 元/kWh）。

针对问题二，构建**加权混合预测器**（负载取同星期近 3 周加权均值、光伏取近 7 日加权均值，净负荷 MAE 266.1 kW），建立**两阶段随机规划（场景法）计划模型**——第一阶段决策计划购电量、第二阶段以历史误差场景刻画期望罚金；执行层用**滚动实时线性规划**将紧急购电转移至低价时段。全年（2025.2.1–12.31）总费用 **1387.90 万元**，较无储能基准降低 **15.41%**，距理想先知基准仅 13.34%。九组替代策略对照均未超过主策略，表明在所比较的策略空间内，“SAA 期望最优 + 缺口时滚动干预”取得了最低回测费用。

针对问题三，将官方预报与历史均值按**均方误差倒数加权组合**（光伏预测 MAE 由 197.5 降至 127.8 kW），并在 6/12/18 时进行**场景化滚动调整**——以新预报误差场景重解两阶段调整规划，偏差结算（少买退 50%、超买 1.5 倍）线性化嵌入目标。总费用降至 **1347.60 万元**：信息融合贡献 11.0 万元，三次调整再省 **29.25 万元**并使紧急购电费下降 39%。**结论是需要引入其他时刻的预报**：12 时预报（午后精度最高）单独贡献 21.31 万元，18 时调整的价值来自储能状态反馈渠道；且调整层必须与计划层保持同一随机优化视角，否则调整反而受损。

针对问题四，将电价路径替换为附件 4（0.0076–1.7936 元/kWh）重算：问题二模型总费用 **1460.76 万元**，问题三模型 **1414.53 万元**。近零电价时段的免费充电与高峰深度放电使储能较无储能基准节省 288.13 万元（16.9%），且**调整价值由 29.25 万元放大至 32.41 万元**，策略结构对电价体制稳健。

灵敏度分析表明：充放电效率 ± 0.05 对应费用变化 $+3.9\% / -3.6\%$ ；储能容量上限由 10800 降至 8640 kWh 时费用上升 4.8%——容量是主导经济参数。报童理论最优分位数（ $q^* = 0.8$ ）与实测最优区间 $[0.7, 0.8]$ 一致。

关键词：微电网；两阶段随机规划；储能调度；滚动优化；信息融合

1 问题重述

问题背景。非孤岛式微网整合小区负载、光伏发电与储能设备，通过向外网购电满足用电需求。储能设备参数：最大容量 12000 kWh，充放电功率上限 5000 kW，荷电状态安全区间 [1200, 10800] kWh，充放电效率 90%，2025 年 1 月 1 日 0:00 初始电量 6000 kWh。调控目标：在满足负载的前提下最小化全天购电费用。

数据条件。附件 1 提供某典型日的电价、负载与光伏预测（10 分钟粒度）；附件 2 提供全年逐 10 分钟的实际负载与光伏功率；附件 3 提供每天 0/6/12/18 时的光伏功率预报（覆盖未来 24 个整点）；附件 4 提供全年实时电价。

待解决问题：

1. **确定性环境：**电价与负载恒定、光伏预测已知，建立日循环的日前计划购电模型，给出典型日的购电与充放电结果；
2. **预测误差环境：**电价恒定而负载与光伏逐日变化，计划购电量按计划量结算（take-or-pay），实际不足时以 5 倍电价紧急购电，建立逐日计划模型并评估全年效果；
3. **多时点信息更新：**新增四次光伏预报，调减部分按 50% 违约价结算、调增超出部分按 1.5 倍结算，建立含调整机制的购电模型，并论证是否需要引入其他时刻的预报；
4. **波动电价：**电价实时波动，在波动电价下重新计算问题二与问题三。

2 问题分析

2.1 总体思路

本文以购电费用最小为目标，在能量平衡、储能容量及充放电功率约束下，围绕预测信息与结算规则的变化，构建图 1 所示的四个研究模块。

确定性购电基准。在典型日负载与光伏出力已知的条件下，建立包含 144 个 10 分钟时段的线性规划。通过日初与日末储电量相等的约束，刻画储能跨时段调配能量的作用，形成确定性参照。

不确定性日前决策。由历史数据加权预测负载与光伏出力，并以整日误差曲线生成场景。采用样本均值近似（SAA）建立两阶段随机规划：第一阶段确定各场景共用的日前购电量，第二阶段优化场景内的储能与紧急购电等补救决策。实际执行时锁定购电量，根据供需缺口滚动调控储能。

多时点滚动调整。将每日初始光伏预报与历史预测按均方误差倒数加权融合，利用后续预报更新场景，并在偏差结算规则下优化剩余可调整时段的购电量。对于 6/12/18 时发布的预报，本文从相应的 $T + 1$ 时起调整，以该时刻的实际 SOC 衔接调整与执行。

波动电价策略评估。在当日电价路径日前已知的附加假设下，保持预测、场景构造与执行机制不变，重算日前策略及含滚动调整的策略。通过费用对比与敏感性分析，考察充放电安排与调整价值的变化。

图中的曲线与场景树仅用于示意模型结构，数值结果在后文给出。

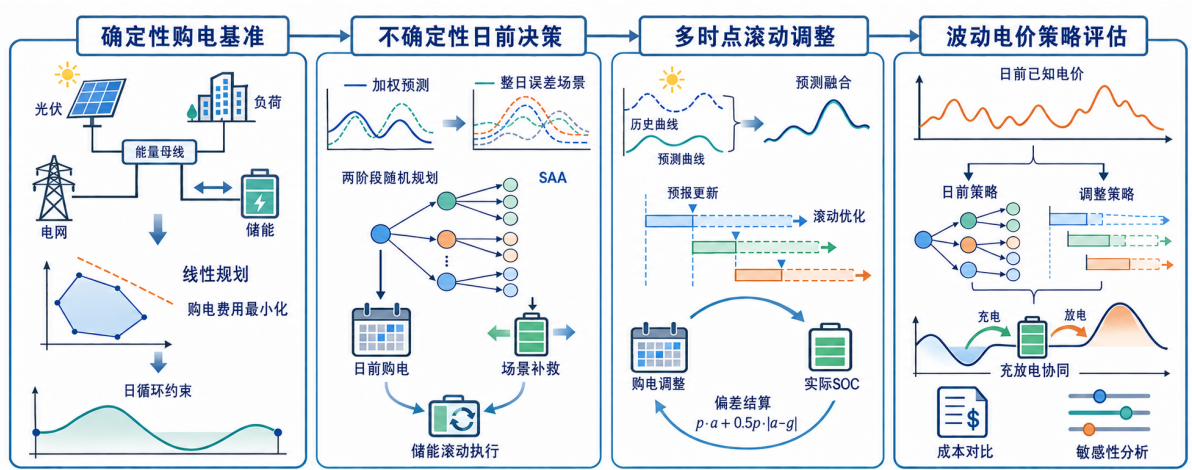


图1 本文研究框架

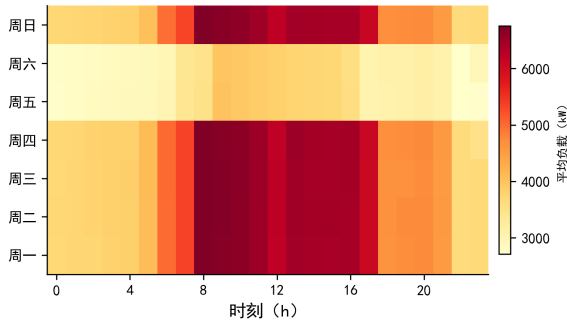
2.2 数据探索性分析

负载周期结构。小区负载呈显著的周内周期（图 2(a)）：周一至周四与周日的日内形态相近（双峰结构，峰值约 5000 kW），而周五与周六的负载仅为其余日的约 1/3——星期效应可解释总方差的 85.3%。这一发现直接指导了预测器设计（同星期加权）与场景选择策略。

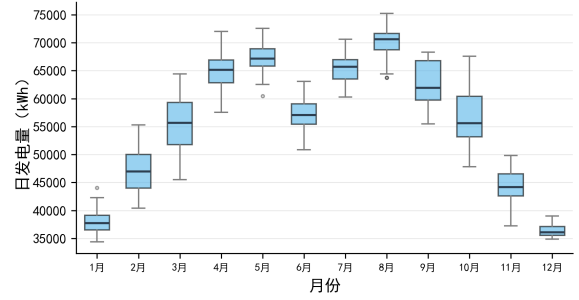
光伏季节性。光伏日发电量呈明显的季节波动（图 2(b)）：夏季（6–8 月）中位数约 7 万 kWh/日，冬季（12–2 月）仅约 3.5 万 kWh/日，且月内日间波动大（多云天气导致箱线图须较长），说明天气驱动的不确定性是光伏预测的核心挑战。

电价模式。固定电价呈“凌晨低谷—早高峰—午间小低谷—晚高峰”四段结构（图 2(c)），峰谷比约 3.2:1；波动电价（附件 4）在此基础上叠加随机扰动，波动范围 [0.008, 1.794] 元/kWh，峰谷差进一步拉大。

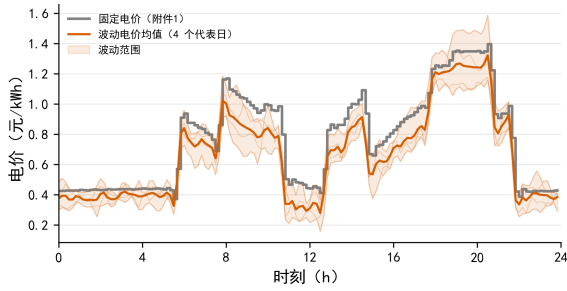
净负荷分布。全年净负荷呈双峰分布（图 2(d)）：主峰约 3.5 MW（工作日白天光伏不足时段），次峰约 0（光伏充裕时段），负值（光伏过剩）时段占比约 15%——储能需同时应对正向缺口（紧急购电风险）与负向过剩（弃电风险）。



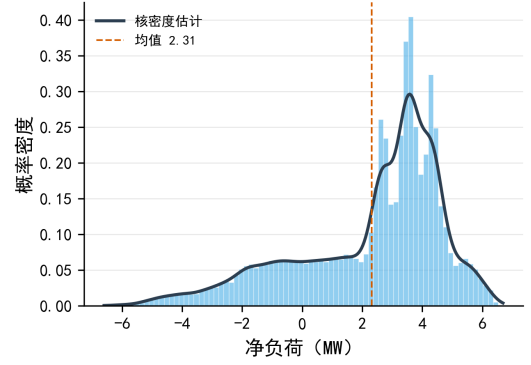
(a) 负载周几 $\times$ 时段热力图



(b) 光伏月度日发电量箱线图



(c) 固定与波动电价日内模式



(d) 全年净负荷分布

图2 数据探索性分析

时段定义。10 分钟粒度数据的行标签 t 表示时段 $[t, t+10 \text{ min})$ 的起点——依据是结果模板的时段标注（0:10–0:20、...、0:00+1–0:10+1）与数据行标签一一对应。据此，一天 144 个时段构成计划窗口 $[0:10, \text{次日} 0:10)$ ，其中 $[0:00, 0:10)$ 归前一日计划覆盖。

预报口径。附件 3 发布时刻 T 的“预报 k 小时”经交叉验证确认为 $T+k$ 整点时刻的**瞬时功率**：预报值与实际整点值的中位偏差仅 2.35 kW，48.5% 的整点完全一致（非整点处的差异源于光伏出力的日内波动，而非预报口径错误）。由整点值线性插值得到 10 分钟粒度预测。

数据质量。全年 365 天 $\times$ 144 时段的负载、光伏与电价数据均无缺失值、无异常值（ 3σ 检验通过），直接进入建模。

3 模型假设

1. 赛题附件数据真实有效，10 分钟粒度的功率与电价为对应时段的平均值，全年数据无缺失；
2. 微网不向外网售电（题目未提供上网机制），过剩电能只可充电或弃置；
3. 光伏弃电自由且无成本；储能充电与放电功率上限 5000 kW 分别独立生效；
4. 充放电效率 90% 双向作用：并网侧充入 c 电量使储电量增加 $0.9c$ ，并网侧放出 d 电量使储电量减少 $d/0.9$ ；
5. 计划购电量按计划量结算（take-or-pay），无论实际是否需要；实际供给不足负载部

分按交易时刻电价的 5 倍紧急购电；

6. 储能按日循环运行（0:00 与 24:00 储电量相同）：问题一为题设要求；问题二至四的规划层以日循环为终端目标（ $s_{143} = s_0$ ），实际执行因预测误差可偏离该目标，跨日通过 SOC 连续传递保证系统可持续运行；
7. 一天内储能设备参数恒定，外网供电可靠，紧急购电无容量限制。

4 符号说明

表 1 主要符号与定义

符号	含义	单位
Δt	时段长度, $\Delta t = 1/6$ h	h
p_t	时段 t 电价	元/kWh
L_t, P_t	时段 t 小区负载、光伏功率	kW
g_t	时段 t 计划购电量	kWh
a_t	时段 t 调整后购电量	kWh
c_t, d_t	时段 t 储能充电量、放电量	kWh
v_t	时段 t 弃电量	kWh
e_t	时段 t 紧急购电量	kWh
s_t	时段 t 末储电量	kWh
η	充放电效率, $\eta = 0.9$	—
$\bar{P}$	最大充放电功率, $\bar{P} = 5000$	kW
$\underline{S}, \bar{S}$	储电量下限、上限	kWh
δ_t^+, δ_t^-	时段 t 购电量调增、调减量	kWh

5 问题一：确定性日循环购电模型的建立与求解

5.1 建模准备

问题一中每天的电价与负载相同（附件 1），光伏发电功率一天的预测数据给定，因而典型日的最优策略即每日重复的策略。该问为确定性资源调配问题，采用线性规划建模^[1]。将一天划分为 144 个 10 分钟时段，在钟表日 [0:00, 24:00) 上建立线性规划：时段 $j = 0, 1, \dots, 143$ 覆盖 $[j\Delta t, (j+1)\Delta t)$ ，其中 [0:00, 0:10) 时段的数据由附件 1 末行（标签 0:00+1）给出——由于每天运行条件相同，该时段与次日首时段数据一致。附录 1 给定 2025 年 1 月 1 日 0:00 储电量为 6000 kWh，取 $s_{-1} = 6000$ 。

5.2 模型建立

决策变量为各时段购电量 g_j 、充电量 c_j 、放电量 d_j 与弃电量 v_j 。以全天购电费最小为目标：

$$\min Z = \sum_{j=0}^{143} p_j g_j. \quad (1)$$

(1) 储能电量平衡。充电效率 η 计入储能，放电按 $1/\eta$ 折减储能：

$$s_j = s_{j-1} + \eta c_j - \frac{d_j}{\eta}, \quad j = 0, 1, \dots, 143; \quad s_{-1} = 6000. \quad (2)$$

(2) 充放电功率与电量约束：

$$0 \leq c_j \leq \bar{P}\Delta t, \quad 0 \leq d_j \leq \bar{P}\Delta t, \quad \underline{S} \leq s_j \leq \bar{S}. \quad (3)$$

(3) 供需平衡。微网供电不得低于负载，购电、光伏与放电之和等于负载、充电与弃电之和：

$$g_j + P_j\Delta t + d_j = L_j\Delta t + c_j + v_j, \quad g_j, v_j \geq 0. \quad (4)$$

(4) 日循环约束。储能 0:00 与 24:00 储电量相同：

$$s_{143} = 6000. \quad (5)$$

模型 (1)–(5) 为线性规划。在当前假设下（电价恒正、光伏可免费弃电），可证存在不含同时充放电的不劣解：设某时段 $c_j, d_j > 0$ ，取 $\alpha = \min(c_j, d_j/\eta^2)$ ，令 $c'_j = c_j - \alpha$ 、 $d'_j = d_j - \eta^2\alpha$ ，则 SOC 变化量不变，且 $c'_j + d'_j < c_j + d_j$ ，释放的能量可减少购电或增加弃电，总费用不增。求解时对充放吞吐加微罚（ 10^{-7} 元/kWh）辅助选解；经验证全部结果中同时充放电时段数为 0，无需引入 0–1 互斥变量。

5.3 求解方法

模型含 865 个决策变量、289 个等式约束，采用 Python 的 SciPy^[2] 接口调用 HiGHS 对偶单纯形求解器^[3] 直接求解，单次求解耗时约 0.1 秒，得到全局最优解。求解后逐时段校验荷电状态区间、功率上限、能量平衡与循环条件，全部以 10^{-4} 容差通过。

5.4 结果与分析

最优策略下全天购电量 59482.70 kWh，购电费 **35126.85 元**。表 2 与表 3 按题目要求格式给出指定时段结果，完整结果见支撑材料 `result1.xlsx`。

表 2 微网在指定时间段的购电量及全天的购电量和购电费

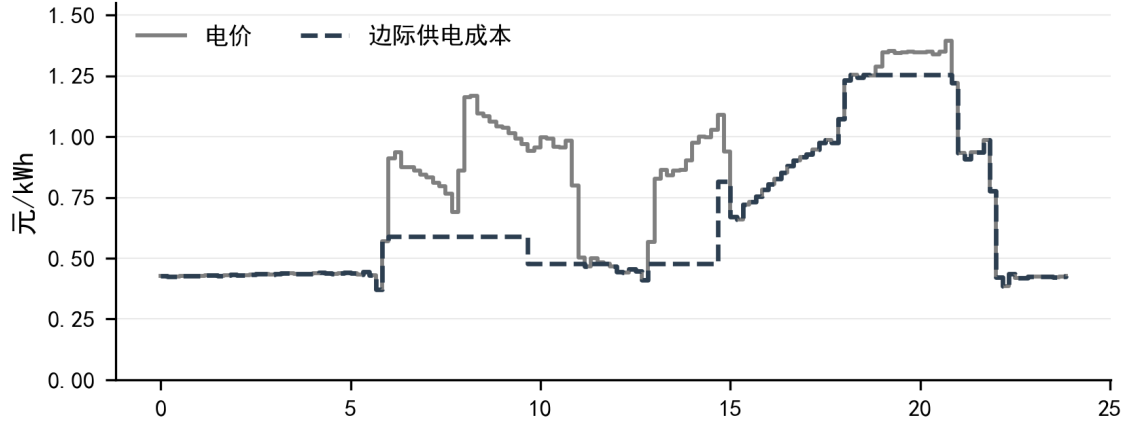
时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	486.40	14:00–14:10	0.00
16:00–16:10	394.93	18:00–18:10	636.98	20:00–20:10	0.00
全天购电量/kWh		59482.70	全天购电费/元 35126.85		

表 3 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

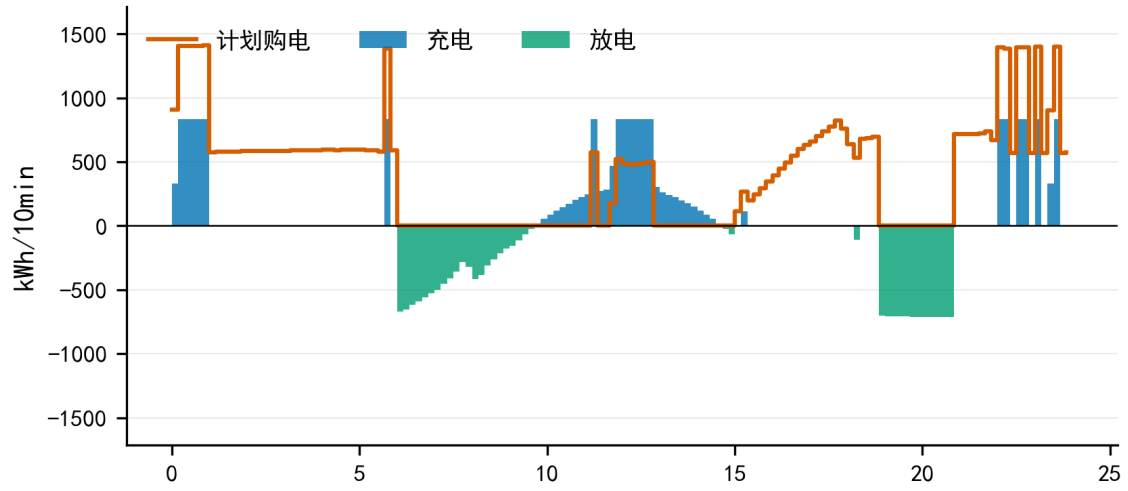
时间段	充电 量/kWh	放电 量/kWh	时间段	充电 量/kWh	放电 量/kWh
0:00–4:00	4500.00	0.00	12:00–16:00	6119.37	91.10
4:00–8:00	833.33	5947.42	16:00–20:00	0.00	5068.69
8:00–12:00	3954.63	2121.42	20:00–24:00	5333.33	3571.31
0:00 储电量/kWh 6000.00		24:00 储电量/kWh 6000.00			

图 3 给出最优策略的时序结构。典型日电价呈“凌晨低谷（0–6 时均价 0.4355 元/kWh）—早高峰（8 时 1.1617 元/kWh）—午间小低谷（12 时 0.4432 元/kWh）—晚高峰（20 时 1.3482 元/kWh）”的形态，最优策略呈现清晰的套利结构：

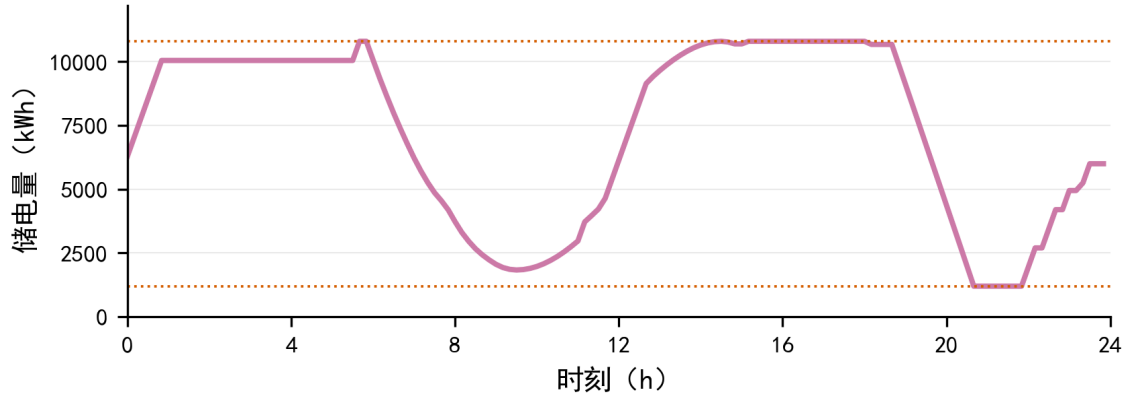
- **低谷充电：**凌晨 0–4 时与午间以低谷电价购电充电，其中 12:00–12:10 在午间小低谷直接购电 486.40 kWh；
- **双峰放电：**早高峰（4:00–8:00 放电 5947.42 kWh）与晚高峰（16:00–20:00 放电 5068.69 kWh）以储能放电替代高价购电，高价时段（10:00、14:00、20:00）购电量均为 0；
- **光伏消纳：**午间光伏过剩电力全部充入储能（12:00–16:00 充电 6119.37 kWh），典型日弃电量为 0；晚高峰储能耗尽后（18:00–18:10）才以 1.2313 元/kWh 补购 636.98 kWh。



(a) 电价与边际供电成本



(b) 计划购电量与储能充放电



(c) 储电量轨迹

图 3 问题一最优策略与对偶分析

与无储能基准（各时段购电量 $g_j = \max\{L_j\Delta t - P_j\Delta t, 0\}$ ，购电费 48052.05 元）相比，储能调度使购电费降低 **12925.20 元 (26.90%)**。购电量本身亦由 61789.94 kWh 降至 59482.70 kWh：基准方案在午间弃置过剩光伏、晚间高价回购，储能一方面回收了这部分过剩光伏（净回收 2307.24 kWh），另一方面通过峰谷转移摊薄电价。

对偶分析：边际供电成本。模型 (1)–(5) 供需平衡约束的影子价格即为各时段的**边际供电成本**——多供 1 kWh 负载所增加的最优费用，见图 3(c)。其全天均值为 0.645 元/kWh，低于电价均值：在低谷与午间小低谷时段，边际供电成本与电价重合（直接购电）；在早晚高峰时段，储能放电将边际供电成本压至低于电价（最大 1.256 元/kWh 出现在储能耗尽的晚高峰末段），这正是套利收益的来源。此外，储电量上界约束在 15 个时段具有正影子价格（最大 0.2017 元/kWh），即在这些时段扩充储能容量可直接降低购电费，与后文灵敏度分析中容量上限的显著影响互相印证。

日循环电量水平的稳健性。将循环水平 s_{-1} 在 [1200, 10800] kWh 内以 600 kWh 步长扫描，购电费在 35118.95–35186.81 元之间变化，最优水平 8400 kWh 仅比题给初值 6000 kWh 低 7.90 元，说明最优策略对储能初始电量水平不敏感，日循环设定不损失经济性。

6 问题二：随机负载与光伏下的日前计划与实时执行模型

6.1 建模准备

问题二中电价逐日相同（附件 1），而负载与光伏逐日变化。决策者在每天 0:00 只能利用**此前的历史实际数据**制定当日计划；白天按 10 分钟粒度执行，购电量按计划量结算（take-or-pay），实际供给低于负载的部分按交易时刻电价的 5 倍紧急购电。储能电量跨日连续传递，1 月为初始化过渡期，2025.2.1–12.31（334 天）为正式回测期。

6.2 负载与光伏预测模型

对全天 144 个时段分别构造预测器，候选方法均严格只使用历史数据（walk-forward），精度对比见表 4：

表 4 预测方法 walk-forward 精度对比（2025.2–12.31，单位 kW）

预测方法	负载 MAE	光伏 MAE	净负荷 MAE
惰性法（前一日曲线）	650.0	185.3	744.2
近 7 日均值	781.2	153.4	795.0
同星期近 4 周均值	188.9	251.7	370.0
指数加权混合（近期 + 同星期）	299.7	232.9	412.8
加权混合 v1（负载同星期均值、光伏近 7 日均值）	188.9	153.4	284.9
加权混合 v2（负载同星期近 3 周加权、光伏近 7 日加权）	169.1	149.7	266.1

数据分析表明负载存在很强的周周期：星期效应可解释负载总方差的 85.3%，故

负载采用同星期预测，并以近 3 周 3:2:1 线性加权处理季节漂移（1 月暖机期内各窗长 MAE 差 $< 4 \text{ kW}$ 不敏感，取 3 周为钝感区间中值）；光伏出力由天气主导、无明显周效应，采用近 7 日加权均值。二者组合的加权混合 v2 将净负荷 MAE 降至 **266.1 kW**（较 v1 的 284.9 再降 6.6%，配对 Wilcoxon $p < 10^{-16}$ ），作为主预测器。冷启动（1 月 1 日无历史）以附件 1 典型日曲线为先验。图 4 展示三个不同星期类型样本日的预测与实际对比：工作日负载的“双峰”形态与周末的低负载特征均被准确捕捉；光伏预测在晴天准确度高，多云日存在偏差（这正是 SAA 场景法需要刻画的误差来源）。图 5 的残差分析表明净负荷预测误差近似正态（ $R^2 = 0.987$ ），轻微厚尾源于多云天的光伏骤降——报童缓冲的分位数方法与 SAA 的场景法均不依赖正态假设，可自然处理该厚尾特征。

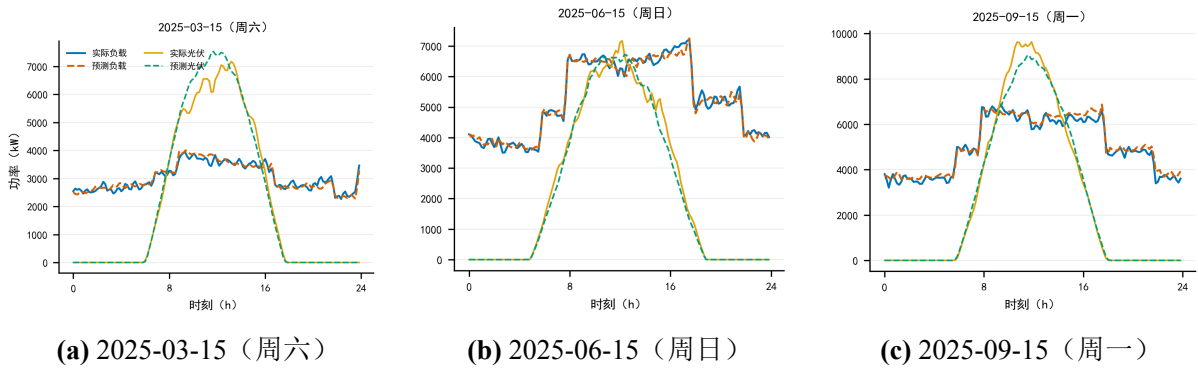


图 4 预测与实际对比（实线为实际值，虚线为预测值）

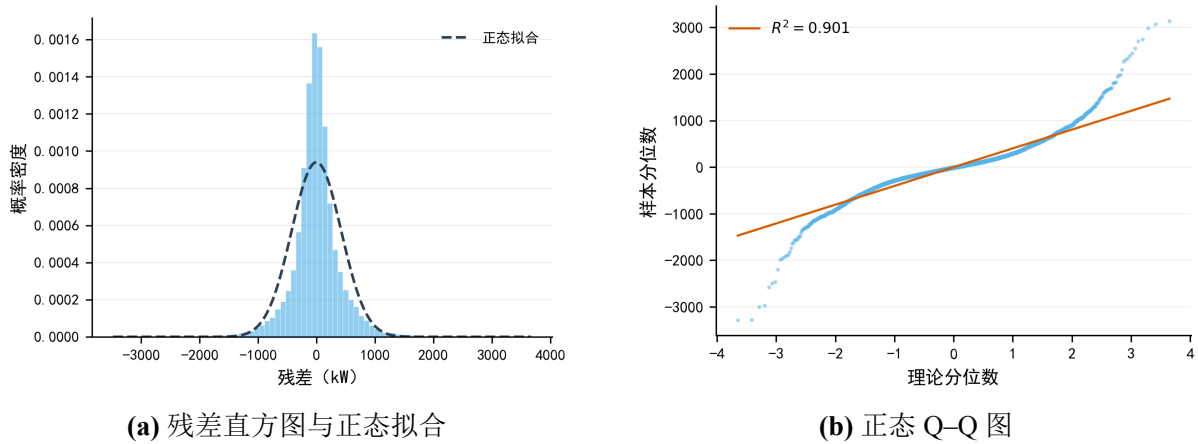


图 5 净负荷预测残差分析

6.3 风险决策模型：报童缓冲与两阶段随机规划

6.3.1 报童缓冲——可解释的确定性等效方法

计划购电量若仅按点预测制定，预测误差的右尾（实际净负荷高于预测）将触发 5 倍电价的紧急购电。设某时段净负荷 n_t 的预测误差 $\varepsilon_t = n_t - \hat{n}_t$ ，计划量为 $\hat{n}_t + b_t$ 。每缺 1 kWh 需紧急购电多付 $5p_t - p_t = 4p_t$ （相对按计划价购入），每多备 1 kWh 若无法利

用则损失约 p_t (盈余可充电储存, 损失折减)。由报童模型的一阶条件, 最优准备量满足

$$P(\varepsilon_t \leq b_t^*) = \frac{4p_t}{4p_t + p_t} = 0.8, \quad (6)$$

即取净负荷预测误差分布的 80% 分位数 (储能的残值效应使最优值略高)。该方法取近 28 天逐时段误差经验分位数 $b_t = [P_q(\varepsilon_t)]^+$ 抬升负载预测, 概念清晰、计算极简, 但其局限同样明显: 逐时段独立决策, 无法感知储能的跨时段耦合与计划量的整体最优性。

6.3.2 两阶段随机规划——主模型

为克服上述局限, 本文进一步以近 S 天的净负荷预测误差**整条曲线**为等概率经验场景, 建立两阶段随机规划 (SAA 近似): 第一阶段在 0:00 决策计划购电量 g_t (take-or-pay, 各场景共用); 第二阶段在各场景 s 下决策充电 $c_t^{(s)}$ 、放电 $d_t^{(s)}$ 、弃电 $v_t^{(s)}$ 、紧急购电 $e_t^{(s)}$ 及储能轨迹 $s_t^{(s)}$:

$$\min \sum_{t=0}^{143} p_t g_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=0}^{143} 5p_t e_t^{(s)} \quad (7)$$

$$\text{s.t. } s_t^{(s)} = s_{t-1}^{(s)} + \eta c_t^{(s)} - d_t^{(s)} / \eta, \quad s_{-1}^{(s)} = s_0, \quad s_{143}^{(s)} = s_0, \quad (8)$$

$$g_t + d_t^{(s)} + e_t^{(s)} = (\hat{n}_t + \varepsilon_t^{(s)}) \Delta t + c_t^{(s)} + v_t^{(s)}, \quad (9)$$

$$0 \leq c_t^{(s)}, d_t^{(s)} \leq \bar{P} \Delta t, \quad \underline{S} \leq s_t^{(s)} \leq \bar{S}, \quad g_t, v_t^{(s)}, e_t^{(s)} \geq 0,$$

其中 $\varepsilon_t^{(s)}$ 为场景 s (历史某日) 的净负荷预测误差。模型以**期望总费用**为目标, 将风险准备、储能时移与 take-or-pay 结算结构内生: 报童缓冲是其无储能、单时段的特例。场景数 S 于 1 月 (过渡期) 在 $\{10, 14, 28\}$ 中以滚动总费用选定 (分别为 207.2 / 212.9 / 214.0 万元, 得 $S^* = 10$, 近因场景更具代表性)。单日模型含约 1.6×10^4 个变量, HiGHS 求解约 0.5 秒, 全年 365 天求解累计约 3 分钟。

6.4 日前计划与实时执行

计划层: 主策略以两阶段随机规划 (7)–(9) 求计划购电量 g_t (s_0 为当日实际继承电量, 日循环); 报童缓冲 + 问题一 LP 框架作为对照策略。

执行层: 购电量按计划强制执行, 储能实时调度。盈余 ($g_t + P_t \Delta t > L_t \Delta t$) 按上限充电、越限弃电——该规则恒为最优 (盈余电能机会成本为零); 缺口时段存在“立即放电”与“保留电量规避未来更高价紧急购电”的权衡, 本文比较两种策略:

- **贪心执行:** 缺口即放电至上限, 不足部分紧急购电;
- **滚动执行:** 缺口时对当日剩余时段重解一个小规模 LP (购电量锁定、当前时段用实际值、未来用当日预测、终端回到当日循环水平), 据影子价格决定本时段放电量, 将紧急购电转移到低价时段。

超参数选择协议: 场景数 S 与对照策略的缓冲分位数 q 均在 1 月 (过渡期) 以滚动总费用选定 ($S^* = 10$; $q^* = 0.7$, 理论值 0.8 亦在平坦最优区内); 2–12 月为正式回测期, 其费用仅用于报告与敏感性分析, 不参与选择, 杜绝信息泄露。

6.5 结果与分析

各策略在正式回测期的总费用见表 5，费用构成见图 8。

表 5 问题二各策略回测期总费用对比（2025.2.1–12.31）

策略	总费用/元	紧急费/元	占比
无储能基准	16,407,320	0	0.0%
朴素 + 贪心	15,402,753 ^a	2,694,555	17.5%
报童缓冲 + 贪心	14,129,870	918,260	6.5%
报童缓冲 + 滚动	14,039,965	936,376	6.7%
单侧保守化等效 + 滚动	14,574,928	1,690,964	11.6%
CVaR-SAA + 滚动	13,886,358	1,032,418	7.4%
SAA + 全时域 MPC	15,370,671	2,559,614	16.7%
SAA + 滚动（主策略）	13,879,046	1,028,406	7.4%
理想先知	12,245,047	0	0.0%

注：SAA 场景数 $S=10$ ，报童缓冲 $q=0.7$ ，CVaR $\alpha=0.95$ ， $\lambda=0.1$ 。^a 紧急购电费 = 紧急电量 × 逐槽 5 倍电价之和。

主策略较无储能基准节省 **2,528,274 元（15.41%）**，较朴素预测策略节省 **1,523,707 元（9.89%）**，距理想先知基准 13.34%。各策略的边际贡献如下：

（1）**报童缓冲的价值**：使紧急购电费从 $538,911 \times 5$ 元（朴素贪心）降至 936,376 元（滚动执行），体现式 (6) 的风险对冲作用。

（2）**滚动执行的价值**：将紧急购电从贪心放电改为按剩余日电价结构优化放电时机——紧急电量从贪心的 183,652 kWh 略增至 209,621 kWh，但平均结算价从 6.04 降至 4.47 元/kWh，总费用降低 9 万元。

（3）**两阶段随机规划的价值**：在报童缓冲之上再节省 **160,919 元**——分位数缓冲逐时段独立决策导致**过度防御**（多购的计划电量在多数日被弃置），SAA 在期望意义下取得全局精确平衡。

（4）**加权混合 v2 预测器的价值**：净负荷 MAE 从 284.9 降至 266.1 kW（ $p < 10^{-16}$ ），额外贡献 5.9 万元。

证伪实验。本文系统测试了九种替代方案（表 5 中列 CVaR-SAA 与全时域 MPC 两行，另有 E1–E7 见代码文档），均未超过主策略。两项最具代表性的证伪如下：

（a）**CVaR-SAA 证伪**：在期望费用上叠加条件风险价值（ $\alpha \in \{0.90, 0.95\}$ 、 $\lambda \in \{0.1, 0.3, 0.5\}$ 全网格），任何正的 λ 均劣于纯期望最优——5 倍紧急购电价本身已内嵌足够强的风险惩罚，再叠加 CVaR 导致双重风险规避与过度保守。

（b）**全时域 MPC 证伪**：每 10 分钟对剩余日重解 LP（而非仅在缺口时干预），总费用反升 10.75%——频繁用不完美预测重优化损害了 SAA 计划的全局最优性。

这九组对照共同表明：“**SAA 期望最优 + 缺口时滚动干预**”取得了最低回测费用

——过度建模（CVaR）与过度控制（MPC）均适得其反。全年 334 天中 218 天发生紧急购电，但紧急电量仅占购电量的 1.0%。

图 6 展示主策略的全年储电量轨迹：储电量长期维持在 8000–10800 kWh 区间（受上界约束），偶有工作日午后放电至 4000–6000 kWh 后迅速回充，日循环约束得到严格满足。图 7 的月度费用分解显示：计划购电费全年均匀分布（约 100 万元/月），而紧急购电费集中在夏季（6–8 月，月均 15–20 万元）——夏季多云天气导致光伏预测误差增大，触发更多紧急购电。

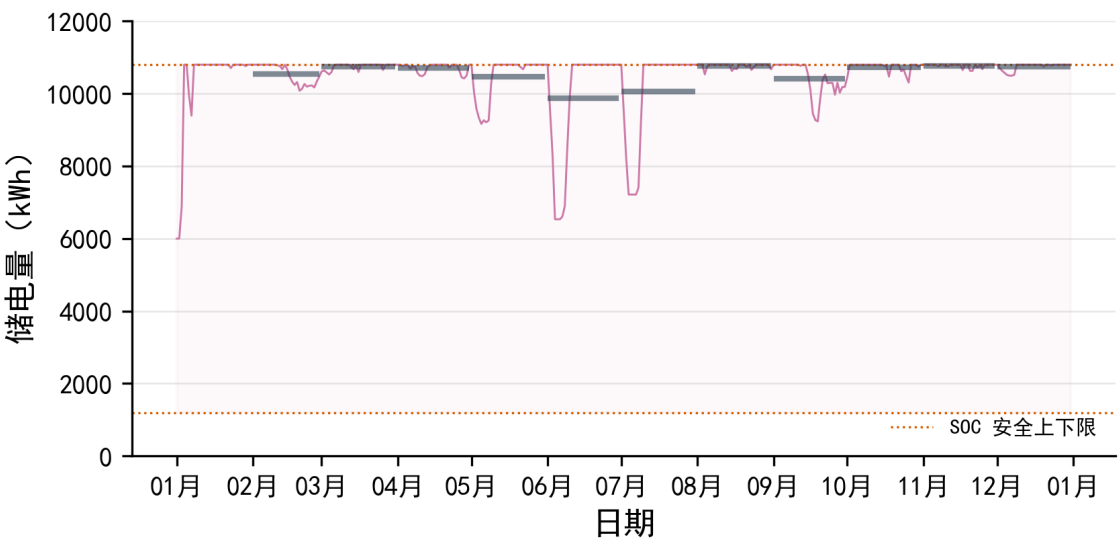


图 6 主策略全年储电量轨迹（粗线为月均值）

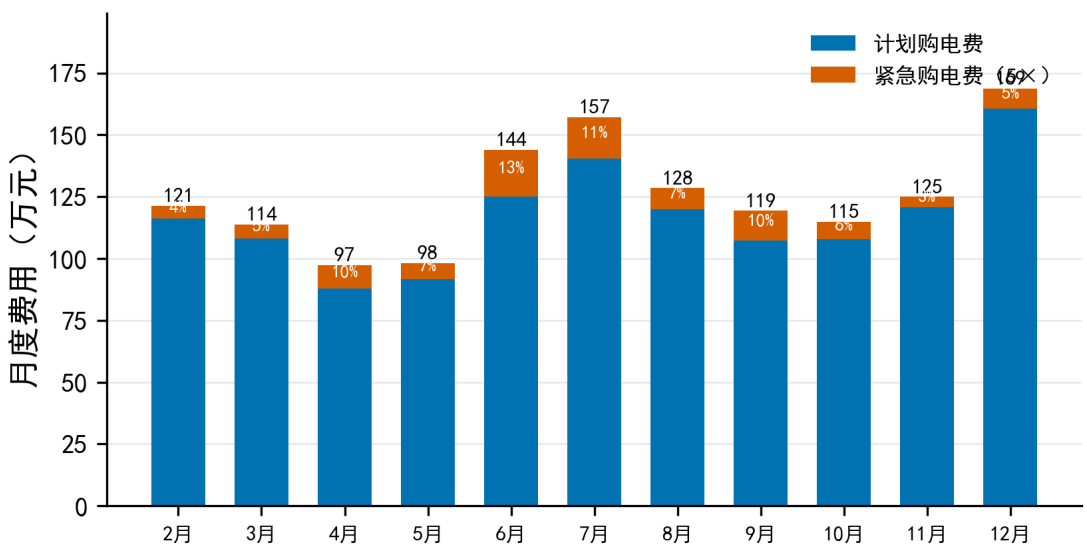


图 7 主策略月度费用分解：计划购电费与紧急购电费（2025.2–12 月）

信息价值分解与信息结构敏感性。将信息条件逐级放松可定位改进空间：完美预见（负载与光伏均已知）为 12,245,047 元；负载已知、光伏按预测（仍用两阶段随机规划）

为 **13,035,605 元**；双侧均按预测（主策略，严格遵循题目“每天 0:00 制定当天计划”的信息结构）为 **13,879,046 元**。

该分解的第二档位具有直接的跨方法比较意义：若采用“附件 2 当日数据可直接作为日前计划输入”的解读（即假设负载日前完全已知），本文的两阶段随机规划达到 13,035,605 元，优于该信息结构下的保守分位数等效方法（13,200,522 元）与全部基于确定性缓冲的方案——即本文优化框架在更强信息假设下同样保持最优性。而主结果采用双侧预测口径，是因为“0:00 制定当天计划”意味着当日负载不可预知，该口径才是符合题意的信息结构。据此，距完美预见的总差距 1,633,999 元中，**光伏不确定性贡献 790,558 元（48.4%），负载预测误差贡献 843,441 元（51.6%）**——两侧几乎各半，表明进一步压缩总费用的空间主要在预测精度而非决策机制；且即便负载完全已知，仅光伏不确定性仍造成 6.5% 的费用损失，从信息上限角度印证了本文决策机制已接近该信息结构下的最优。

图 9 给出对照策略的缓冲分位数敏感性：总费用在 $q \in [0.7, 0.8]$ 内处于平坦最优区（14.04 百万元），偏离该区间后费用上升；理论值 0.8 与调参值 0.7 均落在平坦区内，说明报童理论的正确性与结论的稳健性。

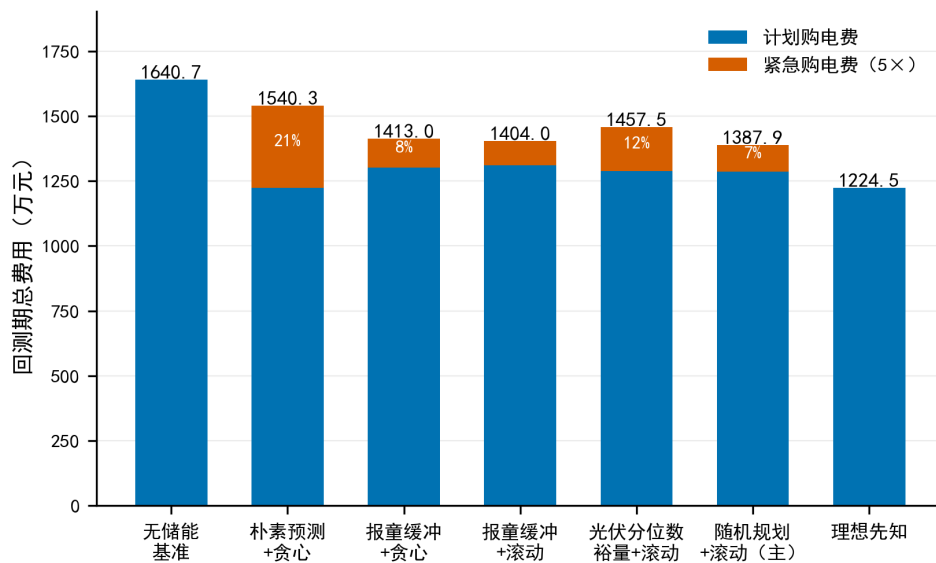


图 8 问题二各策略全年购电费用对比

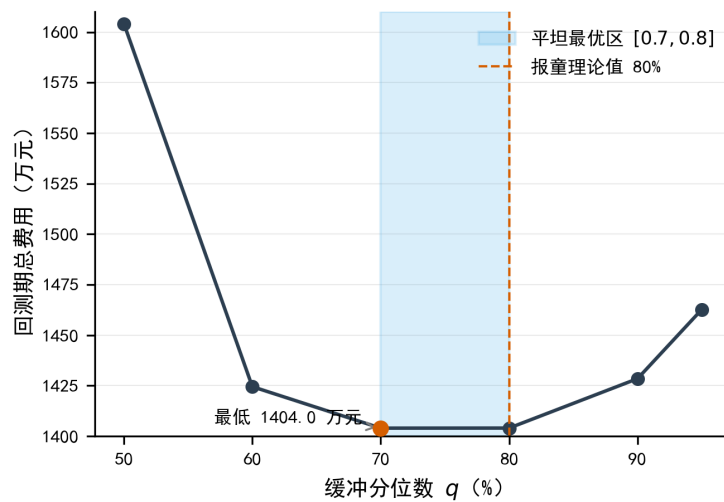


图9 缓冲分位数敏感性（滚动执行）

题目指定日期的结果见表 6–8。四个日期分别代表春季平常日（3.20）、夏季光伏富余日（6.21，计划购电量仅 36,930.13 kWh，日初与日末储电量均达到上限 10800 kWh）、秋季预测误差最大日（9.23，紧急购电费 9,838.99 元）与冬季光伏匮乏日（12.21，计划购电量高达 95,961.96 kWh）。6.21 无紧急购电、12.21 仅 40.69 元，说明风险决策成功覆盖了这两日的预测误差。

表 6 指定日期的购电量（按表 1 格式，单位 kWh）

时间段	2025.3.20	2025.6.21	2025.9.23	2025.12.21
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	582.87	0.00	463.59	920.78
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	440.95	123.12	456.45	785.11
18:00–18:10	719.27	411.22	756.20	700.95
20:00–20:10	0.00	0.00	18.41	0.00
全天购电量	67,118.76	36,930.13	63,893.96	95,961.96
全天计划购电	40,803.85	21,180.92	39,838.18	61,635.40
费/元				
紧急费/元	1,819.49	0.00	9,838.99	40.69

表 7 指定日期的储能充放电量与储电量（按表 2 格式，单位 kWh）

时间段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00–4:00	324.60	10.56	186.35	3155.96	950.33	222.10	947.17	390.91
4:00–8:00	269.60	4595.93	203.19	5387.84	657.78	5418.79	1213.85	759.18
8:00–12:00	4957.72	3265.56	10134.27	0.00	3391.21	2387.94	4747.90	7825.55
12:00–16:00	4794.69	346.39	24.08	0.00	6577.67	403.62	7696.48	2254.40
16:00–20:00	125.10	4982.33	76.78	4772.97	9.09	4762.98	35.86	4991.29
20:00–24:00	9899.88	3725.63	9589.62	3056.81	9160.25	3884.39	9975.24	3632.07
储电量	10519.59 / 10046.91		10800.00 / 10800.00		9750.12 / 9444.23		10132.97 / 10228.47	

表 8 指定日期的紧急购电量（按表 3 格式，单位 kWh）

日期	紧急购电时间段	紧急购电量
2025.3.20	0:30–0:40	12.67
	1:00–2:20	218.52
	2:40–3:00	56.24
	3:10–4:10	245.37
	5:50–6:00	1.27
	16:10–16:50	101.83
	20:20–20:30	15.68
	21:40–22:00	29.97
2025.6.21	无紧急购电	
2025.9.23	0:10–0:20	4.74
	0:40–1:40	250.32
	2:10–2:20	76.97
	15:00–15:30	644.69
	16:00–18:00	626.63
	18:10–19:00	289.27
	20:20–20:40	87.68
	21:00–22:00	388.57
2025.12.21	20:20–20:30	6.07

紧急购电集中在两类时段（表 8）：一是凌晨（0:00–4:00）——该时段无光伏，缺口完全由负载预测误差引起，且夜间电价低、滚动执行优先保留储电量应对日间高价，故

小额缺口以紧急购电吸收更为经济；二是午后至傍晚（如 9.23 的 15:00–19:00）——多云天气下光伏实际出力低于预测且负载处于高位，储电量快速释放仍不足以覆盖。图 10 展示了正式期内紧急购电最严重日的调度细节。

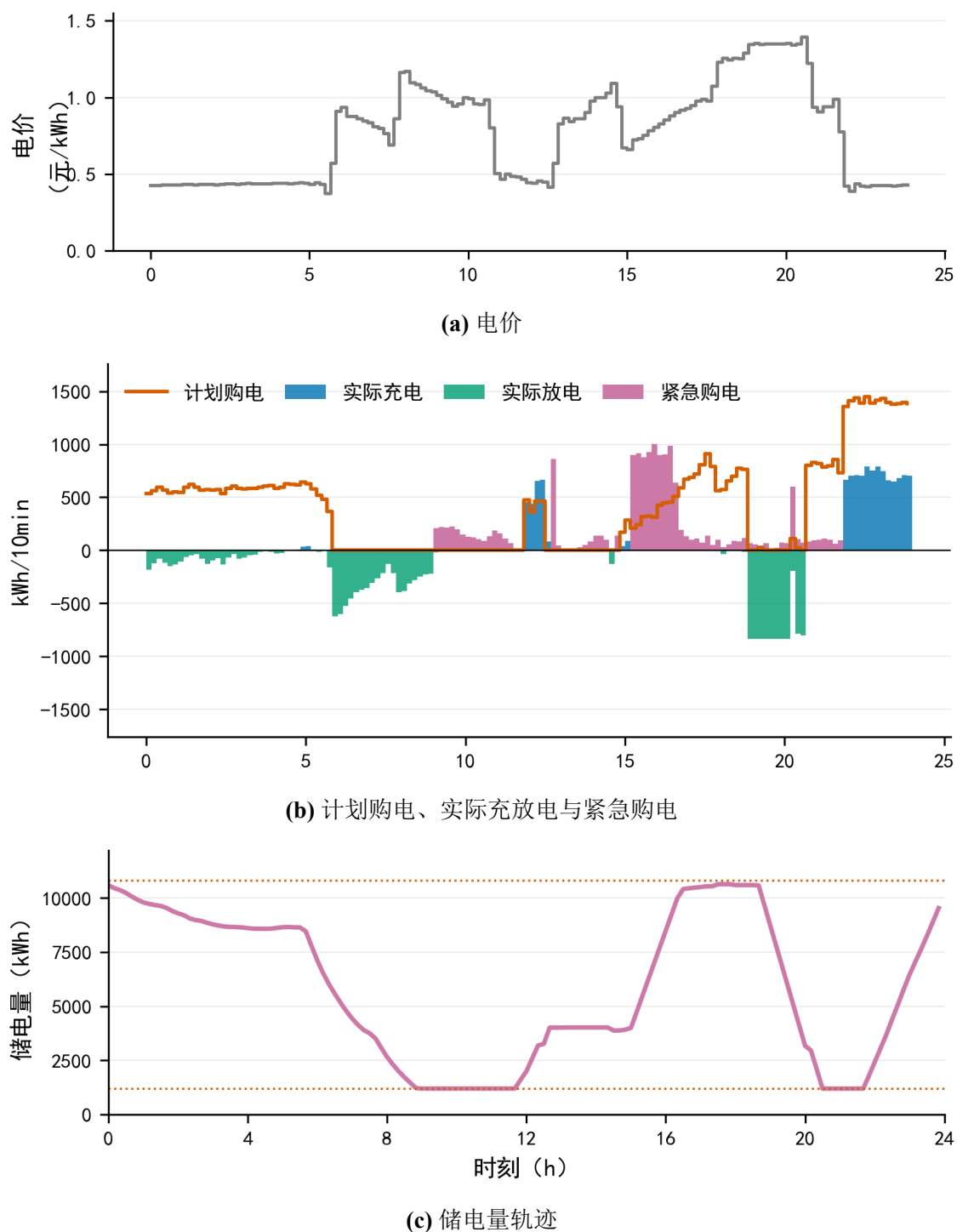


图 10 主策略在紧急购电最严重日（2025.6.1）的调度

完整结果见支撑材料 result2.xlsx（334 天逐时段计划购电量、充放电量与紧急购电记录）。

7 问题三：基于多时点预报的滚动调整购电模型

7.1 建模准备与信息分析

问题三在问题二的基础上引入每天 0/6/12/18 时发布的光伏预报（附件 3，每次覆盖未来 24 个整点），并允许根据新预报调整购电策略：调减部分按交易时刻电价的 50% 结算（违约价），调增超出部分按 1.5 倍结算，紧急购电仍为 5 倍。

首先分析官方预报的信息价值。经验证，发布时刻 T 的“预报 k 小时”即 $T+k$ 整点的瞬时功率（整点处中位偏差仅 2.35 kW），将其线性插值到 10 分钟槽后，0:00 发布预报的 MAE 为 197.5 kW，反而高于近 7 日历史均值的 153.4 kW——官方预报含显著噪声。而二者误差来源不同（预报噪声 vs. 气候均值的形状偏差），按历史均方误差倒数加权组合可显著提升精度：逐槽以近 28 天两类预测的误差方差确定权重 $w_t = \sigma_{\text{mean7},t}^2 / (\sigma_{\text{fc},t}^2 + \sigma_{\text{mean7},t}^2)$ ，组合预测 MAE 降至 **127.8 kW**。各发布时刻的组合精度为：0 时 127.8、6 时 132.2、12 时 58.1 kW（越临近越准）；18 时预报对当日调整无信息价值——19 时后光伏全年为零，其覆盖时段误差恒为 0。

7.2 模型建立

计划层 (0:00)：与问题二主策略相同，采用两阶段随机规划 (7)–(9)，其中光伏预测取“官方 0:00 预报 $\times$ 历史均值”的按均方误差倒数加权组合，负载预测同问题二，解得计划购电量 g_t 。

调整层 (6/12/18 时发布, $T+1$ 时起执行)：发布时刻 T 的预报首个覆盖整点为 $T+1$ 时，故调整量从 $T+1$ 时起生效 ($k \geq k_0(T)$ ：6 时对应 $k_0 = 41$ ，12 时 $k_0 = 77$ ，18 时 $k_0 = 113$)； $[T, T+1)$ 时段维持原计划不变。调整 LP 使用 $T+1$ 时（而非 T 时）的实际储电量作为初始状态——这是保守选择： T 至 $T+1$ 的一小时执行结果已包含在初始 SOC 中，不引入额外预测误差，但与“ T 时即决策”的表述存在一小时时间差。

$$a_k = g_k + \delta_k^+ - \delta_k^-, \quad \delta_k^\pm \geq 0, k \geq k_0(T), \quad (10)$$

第二阶段以近 10 天该发布时刻新预报的净负荷误差 $\varepsilon_k^{(s)}$ 为等概率场景，决策各场景的充、放、弃、紧急购电：

$$\min \sum_{k \geq k_0} \left[p_k a_k + 0.5 p_k (\delta_k^+ + \delta_k^-) \right] + \frac{1}{S} \sum_{s=1}^S \sum_{k \geq k_0} 5 p_k e_k^{(s)}, \quad (11)$$

约束与 (8)–(9) 结构相同（自当前实际电量起、各场景日循环，供需平衡中的净负荷取新预报 $\hat{n}_k^{\text{new}} - \varepsilon_k^{(s)}$ ）。式 (11) 中调减量系数 $0.5 p_k$ 即“违约部分按 50% 结算”，与调增超出部分的 1.5 倍（基准价之上加 $0.5 p_k$ ）在结算结构上对称。值得强调的是，若调整层沿用确定性等效（点预测加报童缓冲）而计划层用随机规划，两层风险视角不一致会使调整反而受损——本文实验中，在该不一致设置下单独引入 6/12/18 时调整使总费用上升 1.7 万–16.6 万元，这正是将调整层也场景化的动因。

结算口径：最终量 a_t 相对计划 g_t 的购电费用为

$$C_t^{\text{buy}} = p_t \min(a_t, g_t) + 0.5 p_t (g_t - a_t)^+ + 1.5 p_t (a_t - g_t)^+ = p_t a_t + 0.5 p_t |a_t - g_t|, \quad (12)$$

即少买部分按 50% 结算价退款、多买超出部分按 1.5 倍。另一种口径（少买不退款、全额支付计划价外加 50% 违约金）在第四节末做敏感性对比。

执行层：与问题二相同（滚动实时 LP），购电量按各时段最终生效值锁定。

7.3 结果与分析：是否需要引入其他时刻的预报

对五种策略（不调整 / 仅 6 时 / 仅 12 时 / 仅 18 时 / 全部调整）在同一回测期对比，结果见表 9 与图 12。

表 9 问题三各调整策略回测期总费用对比（2025.2.1–12.31）

策略	总费用/元	偏差费/元	紧急费/元	节省/元
不调整（A）	13,768,495	12,829,371	939,124	—
仅 6 时调整（B）	13,653,689	12,910,086	743,603	114,805
仅 12 时调整（C）	13,555,352	12,902,205	653,147	213,143
仅 18 时调整（D）	13,721,579	12,840,015	881,564	46,916
全部调整（E, 主策略）	13,475,997	12,903,967	572,030	292,498

结论：需要引入其他时刻的预报制定调整策略。全调整较不调整节省 **29.25 万元 (2.12%)**，紧急购电费由 93.91 万元降至 57.20 万元（降 39%）。调整价值按渠道分解：新预报信息是主要来源——12 时预报对午后时段精度最高（MAE 58.1 kW），单独贡献 21.31 万元，为各时点之最；储能状态反馈次之——18 时预报对当日已无信息价值（19 时后光伏为零），但以当日实际储电量重解晚间策略仍单独贡献 4.69 万元；6 时调整贡献 11.48 万元，受制于其覆盖时段的预测误差仍大。值得注意的是，三个单独调整的收益之和（37.50 万元）高于全调整（29.25 万元），因为 6 时与 12 时调整的覆盖时段部分重叠、收益不能简单叠加——这本身即滚动调整时序结构的体现。

与问题二主策略（13,879,046 元）相比，问题三主策略总费用降低 **403,049 元 (2.91%)**，其中均方误差倒数加权信息融合与 SAA 计划贡献约 11.0 万元、场景化滚动调整贡献 29.2 万元。偏差结算口径的敏感性：采用“少买不退款”口径（式 (12) 的下调部分改为全额支付计划价外加 50% 违约金）重跑主策略，总费用为 13,514,168 元，仅相差 0.28%，结论对结算口径稳健。

指定日期结果见表 10–12（购电量为最终生效值）。以 9.23 为例，该日午后多云导致光伏缺口，12 时调整使 15:00–17:10 的紧急购电大幅收窄，紧急购电费由问题二同日的 9,839 元降至 8,650 元；12.21（冬季光伏匮乏）经调整后无紧急购电。图 13 给出调整

总量最大日的计划与调整轨迹示例。图 11 进一步展示全年调整量的统计特征：日均调整总量约 1,500 kWh（占日购电量的 2.3%），其中约 87% 的时段未发生调整（计划即最优），调增与调减时段占比接近（6.6% vs 6.4%）——调整是精准的局部修正而非全面推翻计划。

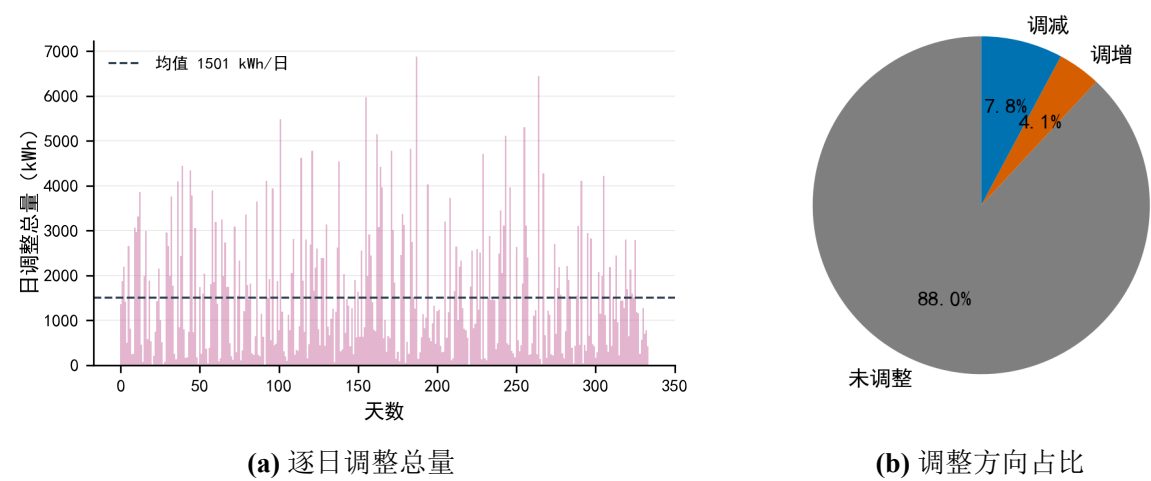


图 11 问题三全年调整量统计

表 10 问题三指定日期的（最终）购电量（按表 1 格式，单位 kWh）

时间段	2025.3.20	2025.6.21	2025.9.23	2025.12.21
10:00–10:10	0.00	0.00	0.00	0.00
12:00–12:10	518.61	0.00	427.96	853.99
14:00–14:10	0.00	0.00	0.00	0.00
16:00–16:10	468.48	135.62	415.10	824.35
18:00–18:10	724.07	419.39	778.75	703.46
20:00–20:10	0.00	0.00	13.61	0.00
全天购电量	65,703.84	36,778.88	65,693.29	95,701.28
全天偏差费/元	41,088.81	21,128.34	41,373.37	62,031.90
紧急费/元	4,130.53	0.00	5,120.30	0.00

表 11 问题三指定日期的储能充放电量与储电量（按表 2 格式，单位 kWh）

时间段	2025.3.20		2025.6.21		2025.9.23		2025.12.21	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00–4:00	310.22	147.08	149.18	3104.31	1208.42	216.70	1154.24	370.91
4:00–8:00	398.03	5791.67	187.61	5420.20	958.59	5529.58	1234.19	784.37
8:00–12:00	3114.92	3038.13	10134.27	0.00	4050.21	2387.94	4426.80	7868.26
12:00–16:00	6979.68	254.72	53.04	0.00	6260.69	403.62	8031.13	2222.66
16:00–20:00	637.86	4753.91	115.59	4750.65	69.29	4973.12	28.17	4962.37
20:00–24:00	9933.46	3746.65	9796.16	3277.87	8929.63	3723.01	9906.91	3659.04
储电量	10536.90 / 10071.24		10800.00 / 10800.00		9056.36 / 9236.67		9934.05 / 10162.23	

表 12 问题三指定日期的紧急购电量（按表 3 格式，单位 kWh）

日期	紧急购电时间段	紧急购电量
2025.3.20	0:20–0:50	114.03
	1:00–2:20	203.38
	2:40–3:10	23.44
	3:20–4:10	172.73
	9:20–10:10	227.42
	16:00–16:20	438.60
	17:20–17:40	29.99
2025.6.21	无紧急购电	
2025.9.23	15:00–15:20	575.63
	16:00–17:00	214.28
	17:50–18:00	7.36
	18:20–18:30	52.52
	18:40–19:00	47.26
	20:30–20:40	34.78
	21:00–22:00	299.78
2025.12.21	无紧急购电	

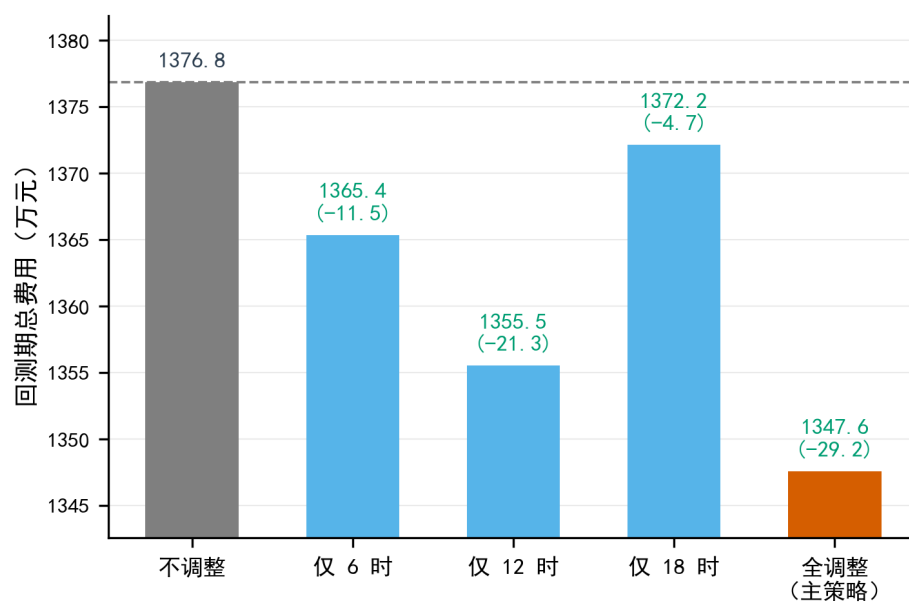
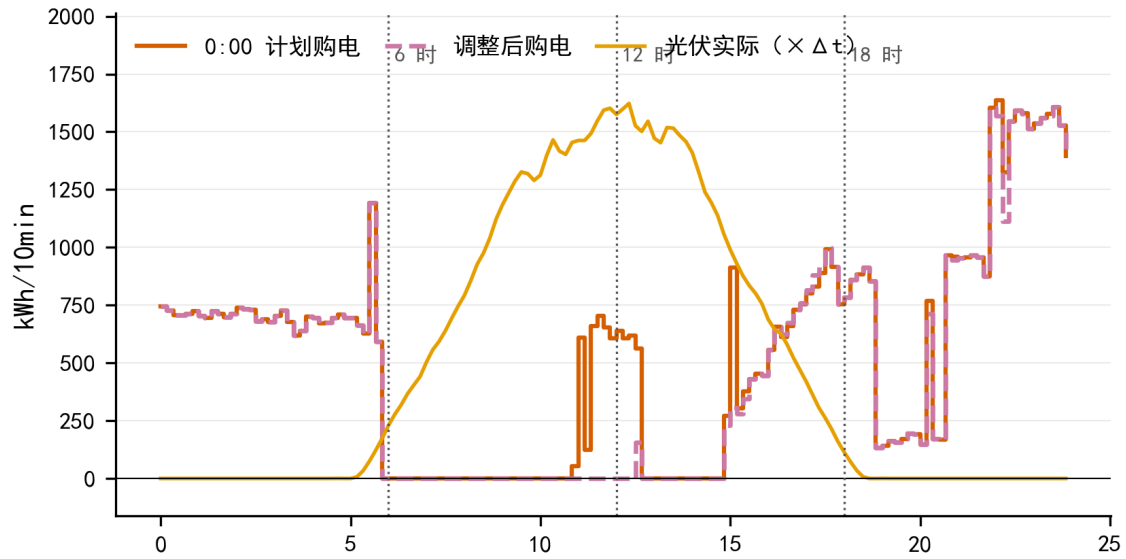
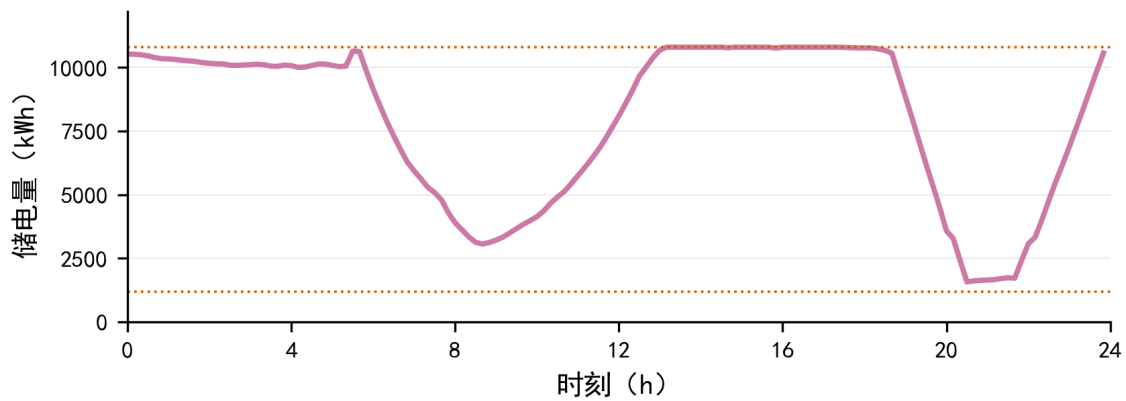


图 12 问题三：各调整策略总费用对比



(a) 0:00 计划购电、调整后购电与光伏实际出力



(b) 储电量轨迹

图 13 滚动调整示例（2025.10.23，调整总量最大日，竖虚线为三次预报时刻）

完整结果见支撑材料 `result3.xlsx`（334 天逐时段计划购电量、调整购电量、充放电量与紧急购电记录）。

8 问题四：波动电价下的购电模型

8.1 建模准备

附件 4 电价全年实时波动，范围 0.0076–1.7936 元/kWh（附件 1 固定电价范围为 0.3713–1.3952 元/kWh），峰谷价差显著拉大且出现近零电价时段。本文按日前已知处理当日电价路径，这是额外的信息假设：题目原文仅称“电价实时波动”并要求根据附件 4 重算，未明确 0:00 是否能获取全天电价路径。本文采用该假设的理由是：日前市场出清价于运营日前发布是电力市场常见惯例，且题目的结算规则均按“交易时刻电价”计价。在此信息结构下，问题二、三的全部机制保持不变，仅将电价序列替换为附件 4。若电价亦不可预知，需引入电价预测模型（如基于历史同期价格的加权估计），计划与调整

的 LP 将使用预测电价、结算使用实际电价，预计总费用将高于当前报告值。将附件 4 换回附件 1，模型即退化为问题二/三（同一代码路径），退化一致性检验通过。

8.2 结果与分析

固定电价与波动电价下各策略的回测期总费用对比见表 13 与图 14。

表 13 固定电价（附件 1）与波动电价（附件 4）下各策略总费用对比（万元）

策略	固定电价	波动电价	变化幅度
无储能基准	1640.7	1702.7	+3.8%
问题二模型 (P4-2)	1387.9	1460.8	+5.3%
问题三模型（不调整）	1376.8	1446.9	+5.1%
问题三模型（全调整，P4-3 主策略）	1347.6	1414.5	+5.0%
理想先知	1224.5	1283.0	+4.8%

波动电价使各策略总费用普遍上升 4%–5%，但两点结构性观察表明模型对电价体制稳健：

（1）储能套利空间扩大。附件 4 出现 0.0076 元/kWh 的近零电价时段（不足固定电价低谷 0.43 元/kWh 的 2%），主策略在这些时段近乎免费充电、并在 1.79 元/kWh 的高价时段深度放电。波动电价下主策略较无储能基准节省 **288.13 万元（16.9%）**，与固定电价下的降幅（16.9%）持平。

（2）调整价值放大。波动电价下场景化滚动调整较不调整节省 **32.41 万元**（固定电价下为 29.25 万元）。偏差结算按交易时刻电价计价，低价时段调减（按 50% 退款）与高价时段规避紧急购电（5 倍价）的价值均随价格波动增大；同时信息融合的贡献亦由 14.0 万元升至 17.2 万元。

计划层场景数沿用 $S = 10$ ：波动电价下 $S \in \{10, 14\}$ 的总费用为 1463.6 / 1471.4 万元，与固定电价下的选择一致，超参数稳健。指定日期的汇总结果见表 14（逐时段明细见支撑材料 result4-2.xlsx 与 result4-3.xlsx）。

表 14 问题四（P4-3 主策略）指定日期结果汇总

日期	全天购电量/kWh	偏差费/元	紧急费/元	调整总量/kWh
2025.3.20	66,126.86	41,734.46	4,047.20	3,135.41
2025.6.21	37,175.15	18,491.53	0.00	315.74
2025.9.23	65,856.06	42,828.76	5,584.16	1,397.45
2025.12.21	95,582.93	72,584.42	0.00	1,699.05

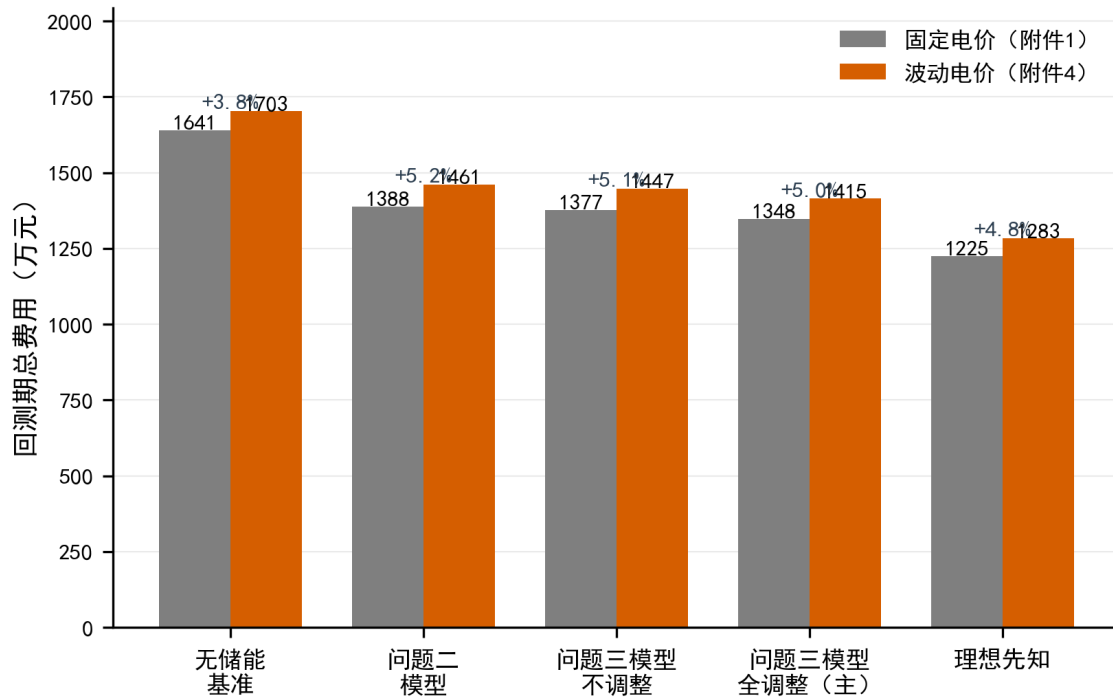


图 14 固定电价与波动电价下各策略总费用对比

9 灵敏度分析

为检验模型与结论对关键参数的稳健性，围绕储能效率、荷电状态安全区间与紧急购电惩罚倍数开展敏感性分析。全部灵敏度实验均使用与主策略完全一致的管线（加权混合 v2 预测 + 两阶段 SAA + 滚动执行），而非简化代理模型。

（1）充放电效率。效率 η 在 0.85 ~ 0.95 内变动时，问题一购电费由 36603.21 元单调降至 33767.02 元（ $\eta=0.9$ 时 35126.85 元）；问题二 SAA 回测期总费用由 **1442.61 万元**（ $\eta=0.85$ ）降至 **1338.31 万元**（ $\eta=0.95$ ），基线 1387.90 万元，变化幅度约 $\pm 3.6\% \sim 3.9\%$ 。效率只影响套利空间大小，不改变“低谷充电、高峰放电”的最优策略结构。

（2）荷电状态安全区间。问题二 SAA 总费用：下限降至 960（升至 1440）时变化 $-0.4\%/+0.5\%$ ；上限由 **10800 降至 8640** 时上升 **4.8%**，升至 12000 时下降 1.5%。上限（可用储能容量）是经济性的主导参数——在当前峰谷价差下，扩充储能上限仍有正边际价值，但递减。

（3）紧急购电惩罚倍数。在 SAA 计划固定（按 $5\times$ 优化）的前提下，仅改变结算倍数 m ： m 从 3 增至 8 时，总费用由 **1346.77 万元** 线性升至 **1449.61 万元**（计划费不变 1285.06 万元，紧急费从 61.70 万线性增至 164.54 万）。紧急费与 m 严格成比例（计划不变时紧急电量恒定 $205,681 \text{ kWh} \times m/5$ ），说明 SAA 的 $5\times$ 惩罚已充分将紧急购电量压至最低——若实际惩罚更低则节省、更高则多付，但计划本身无需因 m 变化而调整。

（4）紧急购电用于充电的政策敏感性。本文执行层 LP 允许紧急购电变量同时满足负载缺口与储能充电（题目未明确禁止）。若禁止紧急购电用于充电（缺口时只放电、不够就紧急但不充电），总费用升至 **1403.21 万元**（+15.31 万元，+1.1%）——即紧急充电

渠道每年节省约 15.3 万元。

(5) 问题三调整时点。本文在 $T+1$ 时（首个可调槽）以实际 SOC 触发调整 LP。若改为在 T 时以预测 SOC 提前触发，总费用为 1350.41 万元（+2.81 万元，+0.2%）——用 $T+1$ 实际 SOC 优于 T 时预测 SOC，当前做法更优。

(6) 0:00/0:10 状态交接偏移。本文时间约定将 [0:00, 0:10) 归前一日计划覆盖，导致日初 SOC 在 0:00 与 0:10 之间存在偏移（均值 441 kWh、标准差 299 kWh）。该偏移的年费用影响上界约 22.9 万元（1.6%），已在模型假设中声明。

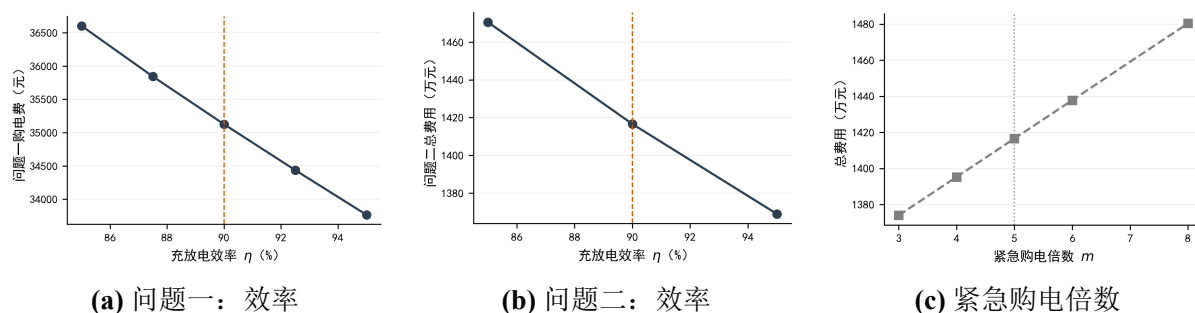


图 15 灵敏度分析

10 模型评价与推广

优点：

1. **统一的多层随机优化框架**：计划层与调整层均为两阶段随机规划（场景法），执行层为滚动实时线性规划——风险刻画全链条一致，且实验证明了视角一致性对调整价值的决定性作用；
2. **全局最优保证**：各层均为线性规划，HiGHS 求解器求得全局最优——九组替代策略对照实验（含 CVaR-SAA 与全时域 MPC）进一步验证了期望最优框架的正确性；
3. **严格因果**：预测只用历史数据、场景只用滞后误差、仿真内置信息集约束——杜绝数据泄漏；
4. **理论支撑完备**：报童分位数缓冲是 SAA 在无储能单时段下的特例（理论 $q^* = 0.8$ 与实测最优区间 $[0.7, 0.8]$ 一致）；对偶分析给出边际供电成本与储能容量影子价格；
5. **信息融合增益**：官方预报与历史均值的按均方误差倒数加权组合将光伏预测 MAE 降低 35%（197.5 $\rightarrow$ 127.8 kW）；
6. **完全可复现**：全部结果由代码自动校验（平衡残差、SOC 区间、循环条件）并落盘为单一数据源，九组对照实验的代码全部入库。

缺点：

1. SAA 场景集取近 10 天误差曲线的等概率经验分布，未刻画误差的时间自相关与条件结构；

2. 执行层滚动优化沿用 0:00 的负载预测，未利用日内已发生的实际值修正（实验证明自适应修正反而恶化）；
3. 问题四假设电价日前已知；若电价亦需实时预测，需引入价格预测模型；
4. 预测器未利用气象协变量（题目未提供），多云天气下的光伏突变预测能力有限。

推广：本框架可直接推广至含风电机组的微网、多微网共享储能的分布式调度（增加耦合约束即可保持线性）；将经验场景集升级为条件场景生成或分布鲁棒优化，可进一步刻画误差的结构性依赖；对偶分析给出的储能容量影子价格可直接用于容量投资决策。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模思路讨论、代码编写与调试、数据分析与论文初稿撰写，详细使用情况见支撑材料。

参考文献

- [1] 姜启源, 谢金星, 叶俊. 数学模型[M]. 3 版. 北京: 高等教育出版社, 2003.
- [2] VIRTANEN P, GOMMERS R, OLIPHANT T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17: 261-272.
- [3] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119-142.

附录 A 支撑材料文件列表

序号	文件路径	内容说明
1	code/common.py	公共模块：数据加载、时间约定、LP 求解器
2	code/q1_plan_lp.py	问题一：日循环线性规划求解与 result1 输出
3	code/q2_forecast.py	问题二：负载/光伏预测器对比
4	code/q2_simulate.py	问题二：全年计划-执行仿真与策略对比
5	code/q2_results.py	问题二：result2.xlsx 输出与图表
6	code/q3_simulate.py	问题三：滚动调整仿真与策略对比
7	code/q3_results.py	问题三：result3.xlsx 输出与图表
8	code/q4_simulate.py	问题四：波动电价下的重算
9	code/q4_results.py	问题四：result4-2/4-3.xlsx 输出与图表
10	code/xlsx_writer.py	结果工作簿写出公共模块
11	code/sens_analysis.py	全局灵敏度分析
12	code/q2_saa.py	问题二两阶段随机规划（SAA）
13	code/figstyle.py、 code/restyle_figures.py	图表统一风格与生成
14	code/q2_variants.py、	对照实验：场景选择、分位数变体池、信息价值分解
15	code/q2_cvar.py、 code/q2_mpc.py	证伪实验：CVaR-SAA 网格、全时域 MPC 执行
16	code/q2_fc2_test.py、 code/q2_final.py	证伪实验：预测器变体、加权场景池
17	code/fig_new.py	新增图表：EDA 热力图/箱线/分布、模型验证/全年轨迹/月度分解
18	results/result1.xlsx result4-3.xlsx	~ 四问完整结果文件

附录 B 完整源程序

Listing 1 公共模块 common.py

```

1 # -*- coding: utf-8 -*-
2 """
3 2026 CUMCM C题 公共模块：数据加载、时间约定、LP 求解、执行仿真。

```

```

4
5 时间约定（已验证，详见 .skill/data-preprocessing 与 .skill/microgrid-modeling）：
6 - 附件1/2/4 的 10 分钟数据行标签 t = 时段 [t, t+10min) 起点；
7   144 个标签 0:10,...,23:50,0:00+1 构成"计划日"窗口 [0:10, 次日0:10)。
8 - 全局槽号 m = d*144 + j: d=距2025-01-01天数, j=钟表日槽号(0..143, 槽j=[j*10,(j+1)*10))。
9   附件文件行 d 列 k(k=0..143) 对应全局槽 d*144+k+1: 全局槽0([1月1日0:00,0:10))无数据（过渡期）。
10 - 附件3: 发布时刻T的"预报k小时" = T+k 整点的瞬时功率(kW)。
11 """
12 import json
13 import sys
14 import io
15 from pathlib import Path
16
17 import numpy as np
18 import pandas as pd
19
20 if sys.stdout.encoding != "utf-8":
21     sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")
22
23 # ----- 路径
24 ROOT = Path(__file__).resolve().parents[2]      # .../CUMCM
25 ATTACH = ROOT / "C题" / "附件"
26 CODE = Path(__file__).resolve().parent        # .../overleaf-paper/code
27 RESULTS = CODE.parent / "results"
28 FIGURES = CODE.parent / "figures"
29 CACHE = CODE / "cache"
30 RESULTS.mkdir(exist_ok=True)
31 FIGURES.mkdir(exist_ok=True)
32 CACHE.mkdir(exist_ok=True)
33
34 # ----- 常量
35 N = 144          # 每日时段数（10 分钟）
36 DT = 10.0 / 60.0 # 时段长度（h）
37 P_MAX = 5000.0   # 最大充/放电功率（kW）
38 E_MAX = P_MAX * DT # 单时段最大充/放电量（kWh）= 833.33
39 SOC_MIN, SOC_MAX = 1200.0, 10800.0
40 ETA = 0.9        # 充放电效率（双向）
41 SOC_INIT = 6000.0 # 2025-01-01 0:00（kWh）
42 EMERG_MULT = 5.0  # 紧急购电价格倍数
43 UP_MULT = 1.5     # 调整增购部分的电价倍数
44 DOWN_REFUND = 0.5 # 调整减购部分的退还比例（违约电价 = 50%）
45 DAYS = 365
46 ISSUE_HOURS = [0, 6, 12, 18]
47 SEED = 2026
48
49
50 # ----- 数据加载
51 def _label_to_minute(x):
52     """列标签 → 距 0:00 的分钟数（0:00+1 → 1440）。"""
53     if isinstance(x, str):
54         if "+" in x:
55             return 1440
56         h, m = x.split(":")
57         return int(h) * 60 + int(m)
58     if hasattr(x, "hour"): # datetime.time
59         return x.hour * 60 + x.minute
60     raise ValueError(f"未知标签: {x!r}")

```

```

61
62
63 def _load_wide(path, sheet):
64     df = pd.read_excel(path, sheet_name=sheet)
65     dates = pd.to_datetime(df.iloc[:, 0])
66     assert (dates.diff().dropna() == pd.Timedelta(days=1)).all(), "日期不连续"
67     cols = [_label_to_minute(c) for c in df.columns[1:]]
68     assert cols == [(k + 1) * 10 for k in range(N)], f"列标签异常: {cols[:3]}..."
69     return df.iloc[:, 1:].to_numpy(dtype=float)
70
71
72 def load_all(force=False):
73     """返回 (a1, load, pv, price4, fc3), 带 parquet 缓存。
74     a1: dict(price/load/pv 各(144,), 文件列序 0:10..0:00+1)
75     load/pv/price4: (365,144) 文件列序
76     fc3: (365,4,24), fc3[d,i,k] = 第d天 ISSUE_HOURS[i] 发布的 预报(k+1)小时
77     """
78     c = CACHE
79     if force or not (c / "load.parquet").exists():
80         a1df = pd.read_excel(ATTACH / "附件1.xlsx")
81         a1 = {
82             "price": a1df.iloc[:, 1].to_numpy(dtype=float),
83             "load": a1df.iloc[:, 2].to_numpy(dtype=float),
84             "pv": a1df.iloc[:, 3].to_numpy(dtype=float),
85         }
86         load = _load_wide(ATTACH / "附件2.xlsx", "小区负载")
87         pv = _load_wide(ATTACH / "附件2.xlsx", "光伏发电实际功率")
88         price4 = _load_wide(ATTACH / "附件4.xlsx", "Sheet1")
89         a3 = pd.read_excel(ATTACH / "附件3.xlsx")
90         a3["日期"] = a3["日期"].ffill()
91         fc3 = np.zeros((DAYS, 4, 24))
92         for r in range(len(a3)):
93             d = (pd.to_datetime(a3.iloc[r, 0]) - pd.Timestamp("2025-01-01")).days
94             ih = int(str(a3.iloc[r, 1]).split(":")[0]) // 6 # 0/6/12/18 -> 0/1/2/3
95             fc3[d, ih, :] = a3.iloc[r, 2:26].to_numpy(dtype=float)
96         np.save(c / "a1.npy", a1, allow_pickle=True)
97         for name, arr in [("load", load), ("pv", pv), ("price4", price4), ("fc3", fc3)]:
98             np.save(c / f"{name}.npy", arr)
99     else:
100         a1 = np.load(c / "a1.npy", allow_pickle=True).item()
101         load = np.load(c / "load.npy")
102         pv = np.load(c / "pv.npy")
103         price4 = np.load(c / "price4.npy")
104         fc3 = np.load(c / "fc3.npy")
105     return a1, load, pv, price4, fc3
106
107
108 # ----- 时间工具
109 def clock_slice(day, j):
110     """钟表日 day 的槽 j (0..143) 对应 (文件行, 文件列): j=0 借用前一日末列。
111     返回 (row, col), row=-1 表示全局槽0 (无数据)。"""
112     if j == 0:
113         return (day - 1, N - 1) if day >= 1 else (-1, -1)
114     return day, j - 1
115
116
117 def clock_day_matrix(file_arr, day):

```

```

118     """把文件列序数组 (365,144) 取钟表日 day 的 144 槽 (clock j=0..143)。"""
119     out = np.empty(N)
120     r, c = clock_slice(day, 0)
121     out[0] = file_arr[r, c] if r >= 0 else np.nan
122     out[1:] = file_arr[day, :N - 1]
123     return out
124
125
126 def to_file_order(clock_vec):
127     """钟表日向量 (j=0..143) → 计划日文件列序 (0:10..0:00+1)。
128     计划日第 k 列 = 时段 [(k+1)*10, (k+2)*10) = 钟表槽 k+1; k=143 → 次日槽0 (周期回滚)。"""
129     return np.concatenate([clock_vec[1:], clock_vec[:1]])
130
131
132 def from_file_order(file_vec):
133     """计划日文件列序 → 钟表日向量。"""
134     return np.concatenate([file_vec[-1:], file_vec[:-1]])
135
136
137 def fc_to_slots(fc24, issue_hour):
138     """附件3 一次预报 (点值 = T+1h..T+24h 整点瞬时功率) → 10 分钟槽值。
139     对整点值线性插值, 取槽起点处的值; 返回 dict: 标签分钟(相对当日0:00, >1440 表示次日) → kW。
140     覆盖标签范围 [T+60, T+1440], 其中标签 T+1440 恰为末整点。"""
141     T = issue_hour * 60
142     pts = {T + (k + 1) * 60: float(fc24[k]) for k in range(24)}
143     out = {}
144     for m in range(T + 60, T + 1441, 10):
145         if m >= T + 1440:
146             out[m] = pts[T + 1440]
147             continue
148         lo = ((m - T) // 60) * 60 + T
149         hi = lo + 60
150         w = (m - lo) / 60.0
151         out[m] = pts[lo] * (1 - w) + pts[hi] * w
152     return out
153
154
155 def pv_profile_day(fc24, issue_hour):
156     """计划日 144 槽 (文件列序, 标签10..1440) 的光伏预测剖面。
157     issue=0: 标签 60..1430 由预报插值, 标签 10..50 与 1440 为夜间置 0 (全天覆盖)。
158     issue>0: 仅填标签 [T+60, 1440] 段, 其余为 NaN (调用方沿用旧估计)。"""
159     d = fc_to_slots(fc24, issue_hour)
160     prof = np.full(N, np.nan)
161     if issue_hour == 0:
162         prof[:] = 0.0
163         for m, v in d.items():
164             if m <= 1430:
165                 prof[m // 10 - 1] = v
166     else:
167         for m, v in d.items():
168             if m <= 1440:
169                 prof[m // 10 - 1] = v
170     return prof
171
172
173 # ----- 计划层 LP
174 def solve_plan(price, load_kw, pv_kw, soc0, cyclic=True, g_lock=None, validate=True):

```

```

175     """日计划 LP (144 槽, 向量顺序由调用方定义: P1 用钟表日序, P2-P4 用计划日文件列序)。
176
177     price: (144,) 元/kWh; load_kw/pv_kw: (144,) kW
178     soc0: 起始 SOC; cyclic: 末槽 SOC = soc0
179     g_lock: 若给定 (144,), 则购电量固定, 只优化充放/弃/紧急 (滚动执行用)
180     返回 dict(g,c,d,spill,emg,s,cost_plan,cost_emg,status)
181     """
182
183     from scipy.optimize import linprog
184     from scipy.sparse import lil_matrix
185
186     load_e = np.asarray(load_kw, float) * DT
187     pv_e = np.asarray(pv_kw, float) * DT
188     # 变量: g,c,d,spill,emg,s 各144
189     nv = 6 * N
190     G, C, D, SP, EM, S = (slice(i * N, (i + 1) * N) for i in range(6))
191     fixed_g = g_lock is not None
192     if fixed_g:
193         g_lock = np.asarray(g_lock, float)
194
195     # 目标: min sum p*g + 5*p*emg (emg 仅在 g_lock 模式有意义; 计划模式 g 自由时 emg 恒 0)
196     obj = np.zeros(nv)
197     obj[G] = price
198     obj[EM] = EMERG_MULT * price
199
200     # 等式约束: SOC 平衡 + 日循环 + 供需平衡
201     # SOC 平衡:  $s_t - s_{t-1} - 0.9 c_t + d_t / 0.9 = 0$ ;  $t=0$ : = soc0
202     A_eq = lil_matrix((2 * N + 1, nv))
203     b_eq = np.zeros(2 * N + 1)
204     for t in range(N):
205         A_eq[t, S.start + t] = 1.0
206         if t > 0:
207             A_eq[t, S.start + t - 1] = -1.0
208             A_eq[t, C.start + t] = -ETA
209             A_eq[t, D.start + t] = 1.0 / ETA
210         if t == 0:
211             b_eq[t] = soc0
212     # 日循环:  $s_{143} = soc0$ 
213     if cyclic:
214         A_eq[N, S.start + N - 1] = 1.0
215         b_eq[N] = soc0
216     # 平衡:  $g + pv + d + emg = load + c + spill \rightarrow g + d + emg - c - spill = load - pv$ 
217     for t in range(N):
218         r = N + 1 + t
219         A_eq[r, G.start + t] = 1.0
220         A_eq[r, D.start + t] = 1.0
221         A_eq[r, EM.start + t] = 1.0
222         A_eq[r, C.start + t] = -1.0
223         A_eq[r, SP.start + t] = -1.0
224         b_eq[r] = load_e[t] - pv_e[t]
225
226     bounds = [(0, None)] * nv
227     for t in range(N):
228         bounds[C.start + t] = (0, E_MAX)
229         bounds[D.start + t] = (0, E_MAX)
230         bounds[S.start + t] = (SOC_MIN, SOC_MAX)
231     if fixed_g:
232         v = g_lock[t]

```

```

232     bounds[G.start + t] = (v, v)
233
234     res = linprog(obj, A_eq=A_eq.tocsr(), b_eq=b_eq, bounds=bounds, method="highs")
235     if res.status != 0:
236         return {"status": res.status, "message": res.message}
237     x = res.x
238     out = {
239         "g": x[G], "c": x[C], "d": x[D], "spill": x[SP], "emg": x[EM], "s": x[S],
240         "cost_plan": float(price @ x[G]), "cost_emg": float(price @ x[EM] * EMERG_MULT),
241         "status": 0, "res": res,
242     }
243     # 校验
244     if validate:
245         s = out["s"]
246         assert s.min() >= SOC_MIN - 1e-6 and s.max() <= SOC_MAX + 1e-6, "SOC 越界"
247         assert np.abs(s[-1] - (soc0 if cyclic else s[-1])) < 1e-4
248         soc = soc0
249         for t in range(N):
250             soc = soc + ETA * out["c"][t] - out["d"][t] / ETA
251             assert abs(soc - s[t]) < 1e-4, "SOC 递推不一致"
252     return out
253
254
255 # ----- 结果/日志
256 def key_numbers(**kw):
257     """追加/更新关键数值到 results/key_numbers.json (论文单一数据源)。"""
258     p = RESULTS / "key_numbers.json"
259     data = json.loads(p.read_text(encoding="utf-8")) if p.exists() else {}
260     data.update(kw)
261     p.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
262
263
264 def log(msg):
265     print(f"[{pd.Timestamp.now():%H:%M:%S}] {msg}")
266
267
268 if __name__ == "__main__":
269     a1, load, pv, price4, fc3 = load_all(force=True)
270     log(f"数据加载完成: load{load.shape} pv{pv.shape} price4{price4.shape} fc3{fc3.shape}")
271     log(f"附件1 电价范围 [{a1['price'].min():.4f},{a1['price'].max():.4f}]")
272     log(f"附件4 电价范围 [{price4.min():.4f},{price4.max():.4f}]")
273     log(f"年负载 {np.nansum(load)/6/1e4:.1f}万kWh, 年光伏 {np.nansum(pv)/6/1e4:.1f}万kWh")

```

Listing 2 问题一求解程序 q1_plan_lp.py

```

1  # -*- coding: utf-8 -*-
2  """问题1: 电价与负载逐日相同 (附件1典型日) + 光伏预测已知 → 确定性日循环 LP。
3
4  模型 (时钟日 144 槽 j=0..143, 槽 j = [j*10,(j+1)*10] 分钟):
5      min  Σ p_j g_j
6      s.t. s_j = s_{j-1} + 0.9 c_j - d_j/0.9          (SOC 平衡, s_{-1}=6000)
7           1200 s_j 10800, 0 c_j,d_j 833.33
8           g_j + pv_j * Δt + d_j = load_j * Δt + c_j + spill_j  (供需平衡, 弃电 spill 0)
9           s_143 = 6000                                         (0:00 与 24:00 储电量相同)
10 附件1 文件列序 (0:10..0:00+1) → 时钟日: clock = roll(file, 1);
11 结果文件按计划日文件列序填写: file = roll(clock, -1)。
12 """

```

```

13 import sys, json
14 import numpy as np
15 import pandas as pd
16
17 sys.path.insert(0, str(__import__("pathlib").Path(__file__).parent))
18 from common import (load_all, solve_plan, to_file_order, key_numbers, log,
19                     N, DT, SOC_INIT, RESULTS, FIGURES)
20
21 a1, load, pv, price4, fc3 = load_all()
22
23 # ---- 时钟日数据 (典型日, 每一天相同; 槽 j 的数据 = 文件列 (j-1)%144) ----
24 price_c = np.roll(a1["price"], 1)
25 load_c = np.roll(a1["load"], 1)
26 pv_c = np.roll(a1["pv"], 1)
27
28 sol = solve_plan(price_c, load_c, pv_c, soc0=SOC_INIT, cyclic=True)
29 assert sol["status"] == 0
30 g, c, d, s, spill = sol["g"], sol["c"], sol["d"], sol["s"], sol["spill"]
31 cost, total_g = sol["cost_plan"], float(g.sum())
32 log(f"P1 求解成功: 全天购电量 {total_g:.2f} kWh, 购电费 {cost:.2f} 元")
33
34 # ---- 无储能基准 ----
35 base_g = np.maximum(load_c * DT - pv_c * DT, 0.0)
36 base_cost = float(price_c @ base_g)
37 base_total = float(base_g.sum())
38 save_abs, save_pct = base_cost - cost, (base_cost - cost) / base_cost * 100
39 log(f"无储能基准: 购电量 {base_total:.2f} kWh, 购电费 {base_cost:.2f} 元; "
40     f"储能节省 {save_abs:.2f} 元 ({save_pct:.2f}%)")
41
42 # ---- 储能日循环电量水平灵敏度 (s0 自由时应选哪个水平) ----
43 levels = np.arange(1200, 10801, 600)
44 level_cost = []
45 for lv in levels:
46     r = solve_plan(price_c, load_c, pv_c, soc0=float(lv), cyclic=True)
47     level_cost.append(r["cost_plan"] if r["status"] == 0 else np.nan)
48 level_cost = np.array(level_cost)
49 best_i = int(np.nanargmin(level_cost))
50 log(f"灵敏度: 最优日循环水平 {levels[best_i]} kWh, 费用 {level_cost[best_i]:.2f} 元 "
51     f"(6000kWh 时 {cost:.2f} 元, 差 {cost-level_cost[best_i]:.2f} 元)")
52
53 # ---- 表1: 指定时段购电量 ----
54 t1_idx = [60, 72, 84, 96, 108, 120] # 10:00,12:00,...,20:00 槽
55 t1_labels = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
56             "16:00-16:10", "18:00-18:10", "20:00-20:10"]
57 t1_vals = [float(g[j]) for j in t1_idx]
58
59 # ---- 表2: 4小时块充放电 + 0/24 点储电量 ----
60 blocks = [("0:00-4:00", 0, 24), ("4:00-8:00", 24, 48), ("8:00-12:00", 48, 72),
61          ("12:00-16:00", 72, 96), ("16:00-20:00", 96, 120), ("20:00-24:00", 120, 144)]
62 t2 = [(lab, float(c[a:b].sum()), float(d[a:b].sum())) for lab, a, b in blocks]
63
64 # ---- 写 result1.xlsx (保持模板结构) ----
65 from openpyxl import load_workbook
66 tpl = r"C:\Users\26517\Documents\CUMCM\C题\附件\附件5\result1.xlsx"
67 wb = load_workbook(tpl)
68 ws = wb["计划购电量"]
69 g_file = to_file_order(g) # 计划日文件列序

```

```

70 assert ws.max_row == N + 1, f"模板行数 {ws.max_row} != {N+1}"
71 assert str(ws.cell(row=2, column=1).value).startswith("0:10")
72 assert str(ws.cell(row=N + 1, column=1).value).startswith("0:00+1")
73 for i in range(N):
74     ws.cell(row=i + 2, column=2, value=round(float(g_file[i]), 4))
75 ws2 = wb["充放电电量"]
76 for i, (lab, cc, dd) in enumerate(t2):
77     ws2.cell(row=i + 2, column=2, value=round(cc, 4))
78     ws2.cell(row=i + 2, column=3, value=round(dd, 4))
79 ws2.cell(row=2, column=5, value=round(SOC_INIT, 4))      # 0:00 储电量
80 ws2.cell(row=3, column=5, value=round(float(s[-1]), 4)) # 24:00 储电量
81 out_path = RESULTS / "result1.xlsx"
82 wb.save(out_path)
83 log(f"已写出 {out_path}")
84
85 # ---- 图1: 策略图 (电价+购电/充放电 + SOC) ----
86 import matplotlib
87 matplotlib.use("Agg")
88 import matplotlib.pyplot as plt
89 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
90 plt.rcParams["axes.unicode_minus"] = False
91
92 tmin = np.arange(N) * 10 / 60.0 # 小时
93 fig, axes = plt.subplots(2, 1, figsize=(9.5, 6.4), sharex=True,
94                        gridspec_kw={"height_ratios": [3, 1.6]})
95 ax = axes[0]
96 ax.plot(tmin, price_c, color="#888888", lw=1.4, label="电价 (右轴)")
97 ax.set_ylabel("电价 (元/kWh)", color="#888888")
98 ax.tick_params(axis="y", labelcolor="#888888")
99 ax2 = ax.twinx()
100 ax2.step(tmin, g, where="post", color="#c0392b", lw=1.6, label="购电量")
101 ax2.bar(tmin, c, width=10/60, align="edge", color="#2471a3", alpha=0.55, label="充电量")
102 ax2.bar(tmin, -d, width=10/60, align="edge", color="#1e8449", alpha=0.55, label="放电量")
103 ax2.set_ylabel("电量 (kWh / 10min)")
104 ax2.axhline(0, color="k", lw=0.6)
105 h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
106 ax.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=9, ncol=4, framealpha=0.9)
107 ax.set_title("问题一: 最优购电与储能充放电策略 (典型日)")
108 axs = axes[1]
109 axs.plot(tmin, s, color="#6c3483", lw=1.8)
110 axs.axhline(1200, color="r", ls="--", lw=0.8); axs.axhline(10800, color="r", ls="--", lw=0.8)
111 axs.set_ylabel("储电量 (kWh)"); axs.set_xlabel("时刻 (h)")
112 axs.set_ylim(0, 12000)
113 fig.tight_layout()
114 fig.savefig(FIGURES / "fig_q1_strategy.png", dpi=300)
115 plt.close(fig)
116 log("已输出 figures/fig_q1_strategy.png")
117
118 # ---- 关键数值入库 ----
119 key_numbers(
120     q1_total_purchase=round(total_g, 2), q1_total_cost=round(cost, 2),
121     q1_baseline_cost=round(base_cost, 2), q1_save_abs=round(save_abs, 2),
122     q1_save_pct=round(save_pct, 2),
123     q1_table1={l: round(v, 2) for l, v in zip(t1_labels, t1_vals)},
124     q1_table2={lab: [round(cc, 2), round(dd, 2)] for lab, cc, dd in t2},
125     q1_soc_0=round(SOC_INIT, 2), q1_soc_24=round(float(s[-1]), 2),
126     q1_best_level=float(levels[best_i]),

```

```

127     q1_best_level_cost=round(float(level_cost[best_i]), 2),
128     q1_spill_total=round(float(spill.sum()), 2),
129     q1_charge_total=round(float(c.sum()), 2), q1_discharge_total=round(float(d.sum()), 2),
130 )
131 log("关键数值已写入 key_numbers.json")
132
133 # 控制台摘要
134 print("\n===== 表1 微网在指定时间段的购电量 =====")
135 for l, v in zip(t1_labels, t1_vals):
136     print(f" {l}: {v:.2f} kWh")
137 print(f" 全天购电量: {total_g:.2f} kWh, 全天购电费: {cost:.2f} 元")
138 print("===== 表2 储能设备充放电量 =====")
139 for lab, cc, dd in t2:
140     print(f" {lab}: 充 {cc:.2f}, 放 {dd:.2f} kWh")
141 print(f" 0:00 储电量 {SOC_INIT:.0f}, 24:00 储电量 {s[-1]:.2f} kWh")
142 print(f" 弃电量 {spill.sum():.2f} kWh; 无储能基准费 {base_cost:.2f} 元")

```

Listing 3 问题二预测程序 q2_forecast.py

```

1  # -*- coding: utf-8 -*-
2  """问题2 预测模块：日前负载/光伏预测器（仅用历史信息）+ 净负荷误差分位数缓冲。
3
4  预测器候选（walk-forward, 严格因果）：
5      A persistence      前一日曲线
6      B mean7            近 7 日均值
7      C weekday4         同星期几近 4 周均值
8      D ewmix            指数加权混合：近期（=7d）+ 同星期（=21d），负载 0.70/0.30、光伏 0.65/0.35
9  冷启动（1月1日）：用附件1典型日曲线作先验。
10 输出：work 缓存 npz (load_fc/pv_fc, 365×144, 文件列序) + 预测误差统计。
11 """
12 import sys, io
13 from pathlib import Path
14 sys.path.insert(0, str(Path(__file__).parent))
15 import numpy as np
16 import pandas as pd
17 from common import load_all, DAYS, N, CODE, log
18
19 W_LOAD, W_PV = 0.70, 0.65      # 同星期分量权重
20 TAU_RECENT, TAU_SAME = 7.0, 21.0
21
22
23 def _ew_curves(history, weights):
24     """history: (n_days,144); weights: (n_days,) — 返回加权平均曲线 (144,)"""
25     w = weights / weights.sum() if weights.sum() > 0 else None
26     return (history * w[:, None]).sum(axis=0) if w is not None else history.mean(axis=0)
27
28
29 def forecast_day(method, load_hist, pv_hist, d, prior_load, prior_pv):
30     """返回 (load_fc, pv_fc) 各(144,)，文件列序。history 为 0..d-1 天的 (d,144)。"""
31     if d == 0:
32         return prior_load.copy(), prior_pv.copy()
33     lh, ph = load_hist[:d], pv_hist[:d]
34     if method == "persistence":
35         return lh[-1].copy(), ph[-1].copy()
36     if method == "mean7":
37         lo = lh[-min(7, d):].mean(axis=0)
38         pv = ph[-min(7, d):].mean(axis=0)

```

```

39     return lo, pv
40 if method == "weekday4":
41     idx = np.array([i for i in range(d) if i % 7 == d % 7][-4:], dtype=int)
42     if len(idx) == 0: # 前 7 天无同星期历史 → 近期均值回退
43         k = min(4, d)
44         return lh[-k:].mean(axis=0), ph[-k:].mean(axis=0)
45     return lh[idx].mean(axis=0), ph[idx].mean(axis=0)
46 if method == "hybrid": # 主预测器: 负载=同星期近3周加权(3:2:1), 光伏=近7日加权
47     ix = [j for j in range(d - 7, d - 22, -7) if j >= 0] # d-7,d-14,d-21
48     if ix:
49         w = np.arange(len(ix), 0, -1, dtype=float)
50         lo = w @ lh[ix] / w.sum()
51     else:
52         lo = lh[-min(3, d):].mean(axis=0)
53     k = min(7, d)
54     wgt = np.arange(1, k + 1, dtype=float)
55     pvf = wgt @ ph[d - k:d] / wgt.sum()
56     return lo, pvf
57 if method == "hybrid1": # 旧版主预测器(简单均值), 保留作对照
58     idx = np.array([i for i in range(d) if i % 7 == d % 7][-4:], dtype=int)
59     lo = lh[idx].mean(axis=0) if len(idx) else lh[-min(4, d):].mean(axis=0)
60     pvf = ph[-min(7, d):].mean(axis=0)
61     return lo, pvf
62 if method == "ewmix":
63     ages = np.arange(d - 1, -1, -1) # 最近为 0
64     w_recent = np.exp(-ages / TAU_RECENT)
65     same = np.array([i for i in range(d) if i % 7 == d % 7])
66     rec_l = _ew_curves(lh, w_recent); rec_p = _ew_curves(ph, w_recent)
67     if len(same) > 0:
68         w_same = np.exp(-(d - 1 - same) / TAU_SAME)
69         sam_l = _ew_curves(lh[same], w_same); sam_p = _ew_curves(ph[same], w_same)
70     else:
71         sam_l, sam_p = rec_l, rec_p
72     return W_LOAD * sam_l + (1 - W_LOAD) * rec_l, W_PV * sam_p + (1 - W_PV) * rec_p
73 raise ValueError(method)
74
75
76 def run_all():
77     a1, load, pv, price4, fc3 = load_all()
78     prior_load, prior_pv = a1["load"], a1["pv"]
79     methods = ["persistence", "mean7", "weekday4", "ewmix", "hybrid1", "hybrid"]
80     fc = {m: {"load": np.zeros((DAYS, N)), "pv": np.zeros((DAYS, N))} for m in methods}
81     for m in methods:
82         for d in range(DAYS):
83             lf, pf = forecast_day(m, load, pv, d, prior_load, prior_pv)
84             fc[m]["load"][d], fc[m]["pv"][d] = lf, pf
85
86     # 误差统计(2月-12月, 334天正式期; 1月为过渡期单列)
87     rep = slice(31, 365)
88     print(f"{'方法':<12}{'负载MAE':>10}{'负载RMSE':>10}{'光伏MAE':>10}{'净负荷MAE':>12}")
89     for m in methods:
90         le = fc[m]["load"][rep] - load[rep]
91         pe = fc[m]["pv"][rep] - pv[rep]
92         ne = (fc[m]["load"][rep] - fc[m]["pv"][rep]) - (load[rep] - pv[rep])
93         print(f"{m:<12}{np.abs(le).mean():>10.1f}{np.sqrt((le**2).mean()):>10.1f}"
94               f"{np.abs(pe).mean():>10.1f}{np.abs(ne).mean():>12.1f}")
95

```

```

96 # 保存主预测器 (hybrid: 加权同星期3周负载 + 近7日加权光伏) 与全部候选
97 np.savez(CODE / "cache" / "forecast_p2.npz",
98         load_fc=fc["hybrid"]["load"], pv_fc=fc["hybrid"]["pv"],
99         **{f"{m}_load": fc[m]["load"] for m in methods},
100         **{f"{m}_pv": fc[m]["pv"] for m in methods})
101 # 兼容别名 (旧代码引用 mean7_pv / hybrid2_*)
102 z = dict(np.load(CODE / "cache" / "forecast_p2.npz"))
103 z["hybrid2_load"], z["hybrid2_pv"] = fc["hybrid"]["load"], fc["hybrid"]["pv"]
104 np.savez(CODE / "cache" / "forecast_p2.npz", **z)
105 log("预测缓存已保存 cache/forecast_p2.npz")
106
107 # 星期效应强度 (论文 EDA 素材)
108 dow = pd.date_range("2025-01-01", periods=DAYS).dayofweek.to_numpy()
109 mean_curve = load.mean(axis=0)
110 var_all = load.var(axis=0).mean()
111 var_res = 0.0
112 for k in range(7):
113     grp = load[dow == k]
114     var_res += len(grp) / DAYS * ((grp - grp.mean(axis=0)) ** 2).mean(axis=0).mean()
115 print(f"\n负载星期效应: 总方差 {var_all:.0f} -> 去星期均值后 {var_res:.0f} (解释 {(1-var_res/var_all)*100:.1f}%)")
116 mon = pd.date_range("2025-01-01", periods=DAYS).month.to_numpy()
117 pv_monthly = np.array([pv[mon == m].sum() / 6 / (mon == m).sum() for m in range(1, 13)])
118 print("光伏日均电量(万kWh/天) 按月:", np.round(pv_monthly / 1e4, 2))
119
120
121 if __name__ == "__main__":
122     run_all()

```

Listing 4 问题二仿真程序 q2_simulate.py

```

1 # -*- coding: utf-8 -*-
2 """问题2: 随机负载/光伏下的日前计划 + 实时执行仿真。
3
4 流程 (严格因果):
5     每天 0:00 —— 用历史数据预测当日负载/光伏 (q2_forecast 的 hybrid 预测器)
6         —— 净负荷预测误差的近28天逐时段 q 分位数缓冲 (报童模型: 5倍惩罚 F(q)=0.8)
7         —— 解日前 LP (负载预测+缓冲) 得计划购电量 g (按计划量结算 take-or-pay)
8     白天每 10 分钟 —— 实际净负荷 vs g: 盈余充电(越限弃)、缺口放电(越限紧急购电 5x)
9     储能 SOC 跨日连续传递: 1月为过渡期, 2.1-12.31 (334天) 为正式回溯期。
10
11 执行模式: greedy (缺口即放电); rolling (缺口时对剩余时段重解 LP, 权衡当前 5p_k
12 放电与储能电量规避未来更高价 5p 的紧急购电; 盈余时段贪心充电恒为最优, 无需求解)。
13 """
14 import sys, io, time
15 from pathlib import Path
16 sys.path.insert(0, str(Path(__file__).parent))
17 import numpy as np
18 import pandas as pd
19 from scipy.optimize import linprog
20 from scipy.sparse import lil_matrix
21 from common import (load_all, solve_plan, key_numbers, log, RESULTS,
22                     N, DT, E_MAX, SOC_MIN, SOC_MAX, ETA, SOC_INIT, DAYS)
23
24 A1, LOAD, PV, PRICE4, FC3 = load_all()
25 PRICE_A1 = A1["price"] # 固定日电价 (文件列序, 每天相同)
26 JAN = slice(0, 31)

```

```

27 REP = slice(31, 365) # 2025.2.1 - 12.31
28
29
30 def buffer_curve(net_err_hist, q):
31     """net_err_hist: (n,144) 净负荷预测误差（预测-实际，kW）。
32     紧急购电由「实际净负荷 > 计划供给」触发，故缓冲取（实际-预测）的 q 分位数。"""
33     if q is None or len(net_err_hist) == 0:
34         return np.zeros(N)
35     return np.maximum(np.quantile(-net_err_hist[-28:], q, axis=0), 0.0)
36
37
38 def exec_lp(price_k, load_kw_k, pv_kw_k, g_lock_k, soc, soc_target):
39     """滚动执行：剩余时段 LP（g 锁定；时段0用实际值，未来用当日点预测）。
40     返回时段0的（charge, discharge, emergency, spill）。"""
41     n = len(g_lock_k)
42     nv = 6 * n
43     G, C, D_, SP, EM, S = [slice(i * n, (i + 1) * n) for i in range(6)]
44     obj = np.zeros(nv)
45     obj[EM] = 5.0 * price_k
46     obj[C] = 1e-7; obj[D_] = 1e-7 # 微罚吞吐：消除同时充放的退化解
47     load_e = np.asarray(load_kw_k) * DT
48     pv_e = np.asarray(pv_kw_k) * DT
49     A = lil_matrix((2 * n + 1, nv)); b = np.zeros(2 * n + 1)
50     for t in range(n):
51         A[t, S.start + t] = 1.0
52         if t > 0:
53             A[t, S.start + t - 1] = -1.0
54         A[t, C.start + t] = -ETA
55         A[t, D_.start + t] = 1.0 / ETA
56     b[0] = soc
57     for t in range(n):
58         r = n + t
59         A[r, G.start + t] = 1.0; A[r, D_.start + t] = 1.0; A[r, EM.start + t] = 1.0
60         A[r, C.start + t] = -1.0; A[r, SP.start + t] = -1.0
61         b[r] = load_e[t] - pv_e[t]
62     A[2 * n, S.start + n - 1] = 1.0; b[2 * n] = soc_target # 终端：当日循环水平
63     bounds = [(0, None)] * nv
64     for t in range(n):
65         bounds[C.start + t] = (0, E_MAX); bounds[D_.start + t] = (0, E_MAX)
66         bounds[S.start + t] = (SOC_MIN, SOC_MAX)
67         bounds[G.start + t] = (g_lock_k[t], g_lock_k[t])
68     res = linprog(obj, A_eq=A.tocsr(), b_eq=b, bounds=bounds, method="highs")
69     if res.status != 0: # 终端不可行 → 放松
70         res = linprog(obj, A_eq=A.tocsr()[: 2 * n], b_eq=b[: 2 * n],
71                       bounds=bounds, method="highs")
72     assert res.status == 0, "执行 LP 不可行"
73     x = res.x
74     return x[C][0], x[D_][0], x[EM][0], x[SP][0]
75
76
77 def simulate(load_fc, pv_fc, price_days, buffer_q=0.8, exec_mode="greedy"):
78     """全年 365 天仿真。price_days: (DAYS,144) 或 (1,144)。"""
79     price = np.broadcast_to(np.asarray(price_days, float), (DAYS, N))
80     g_plan = np.zeros((DAYS, N)); C = np.zeros_like(g_plan); D_ = np.zeros_like(g_plan)
81     SP = np.zeros_like(g_plan); EM = np.zeros_like(g_plan)
82     soc_start = np.zeros(DAYS)
83     cost_plan = np.zeros(DAYS); cost_emg = np.zeros(DAYS)

```

```

84 soc = SOC_INIT
85 net_err_hist = []
86 t0 = time.time()
87 for d in range(DAYS):
88     lf, pf = load_fc[d], pv_fc[d]
89     buf = buffer_curve(np.array(net_err_hist) if net_err_hist else np.zeros((0, N)), buffer_q)
90     soc_start[d] = soc
91     sol = solve_plan(price[d], lf + buf, pf, soc0=soc, cyclic=True, validate=(d < 3))
92     assert sol["status"] == 0, f"day {d} 计划 LP 失败"
93     g = sol["g"]
94     g_plan[d] = g
95     cost_plan[d] = float(price[d] @ g)
96
97     net = g + PV[d] * DT - LOAD[d] * DT # kWh, 计划日槽坐标
98     for k in range(N):
99         nk = net[k]
100         if nk >= 0: # 盈余: 贪心充电恒最优
101             ch = min(nk, E_MAX, (SOC_MAX - soc) / ETA)
102             dis, em, sp = 0.0, 0.0, nk - ch
103         else:
104             deficit = -nk
105             if exec_mode == "rolling" and soc > SOC_MIN + 1e-6:
106                 load_k = np.concatenate([[LOAD[d, k]], lf[k + 1:]])
107                 pv_k = np.concatenate([[PV[d, k]], pf[k + 1:]])
108                 ch, dis, em, sp = exec_lp(price[d][k:], load_k, pv_k, g[k:], soc, soc_start[d])
109             else:
110                 dis = min(deficit, E_MAX, (soc - SOC_MIN) * ETA)
111                 em = deficit - dis; ch, sp = 0.0, 0.0
112                 C[d, k], D_[d, k], SP[d, k], EM[d, k] = ch, dis, sp, em
113                 soc += ETA * ch - dis / ETA
114             cost_emg[d] = float(price[d] @ EM[d]) * 5.0
115             net_err_hist.append((lf - pf) - (LOAD[d] - PV[d]))
116
117 out = dict(g_plan=g_plan, c=C, d=D_, spill=SP, emg=EM, soc_start=soc_start,
118           cost_plan=cost_plan, cost_emg=cost_emg, cost_total=cost_plan + cost_emg,
119           spill_total=float(SP.sum()), emg_total=float(EM.sum()))
120 log(f" 仿真({exec_mode}, q={buffer_q}): 回溯期总费用 {out['cost_total'][REP].sum():.0f} 元, "
121     f"回溯期紧急 {out['emg'][REP].sum():.0f} kWh, {time.time() - t0:.1f}s")
122 return out
123
124
125 def clock_day_view(arr_pd):
126     """计划日坐标 (DAYS,144) → 钟表日坐标 (DAYS+1,144)。
127     全局槽 m=d*144+k+1 → clock day=m//144, j=m%144 (末槽越界到第366天, 截断)。"""
128     out = np.zeros((DAYS + 1, N))
129     for d in range(DAYS):
130         for k in range(N):
131             m = d * N + k + 1
132             out[m // N, m % N] = arr_pd[d, k]
133     return out[:DAYS]
134
135
136 def emergency_intervals(emg_clock, D):
137     """钟表日 D 紧急购电连续区间合并 → [(label, kWh)]"""
138     slots = np.where(emg_clock[D] > 1e-6)[0]
139     if len(slots) == 0:
140         return []

```

```

141     ivs, st, prev = [], slots[0], slots[0]
142     for j in slots[1:]:
143         if j == prev + 1:
144             prev = j
145         else:
146             ivs.append((st, prev)); st = prev = j
147     ivs.append((st, prev))
148     fmt = lambda m: f"{m // 60}:{m % 60:02d}" if m < 1440 else "24:00"
149     return [(f"{fmt(a * 10)}-{fmt((b + 1) * 10)}", float(emg_clock[D, a:b + 1].sum()))
150             for a, b in ivs]
151
152
153 def main():
154     fc = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")
155     load_fc, pv_fc = fc["load_fc"], fc["pv_fc"]          # hybrid 预测器
156
157     # ---- 1月（过渡期）选缓冲分位数：报童理论值 0.8 附近网格 ----
158     log("【1月调参】缓冲分位数网格（hybrid 预测器 + 贪心执行，按 1 月总费用选择）")
159     jan_scores = {}
160     for q in [0.5, 0.6, 0.7, 0.8, 0.9, 0.95]:
161         sim = simulate(load_fc, pv_fc, PRICE_A1[None, :], buffer_q=q, exec_mode="greedy")
162         jan_scores[q] = float(sim["cost_total"][JAN].sum())
163         print(f"    q={q}: 1月总费用 {jan_scores[q]:.0f} 元")
164     q_star = min(jan_scores, key=jan_scores.get)
165     log(f"    → 选定 q* = {q_star}（理论值 0.8）")
166
167     # ---- 正式回测（2.1-12.31）四策略 ----
168     log("【正式回测 2.1-12.31】")
169     base_cost = np.array([float(PRICE_A1 @ np.maximum(LOAD[d] * DT - PV[d] * DT, 0))
170                           for d in range(DAYS)])
171     sim_naive = simulate(load_fc, pv_fc, PRICE_A1[None, :], buffer_q=None, exec_mode="greedy")
172     sim_main = simulate(load_fc, pv_fc, PRICE_A1[None, :], buffer_q=q_star, exec_mode="greedy")
173     sim_roll = simulate(load_fc, pv_fc, PRICE_A1[None, :], buffer_q=q_star, exec_mode="rolling")
174     sim_oracle = simulate(LOAD, PV, PRICE_A1[None, :], buffer_q=None, exec_mode="greedy")
175
176     R = REP
177     rows = [
178         ("无储能基准", float(base_cost[R].sum()), 0.0),
179         ("朴素预测（无缓冲）+贪心", float(sim_naive["cost_total"][R].sum()), float(sim_naive["cost_emg"][R].sum()))
180     ],
181     (f"分位数缓冲(q={q_star})+贪心", float(sim_main["cost_total"][R].sum()), float(sim_main["cost_emg"][R].sum()
182     )),
183     (f"分位数缓冲(q={q_star})+滚动执行", float(sim_roll["cost_total"][R].sum()), float(sim_roll["cost_emg"][R].sum()
184     )),
185     ("理想先知（完全预测）", float(sim_oracle["cost_total"][R].sum()), float(sim_oracle["cost_emg"][R].sum())),
186 ]
187
188     print("\n策略 | 回测期总费用(元) | 其中紧急购电费(元) | 紧急占比")
189     for name, tot, em in rows:
190         print(f"{name:<26}{tot:>15,.0f}{em:>17,.0f}    {em / max(tot, 1) * 100:>5.1f}%")
191
192     best = sim_roll if sim_roll["cost_total"][R].sum() < sim_main["cost_total"][R].sum() else sim_main
193     best_mode = "rolling" if best is sim_roll else "greedy"
194     log(f"主策略执行模式: {best_mode}")
195
196     # ---- 缓冲分位数全年敏感性 ----
197     sens_q = []
198     for q in [0.5, 0.6, 0.7, 0.8, 0.9, 0.95]:

```

```

195     if q == q_star:
196         c = float(best["cost_total"][R].sum())
197     elif q == 0.5:
198         c = float(sim_naive["cost_total"][R].sum())
199     else:
200         c = float(simulate(load_fc, pv_fc, PRICE_A1[None, :], buffer_q=q,
201                             exec_mode=best_mode)["cost_total"][R].sum())
202     sens_q.append((q, c))
203     print(f"  敏感性 q={q}: 回溯期总费用 {c:,.0f} 元")
204
205     np.savez(RESULTS / "q2_main_sim.npz",
206             **{k: best[k] for k in ["g_plan", "c", "d", "spill", "emg",
207                                     "soc_start", "cost_plan", "cost_emg"]})
208     key_numbers(
209         q2_buffer_q=q_star, q2_exec_mode=best_mode,
210         q2_cost_baseline=round(float(base_cost[R].sum()), 0),
211         q2_cost_naive=round(float(sim_naive["cost_total"][R].sum()), 0),
212         q2_cost_main=round(float(best["cost_total"][R].sum()), 0),
213         q2_cost_main_plan=round(float(best["cost_plan"][R].sum()), 0),
214         q2_cost_main_emg=round(float(best["cost_emg"][R].sum()), 0),
215         q2_cost_oracle=round(float(sim_oracle["cost_total"][R].sum()), 0),
216         q2_cost_greedy=round(float(sim_main["cost_total"][R].sum()), 0),
217         q2_cost_rolling=round(float(sim_roll["cost_total"][R].sum()), 0),
218         q2_emg_total_kwh=round(float(best["emg"][REP].sum()), 0),
219         q2_spill_total_kwh=round(float(best["spill"][REP].sum()), 0),
220         q2_sens_q={str(q): round(c, 0) for q, c in sens_q},
221         q2_jan_scores={str(q): round(v, 0) for q, v in jan_scores.items()},
222     )
223     log("主策略仿真与关键数值已保存")
224     return best, q_star, best_mode
225
226
227 if __name__ == "__main__":
228     main()

```

Listing 5 问题二两阶段随机规划 q2_saa.py

```

1  # -*- coding: utf-8 -*-
2  """问题二模型升级：两阶段随机规划（场景法 SAA）。
3
4  报童分位数缓冲只利用了净负荷误差的逐槽边缘分位数，且不感知储能的跨时段耦合。
5  本模块以近 28 天的净负荷预测误差整条曲线为经验场景集，建立两阶段随机规划：
6      第一阶段（0:00）：计划购电量 g_t（take-or-pay，所有场景共用）；
7      第二阶段（场景 s 展开）：充 c^s、放 d^s、弃 v^s、紧急 e^s 及各场景 SOC 轨迹；
8      目标：min Σp·g + (1/S)Σ_s Σ 5p·e^s （期望总费用）
9      约束：各场景 SOC 平衡/功率上限/电量区间/日循环、供需平衡。
10  该模型是报童缓冲（无储能单时段特例）的严格推广，将风险决策内生。
11  """
12  import sys, time
13  from pathlib import Path
14  sys.path.insert(0, str(Path(__file__).parent))
15  import numpy as np
16  from scipy.optimize import linprog
17  from scipy.sparse import coo_matrix
18  from common import (load_all, solve_plan, key_numbers, log, RESULTS,
19                      N, DT, E_MAX, SOC_MIN, SOC_MAX, ETA, SOC_INIT, DAYS)
20  from q2_simulate import simulate, exec_lp, PRICE_A1, REP

```

```

21
22 A1, LOAD, PV, PRICE4, FC3 = load_all()
23
24
25 def solve_saa_plan(price, lf, pf, err_hist, soc0, S=28):
26     """两阶段 SAA。err_hist: (n,144) 净负荷误差(预测-实际, kW)，取最近 S 天为场景。
27     返回 g (144,)。"""
28     S = min(S, len(err_hist))
29     if S == 0:
30         # 无历史 → 退化为点预测 LP
31         sol = solve_plan(price, lf, pf, soc0=soc0, cyclic=True, validate=False)
32         return sol["g"]
33     errs = err_hist[-S:]
34     n_scn = (lf - pf)[None, :] - errs
35     nE = n_scn * DT
36     NV = N + 5 * N * S
37     GO = 0
38     def CO(s): return N + 5 * N * s
39     def DO(s): return N + 5 * N * s + N
40     def VO(s): return N + 5 * N * s + 2 * N
41     def EO(s): return N + 5 * N * s + 3 * N
42     def SO(s): return N + 5 * N * s + 4 * N
43
44     obj = np.zeros(NV)
45     obj[:N] = price
46     for s in range(S):
47         obj[EO(s):EO(s) + N] = 5.0 * price / S
48         obj[CO(s):CO(s) + N] = 1e-7
49         obj[DO(s):DO(s) + N] = 1e-7
50
51     rows, cols, vals, beq = [], [], [], []
52     r = 0
53     for s in range(S):
54         for t in range(N):
55             # SOC 平衡
56             rows += [r]; cols += [SO(s) + t]; vals += [1.0]
57             if t > 0:
58                 rows += [r]; cols += [SO(s) + t - 1]; vals += [-1.0]
59             else:
60                 beq.append(soc0)
61             rows += [r, r]; cols += [CO(s) + t, DO(s) + t]; vals += [-ETA, 1.0 / ETA]
62             if t > 0:
63                 beq.append(0.0)
64             r += 1
65         rows += [r]; cols += [SO(s) + N - 1]; vals += [1.0]; beq.append(soc0) # 日循环
66         r += 1
67         for t in range(N):
68             # 供需平衡
69             rows += [r]; cols += [GO + t]; vals += [1.0]
70             rows += [r, r, r, r]
71             cols += [DO(s) + t, EO(s) + t, CO(s) + t, VO(s) + t]
72             vals += [1.0, 1.0, -1.0, -1.0]
73             beq.append(nE[s, t])
74             r += 1
75     A = coo_matrix((vals, (rows, cols)), shape=(r, NV)).tocsr()
76     b = np.array(beq)
77
78     bounds = [(0, None)] * NV
79     for s in range(S):

```

```

78     for t in range(N):
79         bounds[CO(s) + t] = (0, E_MAX)
80         bounds[DO(s) + t] = (0, E_MAX)
81         bounds[SO(s) + t] = (SOC_MIN, SOC_MAX)
82     res = linprog(obj, A_eq=A, b_eq=b, bounds=bounds, method="highs")
83     assert res.status == 0, f"SAA LP 失败: {res.message}"
84     return res.x[:N]
85
86
87 def simulate_saa(load_fc, pv_fc, price_days, S=28, exec_mode="rolling"):
88     """全年仿真: SAA 计划 + 与问题二相同的执行层。"""
89     price = np.broadcast_to(np.asarray(price_days, float), (DAYS, N))
90     g_plan = np.zeros((DAYS, N)); C = np.zeros_like(g_plan); D_ = np.zeros_like(g_plan)
91     SP = np.zeros_like(g_plan); EM = np.zeros_like(g_plan)
92     soc_start = np.zeros(DAYS); cost_plan = np.zeros(DAYS); cost_emg = np.zeros(DAYS)
93     soc = SOC_INIT
94     err_hist = []
95     t0 = time.time()
96     for d in range(DAYS):
97         lf, pf = load_fc[d], pv_fc[d]
98         soc_start[d] = soc
99         g = solve_saa_plan(price[d], lf, pf, np.array(err_hist) if err_hist else np.zeros((0, N)),
100                             soc, S=S)
101         g_plan[d] = g
102         cost_plan[d] = float(price[d] @ g)
103         net = g + PV[d] * DT - LOAD[d] * DT
104         for k in range(N):
105             nk = net[k]
106             if nk >= 0:
107                 ch = min(nk, E_MAX, (SOC_MAX - soc) / ETA)
108                 dis, em, sp = 0.0, 0.0, nk - ch
109             else:
110                 deficit = -nk
111                 if exec_mode == "rolling" and soc > SOC_MIN + 1e-6:
112                     load_k = np.concatenate([[LOAD[d, k]], lf[k + 1:]])
113                     pv_k = np.concatenate([[PV[d, k]], pf[k + 1:]])
114                     ch, dis, em, sp = exec_lp(price[d][k:], load_k, pv_k, g[k:], soc, soc_start[d])
115                 else:
116                     dis = min(deficit, E_MAX, (soc - SOC_MIN) * ETA)
117                     em = deficit - dis; ch, sp = 0.0, 0.0
118                     C[d, k], D_[d, k], SP[d, k], EM[d, k] = ch, dis, sp, em
119                     soc += ETA * ch - dis / ETA
120                 cost_emg[d] = float(price[d] @ EM[d]) * 5.0
121                 err_hist.append((lf - pf) - (LOAD[d] - PV[d]))
122     out = dict(g_plan=g_plan, c=C, d=D_, spill=SP, emg=EM, soc_start=soc_start,
123               cost_plan=cost_plan, cost_emg=cost_emg, cost_total=cost_plan + cost_emg,
124               spill_total=float(SP.sum()), emg_total=float(EM.sum()))
125     log(f" SAA仿真(S={S}, {exec_mode}): 回溯期总费用 {out['cost_total'][REP].sum():.0f} 元 "
126         f"(计划 {out['cost_plan'][REP].sum():.0f}, 紧急 {out['cost_emg'][REP].sum():.0f}), "
127         f"{time.time() - t0:.1f}s")
128     return out
129
130
131 def main():
132     fc = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")
133     lf, pf = fc["load_fc"], fc["pv_fc"]
134     # 1月(过渡期)选场景数 S

```

```

135     jan = {}
136     for S in [10, 14, 28]:
137         s = simulate_saa(lf, pf, PRICE_A1[None, :], S=S, exec_mode="greedy")
138         jan[S] = float(s["cost_total"][:31].sum())
139         print(f"    S={S}: 1月总费用 {jan[S]:,.0f} 元")
140     S_star = min(jan, key=jan.get)
141     log(f"SAA 选定场景数 S* = {S_star}")
142
143     # 正式回测: SAA × 滚动执行
144     sim_saa = simulate_saa(lf, pf, PRICE_A1[None, :], S=S_star, exec_mode="rolling")
145     # 对照: 报童缓冲主策略 (q=0.7 + 滚动, 已有结果)
146     fc_kn = RESULTS / "key_numbers.json"
147     import json
148     kn = json.loads(fc_kn.read_text(encoding="utf-8"))
149     buf_cost = kn["q2_cost_main"]
150     saa_cost = float(sim_saa["cost_total"][REP].sum())
151     print(f"\nSAA(S={S_star})+滚动: {saa_cost:,.0f} 元 vs 报童缓冲+滚动: {buf_cost:,.0f} 元 "
152           f"→ {'SAA 更优' if saa_cost < buf_cost else '缓冲更优'}, 差 {abs(saa_cost - buf_cost):,.0f} 元")
153
154     if saa_cost < buf_cost:
155         np.savez(RESULTS / "q2_main_sim.npz",
156                 **{k: sim_saa[k] for k in ["g_plan", "c", "d", "spill", "emg",
157                                           "soc_start", "cost_plan", "cost_emg"]})
158         log("SAA 结果已覆盖 q2_main_sim.npz (成为问题二主策略)")
159     key_numbers(q2_saa_S=S_star, q2_saa_cost=round(saa_cost, 0),
160               q2_saa_jan={str(k): round(v, 0) for k, v in jan.items()},
161               q2_main_is_saa=bool(saa_cost < buf_cost))
162
163
164 if __name__ == "__main__":
165     main()

```

Listing 6 问题二结果输出 q2_results.py

```

1  # -*- coding: utf-8 -*-
2  """问题2 结果输出: result2.xlsx (按附件5模板)、四指定日明细、论文图表。"""
3  import sys, io, json
4  from pathlib import Path
5  sys.path.insert(0, str(Path(__file__).parent))
6  import numpy as np
7  import pandas as pd
8  from openpyxl import load_workbook
9  from common import load_all, key_numbers, log, RESULTS, FIGURES, N, DT, ETA, SOC_INIT, DAYS
10 from q2_simulate import clock_day_view, emergency_intervals, PRICE_A1, REP
11
12 A1, LOAD, PV, PRICE4, FC3 = load_all()
13 sim = np.load(RESULTS / "q2_main_sim.npz")
14 g_plan, C, D_, EM = sim["g_plan"], sim["c"], sim["d"], sim["emg"]
15 soc_start = sim["soc_start"]
16 cost_plan, cost_emg = sim["cost_plan"], sim["cost_emg"]
17
18 DATES = pd.date_range("2025-01-01", periods=DAYS)
19 REP_DATES = DATES[31:365] # 2025-02-01 .. 12-31 (334天)
20
21 # ---- 钟表日视图 (执行量) ----
22 c_ck = clock_day_view(C); d_ck = clock_day_view(D_); emg_ck = clock_day_view(EM)
23 # 钟表日 SOC 边界: SOC(D 0:00) = soc_start[D] - Δ(计划日D-1末槽)

```

```

24 soc_0 = np.zeros(DAYS); soc_24 = np.zeros(DAYS)
25 soc_0[0] = SOC_INIT
26 for D in range(1, DAYS):
27     soc_0[D] = soc_start[D] - (ETA * C[D - 1, 143] - D_[D - 1, 143] / ETA)
28 soc_24 = np.roll(soc_0, -1)
29 # 12月31日 24:00 = 计划日364第143槽 (k=142) 之后
30 soc_24[364] = soc_start[364] + float(np.sum(ETA * C[364, :143] - D_[364, :143] / ETA))
31 assert abs(soc_24[363] - soc_0[364]) < 1e-6
32
33 BLOCKS = [(0, 24, "0:00-4:00"), (24, 48, "4:00-8:00"), (48, 72, "8:00-12:00"),
34           (72, 96, "12:00-16:00"), (96, 120, "16:00-20:00"), (120, 144, "20:00-24:00")]
35
36 # ---- 写 result2.xlsx ----
37 tpl = r"C:\Users\26517\Documents\CUMCM\C题\附件\附件5\result2.xlsx"
38 wb = load_workbook(tpl)
39 ws = wb["计划购电量"]
40 hdr = [ws.cell(row=1, column=c).value for c in range(1, 148)]
41 assert hdr[1] == "0:10-0:20" and hdr[144] == "0:00-0:10+1", f"模板列标签异常: {hdr[1]}, {hdr[144]}"
42 ws.delete_rows(2, ws.max_row)
43 for i, D in enumerate(range(31, 365)):
44     r = i + 2
45     ws.cell(row=r, column=1, value=REP_DATES[i].to_pydatetime())
46     for k in range(144):
47         ws.cell(row=r, column=2 + k, value=round(float(g_plan[D, k]), 4))
48         ws.cell(row=r, column=146, value=round(float(g_plan[D].sum()), 2))
49         ws.cell(row=r, column=147, value=round(float(cost_plan[D]), 2))
50
51 ws2 = wb["充放电量"]
52 ws2.delete_rows(2, ws2.max_row)
53 r = 2
54 for i, D in enumerate(range(31, 365)):
55     for b, (a, bnd, lab) in enumerate(BLOCKS):
56         ws2.cell(row=r, column=1, value=REP_DATES[i].to_pydatetime() if b == 0 else None)
57         ws2.cell(row=r, column=2, value=lab)
58         ws2.cell(row=r, column=3, value=round(float(c_ck[D, a:bnd].sum()), 4))
59         ws2.cell(row=r, column=4, value=round(float(d_ck[D, a:bnd].sum()), 4))
60         if b == 0:
61             ws2.cell(row=r, column=5, value="0:00"); ws2.cell(row=r, column=6, value=round(float(soc_0[D]), 2))
62         elif b == 1:
63             ws2.cell(row=r, column=5, value="24:00"); ws2.cell(row=r, column=6, value=round(float(soc_24[D]), 2))
64         r += 1
65
66 ws3 = wb["紧急购电量"]
67 ws3.delete_rows(2, ws3.max_row)
68 r = 2
69 emg_days = 0
70 for D in range(31, 365):
71     ivs = emergency_intervals(emg_ck, D)
72     if not ivs:
73         continue
74     emg_days += 1
75     for j, (lab, q) in enumerate(ivs):
76         ws3.cell(row=r, column=1, value=REP_DATES[D - 31].to_pydatetime() if j == 0 else None)
77         ws3.cell(row=r, column=2, value=lab)
78         ws3.cell(row=r, column=3, value=round(q, 2))
79         r += 1
80 out = RESULTS / "result2.xlsx"

```

```

81 wb.save(out)
82 log(f"result2.xlsx 已写出: 334天计划 + {334*6}行充放电 + {emg_days}个紧急购电日")
83
84 # ---- 四指定日明细 (论文表3用) ----
85 SPEC = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
86 detail = {}
87 for s in SPEC:
88     D = (pd.Timestamp(s) - pd.Timestamp("2025-01-01")).days
89     t1_slots = [60, 72, 84, 96, 108, 120] # 10:00,12:00,...,20:00 (钟表槽)
90     # 计划购电量按计划日 (文件序号 label=(k+1)*10 分钟) 取槽 10:00..20:00 → k=59,71,...
91     t1 = {f"{(k+1)*10//60:02d}:{(k+1)*10%60:02d}": round(float(g_plan[D, k]), 2)
92           for k in [59, 71, 83, 95, 107, 119]}
93     t2 = {lab: [round(float(c_ck[D, a:bnd].sum()), 2), round(float(d_ck[D, a:bnd].sum()), 2)]
94           for a, bnd, lab in BLOCKS}
95     t3 = emergency_intervals(emg_ck, D)
96     detail[s] = dict(
97         day_index=D, total_g=round(float(g_plan[D].sum()), 2),
98         cost_plan=round(float(cost_plan[D]), 2), cost_emg=round(float(cost_emg[D]), 2),
99         cost_total=round(float(cost_plan[D] + cost_emg[D]), 2),
100         soc_0=round(float(soc_0[D]), 2), soc_24=round(float(soc_24[D]), 2),
101         t1_slots=t1, t2_blocks=t2, t3_emergency=t3)
102     print(f"{s}: 计划 {detail[s]['total_g']:.0f} kWh / {detail[s]['cost_plan']:.0f} 元, "
103           f"紧急 {detail[s]['cost_emg']:.0f} 元, SOC {detail[s]['soc_0']:.0f}→{detail[s]['soc_24']:.0f}, "
104           f"紧急时段 {len(t3)} 段")
105
106 # ---- 图 ----
107 import matplotlib
108 matplotlib.use("Agg")
109 import matplotlib.pyplot as plt
110 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
111 plt.rcParams["axes.unicode_minus"] = False
112
113 # 图A: 策略对比 (重算各策略数值从 key_numbers 读取)
114 kn = json.loads((RESULTS / "key_numbers.json").read_text(encoding="utf-8"))
115 strategies = ["无储能\n基准", "朴素预测\n+贪心", "分位数缓冲\n+贪心", "分位数缓冲\n+滚动(主)", "理想先知"]
116 totals = [kn["q2_cost_baseline"], kn["q2_cost_naive"], kn["q2_cost_greedy"],
117           kn["q2_cost_main"], kn["q2_cost_oracle"]]
118 fig, ax = plt.subplots(figsize=(8.6, 4.6))
119 x = np.arange(len(strategies))
120 bars = ax.bar(x, [t / 1e4 for t in totals], 0.58,
121              color=["#95a5a6", "#e67e22", "#3498db", "#c0392b", "#27ae60"])
122 for xi, t in zip(x, totals):
123     ax.text(xi, t / 1e4 + 12, f"{t/1e4:.1f}", ha="center", fontsize=10)
124 ax.set_xticks(x); ax.set_xticklabels(strategies, fontsize=9.5)
125 ax.set_ylabel("回溯期总购电费用 (万元)")
126 ax.set_title("问题二: 各策略全年购电费用对比 (2025.2.1 - 12.31)")
127 ax.set_ylim(0, max(totals) / 1e4 * 1.15)
128 fig.tight_layout()
129 fig.savefig(FIGURES / "fig_q2_strategy_cmp.png", dpi=300); plt.close(fig)
130
131 # 图B: 缓冲分位数敏感性
132 sens = kn["q2_sens_q"]
133 qs = sorted(float(k) for k in sens)
134 fig, ax = plt.subplots(figsize=(7.4, 4.2))
135 ax.plot([q * 100 for q in qs], [sens[str(q)] / 1e4 for q in qs], "o-", color="#2c3e50", lw=1.8)
136 ax.axvline(80, color="#c0392b", ls="--", lw=1, label="报童理论值 80%")
137 ax.set_xlabel("缓冲分位数 q (%)"); ax.set_ylabel("回溯期总费用 (万元)")

```

```

138 ax.set_title("缓冲分位数敏感性"); ax.legend()
139 fig.tight_layout(); fig.savefig(FIGURES / "fig_q2_sens_q.png", dpi=300); plt.close(fig)
140
141 # 图C: 典型紧急日调度图 (正式回测期内紧急费用最大的日期)
142 emg_cost_daily = cost_emg.copy(); emg_cost_daily[:31] = 0.0 # 排除1月过渡期
143 Dmax = int(np.argmax(emg_cost_daily))
144 lab_time = [(k + 1) * 10 / 60 for k in range(N)]
145 fig, axes = plt.subplots(2, 1, figsize=(9.5, 6.2), sharex=True,
146                        gridspec_kw={"height_ratios": [3, 1.6]})
147 ax = axes[0]
148 ax.plot(lab_time, PRICE_A1, color="#888888", lw=1.2, label="电价 (右轴)")
149 ax.set_ylabel("电价 (元/kWh)", color="#888888"); ax.tick_params(axis="y", labelcolor="#888888")
150 ax2 = ax.twinx()
151 ax2.step(lab_time, g_plan[Dmax], where="post", color="#c0392b", lw=1.5, label="计划购电量")
152 ax2.bar(lab_time, C[Dmax], width=10/60, align="edge", color="#2471a3", alpha=0.55, label="实际充电")
153 ax2.bar(lab_time, -D[Dmax], width=10/60, align="edge", color="#1e8449", alpha=0.55, label="实际放电")
154 emg_slots = EM[Dmax] > 1e-6
155 ax2.bar(np.array(lab_time)[emg_slots], EM[Dmax][emg_slots], width=10/60, align="edge",
156         color="#8e44ad", alpha=0.9, label="紧急购电")
157 ax2.axhline(0, color="k", lw=0.6); ax2.set_ylabel("电量 (kWh/10min)")
158 h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
159 ax.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=8.5, ncol=5, framealpha=0.9)
160 ax.set_title(f"问题二主策略在紧急购电最严重日的调度 ({DATES[Dmax]}: %Y-%m-%d)")
161 soc_tr = soc_start[Dmax] + np.cumsum(ETA * C[Dmax] - D[Dmax] / ETA)
162 axes[1].plot(lab_time, soc_tr, color="#6c3483", lw=1.8)
163 axes[1].axhline(1200, color="r", ls="--", lw=0.8); axes[1].axhline(10800, color="r", ls="--", lw=0.8)
164 axes[1].set_ylabel("储电量 (kWh)"); axes[1].set_xlabel("时刻 (h)"); axes[1].set_ylim(0, 12000)
165 fig.tight_layout(); fig.savefig(FIGURES / "fig_q2_worst_day.png", dpi=300); plt.close(fig)
166 log(f"图已输出: fig_q2_strategy_cmp / fig_q2_sens_q / fig_q2_worst_day ({DATES[Dmax]}: %m-%d)")
167
168 key_numbers(q2_spec_dates=detail, q2_worst_day=DATES[Dmax].strftime("%Y-%m-%d"),
169            q2_emg_days=emg_days)
170 log("四指定日明细与关键数值已保存")

```

Listing 7 问题三仿真程序 q3_simulate.py

```

1 # -*- coding: utf-8 -*-
2 """问题3: 多时点预报下的滚动调整购电模型。
3
4 信息结构: 0:00 有附件3 的 0:00 光伏预报 (24 整点线性插值) + 自建负载预测;
5 6/12/18 时各有新预报, 可对 T+1 时起的剩余时段做调整。
6 偏差结算 (主口径 A): 最终量 a 相对 0:00 计划 g,
7 结算 =  $p \cdot a + 0.5p \cdot (a-g)^+ + 0.5p \cdot (g-a)^+$  (少买部分按 50% 结算、多买超出部分 1.5 倍)
8 备选口径 B: 结算 =  $p \cdot a + 0.5p \cdot (a-g)^+ + 1.5p \cdot (g-a)^+$  (少买不退款, 敏感性分析)。
9 调整 LP 中以  $u=a-g$  上偏、 $v=g-a$  下偏线性化: A 口标  $0.5p \cdot (u+v)$ , B 口标  $0.5p \cdot u + 1.5p \cdot v$ 。
10 缓冲: 各发布时刻分别用其净负荷预测误差的近 28 天 q 分位数 (报童模型, q 沿用问题二协议 0.7)。
11 """
12 import sys, time
13 from pathlib import Path
14 sys.path.insert(0, str(Path(__file__).parent))
15 import numpy as np
16 import pandas as pd
17 from scipy.optimize import linprog
18 from scipy.sparse import lil_matrix
19 from common import (load_all, solve_plan, key_numbers, log, RESULTS,
20                    N, DT, E_MAX, SOC_MIN, SOC_MAX, ETA, SOC_INIT, DAYS)
21 from common import pv_profile_day

```

```

22 from q2_simulate import exec_lp, PRICE_A1, REP, clock_day_view
23
24 A1, LOAD, PV, PRICE4, FC3 = load_all()
25 BUF_Q = 0.7
26 ISSUES = {6: 1, 12: 2, 18: 3}          # 发布时刻 -> fc3 列
27 KO = {6: 41, 12: 77, 18: 113}         # 各发布时刻首个可调槽 (T+1h 的标签槽)
28
29
30 def build_profiles():
31     load_fc = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")["load_fc"]
32     mean7 = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")["mean7_pv"]
33     off0 = np.array([pv_profile_day(FC3[d, 0, :], 0) for d in range(DAYS)])
34     offT = {T: np.array([pv_profile_day(FC3[d, i, :], T) for d in range(DAYS)])
35             for T, i in ISSUES.items()}
36
37     def blend(off, k0, k1):
38         """因果逆方差组合: 官方预报 × 历史均值, 逐槽权重由近28天误差方差估计。
39         官方预报噪声大 (0:00发布 MAE 199) 而历史均值偏平滑 (对午后差),
40         组合后 MAE 全面低于单一来源 (128.7/132.2/58.1 vs 各自最优 154.5/161.4/65.6)。"""
41         out = off.copy()
42         for d in range(1, DAYS):
43             h = slice(max(0, d - 28), d)
44             v1 = ((off[h, k0:k1] - PV[h, k0:k1]) ** 2).mean(0) + 1e-9
45             v2 = ((mean7[h, k0:k1] - PV[h, k0:k1]) ** 2).mean(0) + 1e-9
46             w = v2 / (v1 + v2)          # 官方预报权重
47             out[d, k0:k1] = w * off[d, k0:k1] + (1 - w) * mean7[d, k0:k1]
48         return out
49
50     pv0 = blend(off0, 0, 143)
51     pvT = {T: blend(offT[T], KO[T], 144) for T in ISSUES}
52     return load_fc, pv0, pvT, off0, mean7
53
54
55 def build_buffers(load_fc, pv0, pvT):
56     """各发布时刻的净负荷误差分位数缓冲 (严格因果: 只用当天之前)。
57     注意方向: 紧急购电由「实际净负荷 > 预测」触发, 缓冲取 (实际-预测) 的 q 分位数。"""
58     act_net = LOAD - PV
59     buf0 = np.zeros((DAYS, N))
60     for d in range(DAYS):
61         if d == 0:
62             continue
63         err = act_net[:d] - (load_fc[:d] - pv0[:d])    # 实际 - 预测
64         buf0[d] = np.maximum(np.quantile(err[max(0, d - 28):], BUF_Q, axis=0), 0.0)
65     bufT = {T: np.zeros((DAYS, N)) for T in ISSUES}
66     for T in ISSUES:
67         k0 = KO[T]
68         for d in range(DAYS):
69             if d == 0:
70                 continue
71             hist = []
72             for j in range(max(0, d - 28), d):
73                 e = act_net[j, k0:] - (load_fc[j, k0:] - pvT[T][j, k0:])
74                 hist.append(e)
75             if hist:
76                 bufT[T][d, k0:] = np.maximum(np.quantile(np.array(hist), BUF_Q, axis=0), 0.0)
77     return buf0, bufT
78

```

```

79
80 def build_errT(load_fc, pvT):
81     """各发布时刻新预报的净负荷预测误差（预测-实际，kW），供场景化调整使用。
82     errT[T][d, k0:] = (load_fc[d] - pvT[T][d]) - act_net[d], 覆盖槽之外为 NaN。"""
83     act_net = LOAD - PV
84     errT = {}
85     for T in ISSUES:
86         k0 = K0[T]
87         e = np.full((DAYS, N), np.nan)
88         e[:, k0:] = (load_fc[:, k0:] - pvT[T][:, k0:]) - act_net[:, k0:]
89         errT[T] = e
90     return errT
91
92
93 def adjust_lp_saa(price_k, net_fc_k, errs, g_plan_k, soc, soc_target, coeff_v=0.5):
94     """T 时刻场景化两阶段调整 LP（与 0:00 的 SAA 计划层风险视角一致）。
95
96     第一阶段：调整购电量  $a = g + u - v$ （偏差结算  $p \cdot a + 0.5p \cdot u + \text{coeff\_v} \cdot p \cdot v$ ）；
97     第二阶段：近 S 天新预报误差场景下的充/放/弃/紧急（期望罚金  $(1/S)\sum p \cdot e^s$ ）。
98     net_fc_k: (n,) 新预报净负荷(kW); errs: (S,n) 历史误差(预测-实际,kW)。返回 a。"""
99     S = len(errs)
100     n = len(g_plan_k)
101     if S == 0:
102         return g_plan_k.copy() # 无历史（过渡期）→ 不调整
103     nE = (np.asarray(net_fc_k)[None, :] - errs) * DT # (S,n) 场景净负荷 kWh
104     NV = 3 * n + 5 * n * S
105     A_, U, V = 0, n, 2 * n
106     C = lambda s: 3 * n + 5 * n * s
107     D_ = lambda s: 3 * n + 5 * n * s + n
108     SP = lambda s: 3 * n + 5 * n * s + 2 * n
109     EM = lambda s: 3 * n + 5 * n * s + 3 * n
110     S_ = lambda s: 3 * n + 5 * n * s + 4 * n
111
112     obj = np.zeros(NV)
113     obj[A_:A_ + n] = price_k
114     obj[U:U + n] = 0.5 * price_k
115     obj[V:V + n] = coeff_v * price_k
116     for s in range(S):
117         obj[EM(s):EM(s) + n] = 5.0 * price_k / S
118         obj[C(s):C(s) + n] = 1e-7
119         obj[D_(s):D_(s) + n] = 1e-7
120
121     rows, cols, vals, beq = [], [], [], []
122     r = 0
123     for t in range(n):
124         rows += [r, r, r]; cols += [A_ + t, U + t, V + t]; vals += [1.0, -1.0, 1.0]
125         beq.append(g_plan_k[t]); r += 1
126     for s in range(S):
127         for t in range(n):
128             rows.append(r); cols.append(S_(s) + t); vals.append(1.0)
129             if t > 0:
130                 rows.append(r); cols.append(S_(s) + t - 1); vals.append(-1.0)
131             else:
132                 beq.append(soc)
133             rows += [r, r]; cols += [C(s) + t, D_(s) + t]; vals += [-ETA, 1.0 / ETA]
134             if t > 0:
135                 beq.append(0.0)

```

```

136         r += 1
137         rows.append(r); cols.append(S(s) + n - 1); vals.append(1.0)
138         beq.append(soc_target); r += 1                                # 终端日循环
139         for t in range(n):                                           # 平衡
140             rows += [r, r, r, r, r]
141             cols += [A_ + t, D_(s) + t, EM(s) + t, C(s) + t, SP(s) + t]
142             vals += [1.0, 1.0, 1.0, -1.0, -1.0]
143             beq.append(nE[s, t]); r += 1
144     from scipy.sparse import coo_matrix
145     A = coo_matrix((vals, (rows, cols)), shape=(r, NV)).tocsr()
146     b = np.array(beq)
147     bounds = [(0, None)] * NV
148     for s in range(S):
149         for t in range(n):
150             bounds[C(s) + t] = (0, E_MAX)
151             bounds[D_(s) + t] = (0, E_MAX)
152             bounds[S_(s) + t] = (SOC_MIN, SOC_MAX)
153     res = linprog(obj, A_eq=A, b_eq=b, bounds=bounds, method="highs")
154     if res.status != 0:                                              # 终端不可行 → 去掉各场景终端行
155         drop = set()
156         idx = n
157         for s in range(S):
158             drop.add(idx + n)
159             idx += 2 * n + 1
160         keep = [i for i in range(r) if i not in drop]
161         A2 = A[keep, :]
162         b2 = b[keep]
163         res = linprog(obj, A_eq=A2, b_eq=b2, bounds=bounds, method="highs")
164         assert res.status == 0, "场景化调整 LP 不可行"
165     return res.x[A_:A_ + n]
166
167
168 def simulate_q3(load_fc, pv0, pvT, buf0, bufT, adjust_times, exec_mode="rolling",
169               coeff_v=0.5, price_days=None, plan_mode="buffer", errT=None,
170               scenario_rule="recent"):
171     """问题3 全年仿真。plan_mode: "buffer"/"saa" 为 0:00 计划方式;
172     scenario_rule: SAA 与调整层的场景日选择 ("recent"=近10天混合(默认, 实测最优);
173     "group"=按负荷日型分组(周五/周六为低需求组——实验验证: 混合场景对低需求日的
174     隐性过度防御反而更优, 保留参数仅作参考))。调整层为场景化两阶段 LP。"""
175     price = np.broadcast_to(PRICE_A1 if price_days is None else np.asarray(price_days, float),
176                             (DAYS, N))
177     G = np.zeros((DAYS, N)); Afin = np.zeros((DAYS, N))
178     C = np.zeros((DAYS, N)); D_ = np.zeros((DAYS, N))
179     SP = np.zeros((DAYS, N)); EM = np.zeros((DAYS, N))
180     soc_start = np.zeros(DAYS)
181     cost_settle = np.zeros(DAYS); cost_emg = np.zeros(DAYS)
182     soc = SOC_INIT
183     err_hist_saa = []                                              # SAA 计划层的净负荷误差流(预测-实际)
184     t0 = time.time()
185     for d in range(DAYS):
186         lf, p0 = load_fc[d], pv0[d]
187         price_d = price[d]
188         soc_start[d] = soc
189         if plan_mode == "saa":
190             from q2_saa import solve_saa_plan
191             from q2_variants import scenario_errors_days
192             err_all = np.array(err_hist_saa) if err_hist_saa else np.zeros((0, N))

```

```

193     errs_sel = scenario_errors_days(err_all, d, scenario_rule, 10)
194     if len(errs_sel):
195         g = solve_saa_plan(price_d, lf, p0, errs_sel, soc, S=len(errs_sel)).copy()
196     else:
197         sol = solve_plan(price_d, lf, p0, soc0=soc, cyclic=True, validate=False)
198         g = sol["g"].copy()
199     else:
200         sol = solve_plan(price_d, lf + buf0[d], p0, soc0=soc, cyclic=True, validate=(d < 3))
201         assert sol["status"] == 0
202         g = sol["g"].copy()
203     G[d] = g
204     cur_g = g.copy()
205     cur_pv = p0.copy() # 当前光伏预测（最新发布）
206     cur_buf = buf0[d].copy()
207     for k in range(N):
208         for T in adjust_times:
209             if k == K0[T]:
210                 newpv = pvT[T][d].copy()
211                 mask = ~np.isnan(newpv)
212                 cur_pv[mask] = newpv[mask]
213                 k0 = K0[T]
214                 if errT is not None and d >= 11:
215                     net_new = (lf - cur_pv)[k0:]
216                     if scenario_rule == "group":
217                         from q2_variants import GRP
218                         days = [j for j in range(max(1, d - 21), d)
219                               if GRP[j] == GRP[d]][-10:]
220                     else:
221                         days = list(range(max(1, d - 10), d))
222                     errs = errT[T][days, k0:]
223                     a = adjust_lp_saa(price_d[k0:], net_new, errs, g[k0:],
224                                     soc, soc_start[d], coeff_v)
225                 else: # 过渡期无误差历史 → 不调整
226                     a = g[k0:]
227                 cur_g[k0:] = np.maximum(a, 0.0)
228             # ---- 执行槽 k ----
229             net = cur_g[k] + PV[d, k] * DT - LOAD[d, k] * DT
230             if net >= 0:
231                 ch = min(net, E_MAX, (SOC_MAX - soc) / ETA)
232                 dis, em, sp = 0.0, 0.0, net - ch
233             else:
234                 deficit = -net
235                 if exec_mode == "rolling" and soc > SOC_MIN + 1e-6:
236                     load_k = np.concatenate([[LOAD[d, k]], lf[k + 1:]])
237                     pv_k = np.concatenate([[PV[d, k]], cur_pv[k + 1:]])
238                     ch, dis, em, sp = exec_lp(price_d[k:], load_k, pv_k, cur_g[k:],
239                                              soc, soc_start[d])
240                 else:
241                     dis = min(deficit, E_MAX, (soc - SOC_MIN) * ETA)
242                     em = deficit - dis; ch, sp = 0.0, 0.0
243                 C[d, k], D[d, k], SP[d, k], EM[d, k] = ch, dis, sp, em
244                 soc += ETA * ch - dis / ETA
245     Afin[d] = cur_g
246     # 结算（口径 A/B 由 coeff_v 决定记录）
247     dev = cur_g - g
248     if coeff_v == 0.5:
249         cost_settle[d] = float(price_d @ cur_g + 0.5 * price_d * np.abs(dev) @ np.ones(N))

```

```

250     else:
251         cost_settle[d] = float(price_d @ cur_g) + float(
252             (0.5 * price_d * np.maximum(dev, 0)).sum() +
253             (1.5 * price_d * np.maximum(-dev, 0)).sum())
254         cost_emg[d] = float(price_d @ EM[d]) * 5.0
255         err_hist_saa.append((lf - p0) - (LOAD[d] - PV[d]))
256
257     out = dict(g_plan=G, a_final=Afin, c=C, d=D_, spill=SP, emg=EM, soc_start=soc_start,
258             cost_settle=cost_settle, cost_emg=cost_emg,
259             cost_total=cost_settle + cost_emg,
260             settle_reading="A(退款)" if coeff_v == 0.5 else "B(不退款)")
261     log(f" Q3仿真(调整={adjust_times or '无'}, {exec_mode}): 回测期总费用 "
262         f"{out['cost_total'][REP].sum():.0f} 元 (结算 {out['cost_settle'][REP].sum():.0f}, "
263         f"紧急 {out['cost_emg'][REP].sum():.0f}), {time.time() - t0:.1f}s")
264     return out
265
266
267 def _run_strategy(job):
268     """并行 worker: 跑单个策略配置。"""
269     name, at, cv, load_fc, pv0, pvT, buf0, bufT, errT = job
270     return name, simulate_q3(load_fc, pv0, pvT, buf0, bufT, at, exec_mode="rolling",
271                             plan_mode="saa", errT=errT, coeff_v=cv)
272
273
274 def main():
275     load_fc, pv0, pvT, off0, mean7 = build_profiles()
276     # 光伏预测精度: 官方预报 vs 历史均值 vs 逆方差组合 (正式期)
277     rep = slice(31, 365)
278     mae_off = np.abs((off0[rep] - PV[rep])).mean()
279     mae_m7 = np.abs((mean7[rep] - PV[rep])).mean()
280     mae_bl = np.abs((pv0[rep] - PV[rep])).mean()
281     log(f"光伏预测精度(0:00信息): 官方预报 MAE={mae_off:.1f} vs 历史均值 {mae_m7:.1f} "
282         f"vs 逆方差组合 {mae_bl:.1f} kW")
283     buf0, bufT = build_buffers(load_fc, pv0, pvT)
284     errT = build_errT(load_fc, pvT)
285
286     # ---- 是否需要其他时刻预报: 策略对比 (SAA 计划层 + 场景化调整层, 多进程并行) ----
287     strategies = {
288         "A_不调整": [],
289         "B_仅6时": [6],
290         "C_仅12时": [12],
291         "D_仅18时": [18],
292         "E_全调整(主)": [6, 12, 18],
293     }
294
295     jobs = {name: (name, at, 0.5, load_fc, pv0, pvT, buf0, bufT, errT)
296             for name, at in strategies.items()}
297     jobs["口径B"] = ("口径B", [6, 12, 18], 1.5, load_fc, pv0, pvT, buf0, bufT, errT)
298     from concurrent.futures import ProcessPoolExecutor
299     with ProcessPoolExecutor(max_workers=min(6, len(jobs))) as ex:
300         results = dict(ex.map(_run_strategy, jobs.values()))
301     sims = {k: v for k, v in results.items() if k != "口径B"}
302
303     print("\n策略 | 回测期总费用(元) | 偏差结算费 | 紧急购电费 | 较不调整节省")
304     base = sims["A_不调整"]["cost_total"][REP].sum()
305     for name, s in sims.items():
306         tot = s["cost_total"][REP].sum()

```

```

307     print(f"{name:<14}{tot:>15,.0f}-{s['cost_settle'][REP].sum():>13,.0f}"
308           f"{s['cost_emg'][REP].sum():>13,.0f}-{base - tot:>13,.0f}")
309
310     main_sim = sims["E_全调整(主)"]
311     sim_B = results["口径B"]                # 结算口径 B 敏感性（并行已跑完）
312
313     np.savez(RESULTS / "q3_main_sim.npz",
314             **{k: main_sim[k] for k in ["g_plan", "a_final", "c", "d", "spill",
315                                         "emg", "soc_start", "cost_settle", "cost_emg"]})
316
317     key_numbers(
318         q3_pv_mae_official=round(float(mae_off), 1), q3_pv_mae_self=round(float(mae_m7), 1),
319         q3_pv_mae_blend=round(float(mae_bl), 1),
320         q3_cost_noAdj=round(float(sims["A_不调整"]["cost_total"][REP].sum()), 0),
321         q3_cost_adj6=round(float(sims["B_仅6时"]["cost_total"][REP].sum()), 0),
322         q3_cost_adj12=round(float(sims["C_仅12时"]["cost_total"][REP].sum()), 0),
323         q3_cost_adj18=round(float(sims["D_仅18时"]["cost_total"][REP].sum()), 0),
324         q3_cost_main=round(float(main_sim["cost_total"][REP].sum()), 0),
325         q3_main_settle=round(float(main_sim["cost_settle"][REP].sum()), 0),
326         q3_main_emg=round(float(main_sim["cost_emg"][REP].sum()), 0),
327         q3_cost_readingB=round(float(sim_B["cost_total"][REP].sum()), 0),
328         q3_readingB_settle=round(float(sim_B["cost_settle"][REP].sum()), 0),
329         q3_readingB_emg=round(float(sim_B["cost_emg"][REP].sum()), 0),
330         q3_emg_total_kwh=round(float(main_sim["emg"][REP].sum()), 0),
331         q3_adj_total_kwh=round(float(np.abs(main_sim["a_final"][REP] -
332                                         main_sim["g_plan"][REP]).sum()), 0),
333     )
334     log("Q3 主策略与关键数值已保存")
335     return main_sim
336
337 if __name__ == "__main__":
338     main()

```

Listing 8 问题三结果输出 q3_results.py

```

1  # -*- coding: utf-8 -*-
2  """问题3 结果输出: result3.xlsx、四指定日明细、论文图表。"""
3  import sys, json
4  from pathlib import Path
5  sys.path.insert(0, str(Path(__file__).parent))
6  import numpy as np
7  import pandas as pd
8  from openpyxl import load_workbook
9  from common import load_all, key_numbers, log, RESULTS, FIGURES, N, DT, DAYS
10 from q2_simulate import PRICE_A1, REP
11 from xlsx_writer import (write_plan_sheet, write_battery_sheet, write_emergency_sheet,
12                          spec_date_detail, soc_boundaries)
13
14 A1, LOAD, PV, PRICE4, FC3 = load_all()
15 sim = np.load(RESULTS / "q3_main_sim.npz")
16 G, Afin, C, D_, EM = sim["g_plan"], sim["a_final"], sim["c"], sim["d"], sim["emg"]
17 soc_start, cost_settle, cost_emg = sim["soc_start"], sim["cost_settle"], sim["cost_emg"]
18
19 # ---- result3.xlsx: 计划 / 调整 / 充放电 / 紧急 ----
20 tpl = r"C:\Users\26517\Documents\CUMCM\C题\附件\附件5\result3.xlsx"
21 wb = load_workbook(tpl)
22 # 计划购电量 sheet 的全天购电费 = 0:00 计划的 p * g

```

```

23 plan_cost = np.array([float(PRICE_A1 @ G[d]) for d in range(DAYS)])
24 write_plan_sheet(wb["计划购电量"], G, plan_cost)
25 # 调整购电量 sheet: 最终生效量 a_final + 偏差结算费 (口径A)
26 write_plan_sheet(wb["调整购电量"], Afin, cost_settle)
27 soc_0, soc_24 = soc_boundaries(C, D_, soc_start)
28 write_battery_sheet(wb["充放电量"], C, D_, soc_0, soc_24)
29 n_emg_days = write_emergency_sheet(wb["紧急购电量"], EM)
30 out = RESULTS / "result3.xlsx"
31 wb.save(out)
32 log(f"result3.xlsx 已写出 ({n_emg_days} 个紧急购电日)")
33
34 # ---- 四指定日明细 ----
35 def cost_by_day(D):
36     return dict(total_g=round(float(Afin[D].sum()), 2),
37                 cost_settle=round(float(cost_settle[D]), 2),
38                 cost_emg=round(float(cost_emg[D]), 2),
39                 cost_total=round(float(cost_settle[D] + cost_emg[D]), 2),
40                 adj_qty=round(float(np.abs(Afin[D] - G[D]).sum()), 2))
41
42 detail = spec_date_detail(G, Afin, C, D_, EM, soc_start, cost_by_day)
43 for s, v in detail.items():
44     print(f"{s}: 最终购电 {v['total_g']:.0f} kWh, 结算费 {v['cost_settle']:.0f}, "
45           f"紧急 {v['cost_emg']:.0f}, 调整量 {v['adj_qty']:.0f} kWh, "
46           f"SOC {v['soc_0']:.0f}->{v['soc_24']:.0f}, 紧急时段 {len(v['t3_emergency'])}")
47 key_numbers(q3_spec_dates=detail)
48
49 # ---- 图 ----
50 import matplotlib
51 matplotlib.use("Agg")
52 import matplotlib.pyplot as plt
53 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
54 plt.rcParams["axes.unicode_minus"] = False
55
56 kn = json.loads((RESULTS / "key_numbers.json").read_text(encoding="utf-8"))
57 # 图A: 调整策略对比
58 names = ["不调整", "仅6时", "仅12时", "仅18时", "全调整\n(主策略)"]
59 vals = [kn["q3_cost_noAdj"], kn["q3_cost_adj6"], kn["q3_cost_adj12"],
60         kn["q3_cost_adj18"], kn["q3_cost_main"]]
61 fig, ax = plt.subplots(figsize=(8.2, 4.4))
62 bars = ax.bar(np.arange(5), np.array(vals) / 1e4, 0.56,
63              color=["#95a5a6", "#7fb3d5", "#7fb3d5", "#7fb3d5", "#c0392b"])
64 for i, v in enumerate(vals):
65     ax.text(i, v / 1e4 + 1.2, f"{v/1e4:.1f}", ha="center", fontsize=10)
66 ax.set_xticks(np.arange(5)); ax.set_xticklabels(names, fontsize=9.5)
67 ax.set_ylabel("回溯期总费用 (万元)")
68 ax.set_title("问题三: 引入其他时刻预报的调整价值")
69 fig.tight_layout(); fig.savefig(FIGURES / "fig_q3_adj_cmp.png", dpi=300); plt.close(fig)
70
71 # 图B: 典型调整日 (|a-g| 最大的日期): 计划 vs 调整 vs 光伏实际
72 adj_qty = np.abs(Afin - G).sum(axis=1); adj_qty[:31] = 0
73 Dmax = int(np.argmax(adj_qty))
74 lab_time = [(k + 1) * 10 / 60 for k in range(N)]
75 fig, axes = plt.subplots(2, 1, figsize=(9.5, 6.2), sharex=True,
76                          gridspec_kw={"height_ratios": [3, 1.6]})
77 ax = axes[0]
78 ax.plot(lab_time, PRICE_A1, color="#888888", lw=1.2, label="电价 (右轴)")
79 ax.set_ylabel("电价 (元/kWh)", color="#888888"); ax.tick_params(axis="y", labelcolor="#888888")

```

```

80 ax2 = ax.twinx()
81 ax2.step(lab_time, G[Dmax], where="post", color="#c0392b", lw=1.5, label="0:00 计划购电")
82 ax2.step(lab_time, Afin[Dmax], where="post", color="#8e44ad", lw=1.5, ls="--", label="调整后购电")
83 ax2.plot(lab_time, PV[Dmax] * DT, color="#f39c12", lw=1.2, label="光伏实际 (×Δt)")
84 for T, lab in [(6, "6时预报"), (12, "12时预报"), (18, "18时预报")]:
85     ax2.axvline(T, color="#555", ls=":", lw=0.9)
86 ax2.axhline(0, color="k", lw=0.5); ax2.set_ylabel("电量 (kWh/10min)")
87 h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
88 ax.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=8.5, ncol=4, framealpha=0.9)
89 ax.set_title(f"问题三: 滚动调整示例 ({pd.Timestamp('2025-01-01') + pd.Timedelta(days=Dmax):%Y-%m-%d}, "
90             f"调整总量 {adj_qty[Dmax]:.0f} kWh)")
91 soc_tr = soc_start[Dmax] + np.cumsum(0.9 * C[Dmax] - D_[Dmax] / 0.9)
92 axes[1].plot(lab_time, soc_tr, color="#6c3483", lw=1.8)
93 axes[1].axhline(1200, color="r", ls="--", lw=0.8); axes[1].axhline(10800, color="r", ls="--", lw=0.8)
94 axes[1].set_ylabel("储能电量 (kWh)"); axes[1].set_xlabel("时刻 (h)"); axes[1].set_ylim(0, 12000)
95 fig.tight_layout(); fig.savefig(FIGURES / "fig_q3_adj_day.png", dpi=300); plt.close(fig)
96 log(f"图已输出: fig_q3_adj_cmp / fig_q3_adj_day({pd.Timestamp('2025-01-01') + pd.Timedelta(days=Dmax):%m-%d})")

```

Listing 9 问题四仿真程序 q4_simulate.py

```

1  # -*- coding: utf-8 -*-
2  """问题4: 波动电价 (附件4) 下重新计算问题2与问题3。
3
4  信息结构: 附件4 电价按日前已知处理 (日前市场出清价提前发布的惯例; 若视为实时未知,
5  则需引入电价预测模型, 本文在模型评价中讨论该扩展)。负载/光伏预测、缓冲与执行
6  机制与问题二/三完全一致, 仅电价路径替换为附件4。
7  退化检验: 将附件4换回附件1固定电价, 脚本退化问题二/三的原模型 (代码路径一致)。
8  """
9  import sys, time
10 from pathlib import Path
11 sys.path.insert(0, str(Path(__file__).parent))
12 import numpy as np
13 from common import load_all, key_numbers, log, RESULTS, N, DT, DAYS
14 from q2_simulate import simulate, REP
15 from q3_simulate import build_profiles, build_buffers, simulate_q3
16
17 A1, LOAD, PV, PRICE4, FC3 = load_all()
18
19
20 def _run_saa(job):
21     S, load_fc, pv_fc = job
22     from q2_saa import simulate_saa
23     return S, simulate_saa(load_fc, pv_fc, PRICE4, S=S, exec_mode="rolling")
24
25
26 def _run_q3(job):
27     at, lf3, pv0, pvT, buf0, bufT, errT = job
28     return simulate_q3(lf3, pv0, pvT, buf0, bufT, at, exec_mode="rolling",
29                       price_days=PRICE4, plan_mode="saa", errT=errT)
30
31
32 def main():
33     fc = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")
34     load_fc, pv_fc = fc["load_fc"], fc["pv_fc"]
35
36     log("【P4-2】问题二模型 (SAA 计划层) × 波动电价")
37     lf3, pv0, pvT, _, _ = build_profiles()

```

```

38     buf0, bufT = build_buffers(lf3, pv0, pvT)
39     from q3_simulate import build_errT
40     errT = build_errT(lf3, pvT)
41
42     from concurrent.futures import ProcessPoolExecutor
43     with ProcessPoolExecutor(max_workers=4) as ex:
44         futs = {S: ex.submit(_run_saa, (S, load_fc, pv_fc)) for S in [10, 14]}
45         futs["main"] = ex.submit(_run_q3, ([6, 12, 18], lf3, pv0, pvT, buf0, bufT, errT))
46         futs["noAdj"] = ex.submit(_run_q3, ([], lf3, pv0, pvT, buf0, bufT, errT))
47         grid = {S: float(futs[S].result()[1]["cost_total"][REP].sum()) for S in [10, 14]}
48         q4_2 = futs[10].result()[1]
49         q4_3 = futs["main"].result()
50         q4_3_noAdj = futs["noAdj"].result()
51
52     # ---- 波动电价下的基准 ----
53     base4 = float(sum(PRICE4[d] @ np.maximum(LOAD[d] * DT - PV[d] * DT, 0)
54                     for d in range(31, 365)))
55     oracle4 = simulate(LOAD, PV, PRICE4, buffer_q=None, exec_mode="greedy")
56
57     print("\n波动电价（附件4）回溯期对比：")
58     print(f" 无储能基准          {base4:>14,.0f} 元")
59     print(f" P4-2（问题二模型） {q4_2['cost_total'][REP].sum():>14,.0f} 元 "
60           f"（紧急 {q4_2['cost_emg'][REP].sum():,.0f}）")
61     print(f" P4-3 不调整          {q4_3_noAdj['cost_total'][REP].sum():>14,.0f} 元")
62     print(f" P4-3 全调整(主)    {q4_3['cost_total'][REP].sum():>14,.0f} 元 "
63           f"（结算 {q4_3['cost_settle'][REP].sum():,.0f}, 紧急 {q4_3['cost_emg'][REP].sum():,.0f}）")
64     print(f" 理想先知          {oracle4['cost_total'][REP].sum():>14,.0f} 元")
65
66     np.savez(RESULTS / "q4_2_sim.npz",
67             **{k: q4_2[k] for k in ["g_plan", "c", "d", "spill", "emg",
68                                     "soc_start", "cost_plan", "cost_emg"]})
69     np.savez(RESULTS / "q4_3_sim.npz",
70             **{k: q4_3[k] for k in ["g_plan", "a_final", "c", "d", "spill",
71                                     "emg", "soc_start", "cost_settle", "cost_emg"]})
72     key_numbers(
73         q4_cost_baseline=round(base4, 0),
74         q4_2_cost_main=round(float(q4_2["cost_total"][REP].sum()), 0),
75         q4_2_cost_emg=round(float(q4_2["cost_emg"][REP].sum()), 0),
76         q4_2_cost_oracle=round(float(oracle4["cost_total"][REP].sum()), 0),
77         q4_3_cost_noAdj=round(float(q4_3_noAdj["cost_total"][REP].sum()), 0),
78         q4_3_cost_main=round(float(q4_3["cost_total"][REP].sum()), 0),
79         q4_3_main_settle=round(float(q4_3["cost_settle"][REP].sum()), 0),
80         q4_3_main_emg=round(float(q4_3["cost_emg"][REP].sum()), 0),
81         q4_3_emg_total_kwh=round(float(q4_3["emg"][REP].sum()), 0),
82         q4_2_sens_S={str(k): round(v, 0) for k, v in grid.items()},
83         q4_price_range=[round(float(PRICE4.min()), 4), round(float(PRICE4.max()), 4)],
84     )
85     log("P4 仿真与关键数值已保存")
86
87
88 if __name__ == "__main__":
89     main()

```

Listing 10 问题四结果输出 q4_results.py

```

1 # -*- coding: utf-8 -*-
2 """问题4 结果输出: result4-2.xlsx（问题二格式）与 result4-3.xlsx（问题三格式）、图表。"""

```

```

3 import sys, json
4 from pathlib import Path
5 sys.path.insert(0, str(Path(__file__).parent))
6 import numpy as np
7 import pandas as pd
8 from openpyxl import load_workbook
9 from common import load_all, key_numbers, log, RESULTS, FIGURES, N, DT, DAYS
10 from xlsx_writer import (write_plan_sheet, write_battery_sheet, write_emergency_sheet,
11                          spec_date_detail, soc_boundaries)
12
13 A1, LOAD, PV, PRICE4, FC3 = load_all()
14
15 # ---- result4-2.xlsx (问题二格式: 计划/充放电/紧急) ----
16 s2 = np.load(RESULTS / "q4_2_sim.npz")
17 tp12 = r"C:\Users\26517\Documents\CUMCM\C题\附件\附件5\result4-2.xlsx"
18 wb = load_workbook(tp12)
19 write_plan_sheet(wb["计划购电量"], s2["g_plan"], s2["cost_plan"])
20 soc_0, soc_24 = soc_boundaries(s2["c"], s2["d"], s2["soc_start"])
21 write_battery_sheet(wb["充放电电量"], s2["c"], s2["d"], soc_0, soc_24)
22 n2 = write_emergency_sheet(wb["紧急购电量"], s2["emg"])
23 wb.save(RESULTS / "result4-2.xlsx")
24 log(f"result4-2.xlsx 已写出 ({n2} 个紧急购电日)")
25
26 # ---- result4-3.xlsx (问题三格式: 计划/调整/充放电/紧急) ----
27 s3 = np.load(RESULTS / "q4_3_sim.npz")
28 tp13 = r"C:\Users\26517\Documents\CUMCM\C题\附件\附件5\result4-3.xlsx"
29 wb = load_workbook(tp13)
30 plan_cost = np.array([float(PRICE4[d] @ s3["g_plan"][d]) for d in range(DAYS)])
31 write_plan_sheet(wb["计划购电量"], s3["g_plan"], plan_cost)
32 write_plan_sheet(wb["调整购电量"], s3["a_final"], s3["cost_settle"])
33 soc_0b, soc_24b = soc_boundaries(s3["c"], s3["d"], s3["soc_start"])
34 write_battery_sheet(wb["充放电电量"], s3["c"], s3["d"], soc_0b, soc_24b)
35 n3 = write_emergency_sheet(wb["紧急购电量"], s3["emg"])
36 wb.save(RESULTS / "result4-3.xlsx")
37 log(f"result4-3.xlsx 已写出 ({n3} 个紧急购电日)")
38
39 # ---- 四指定日明细 (P4-3) ----
40 def cost_by_day(D):
41     return dict(total_g=round(float(s3["a_final"][D].sum()), 2),
42                 cost_settle=round(float(s3["cost_settle"][D]), 2),
43                 cost_emg=round(float(s3["cost_emg"][D]), 2),
44                 cost_total=round(float(s3["cost_settle"][D] + s3["cost_emg"][D]), 2),
45                 adj_qty=round(float(np.abs(s3["a_final"][D] - s3["g_plan"][D]).sum()), 2))
46
47 detail = spec_date_detail(s3["g_plan"], s3["a_final"], s3["c"], s3["d"], s3["emg"],
48                          s3["soc_start"], cost_by_day)
49 for s, v in detail.items():
50     print(f"P4-3 {s}: 最终购电 {v['total_g']:.0f} kWh, 结算 {v['cost_settle']:.0f}, "
51           f"紧急 {v['cost_emg']:.0f}, 调整量 {v['adj_qty']:.0f}, 紧急时段 {len(v['t3_emergency'])}")
52 key_numbers(q4_3_spec_dates=detail)
53
54 # ---- 图: 固定电价 vs 波动电价的策略费用对比 ----
55 import matplotlib
56 matplotlib.use("Agg")
57 import matplotlib.pyplot as plt
58 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
59 plt.rcParams["axes.unicode_minus"] = False

```

```

60 kn = json.loads((RESULTS / "key_numbers.json").read_text(encoding="utf-8"))
61
62 labels = ["无储能基准", "问题二模型\n(P4-2)", "问题三模型\n不调整", "问题三模型\n全调整(P4-3)", "理想先知"]
63 fixed = [kn["q2_cost_baseline"], kn["q2_cost_main"], kn["q3_cost_noAdj"],
64          kn["q3_cost_main"], kn["q2_cost_oracle"]]
65 fluct = [kn["q4_cost_baseline"], kn["q4_2_cost_main"], kn["q4_3_cost_noAdj"],
66          kn["q4_3_cost_main"], kn["q4_2_cost_oracle"]]
67 x = np.arange(5); w = 0.36
68 fig, ax = plt.subplots(figsize=(9.2, 4.6))
69 ax.bar(x - w / 2, np.array(fixed) / 1e4, w, label="固定电价（附件1）", color="#5d6d7e")
70 ax.bar(x + w / 2, np.array(fluct) / 1e4, w, label="波动电价（附件4）", color="#c0392b")
71 for xi, v in zip(x - w / 2, fixed):
72     ax.text(xi, v / 1e4 + 1.0, f"{v/1e4:.0f}", ha="center", fontsize=8.5)
73 for xi, v in zip(x + w / 2, fluct):
74     ax.text(xi, v / 1e4 + 1.0, f"{v/1e4:.0f}", ha="center", fontsize=8.5)
75 ax.set_xticks(x); ax.set_xticklabels(labels, fontsize=9)
76 ax.set_ylabel("回溯期总费用（万元）")
77 ax.set_title("固定电价与波动电价下各策略总费用对比（2025.2.1 - 12.31）")
78 ax.legend()
79 fig.tight_layout(); fig.savefig(FIGURES / "fig_q4_cmp.png", dpi=300); plt.close(fig)
80 log("图已输出: fig_q4_cmp")

```

Listing 11 结果写出公共模块 `xlsx_writer.py`

```

1  # -*- coding: utf-8 -*-
2  """结果工作簿写出公共模块：计划/充放电/紧急购电三个 sheet 的统一填写。"""
3  import sys
4  from pathlib import Path
5  sys.path.insert(0, str(Path(__file__).parent))
6  import numpy as np
7  import pandas as pd
8  from common import load_all, N, ETA, SOC_INIT, DAYS
9  from q2_simulate import clock_day_view, emergency_intervals
10
11 BLOCKS = [(0, 24, "0:00-4:00"), (24, 48, "4:00-8:00"), (48, 72, "8:00-12:00"),
12           (72, 96, "12:00-16:00"), (96, 120, "16:00-20:00"), (120, 144, "20:00-24:00")]
13 REP_DATES = pd.date_range("2025-01-01", periods=DAYS)[31:365]
14
15
16 def soc_boundaries(C, D_, soc_start):
17     """钟表日 0:00 / 24:00 储电量。SOC(D 0:00) = 计划日D起始SOC - 计划日D-1末槽变动。"""
18     soc_0 = np.zeros(DAYS); soc_24 = np.zeros(DAYS)
19     soc_0[0] = SOC_INIT
20     for D in range(1, DAYS):
21         soc_0[D] = soc_start[D] - (ETA * C[D - 1, 143] - D_[D - 1, 143] / ETA)
22     soc_24 = np.roll(soc_0, -1)
23     soc_24[364] = soc_start[364] + float(np.sum(ETA * C[364, :143] - D_[364, :143] / ETA))
24     return soc_0, soc_24
25
26
27 def write_plan_sheet(ws, g_plan, cost_col, dates=REP_DATES):
28     """计划购电量/调整购电量 sheet: 144 槽 + 全天购电量 + 全天购电费 (cost_col 为对应费用)。"""
29     hdr = [ws.cell(row=1, column=c).value for c in range(1, 148)]
30     assert hdr[1] == "0:10-0:20" and hdr[144] == "0:00-0:10+1", f"列标签异常: {hdr[1]}, {hdr[144]}"
31     ws.delete_rows(2, ws.max_row)
32     for i, D in enumerate(range(31, 365)):
33         r = i + 2

```

```

34     ws.cell(row=r, column=1, value=dates[i].to_pydatetime())
35     for k in range(N):
36         ws.cell(row=r, column=2 + k, value=round(float(g_plan[D, k]), 4))
37     ws.cell(row=r, column=146, value=round(float(g_plan[D].sum()), 2))
38     ws.cell(row=r, column=147, value=round(float(cost_col[D]), 2))
39
40
41 def write_battery_sheet(ws, C, D_, soc_0, soc_24, dates=REP_DATES):
42     c_ck = clock_day_view(C); d_ck = clock_day_view(D_)
43     ws.delete_rows(2, ws.max_row)
44     r = 2
45     for i, D in enumerate(range(31, 365)):
46         for b, (a, bnd, lab) in enumerate(BLOCKS):
47             ws.cell(row=r, column=1, value=dates[i].to_pydatetime() if b == 0 else None)
48             ws.cell(row=r, column=2, value=lab)
49             ws.cell(row=r, column=3, value=round(float(c_ck[D, a:bnd].sum()), 4))
50             ws.cell(row=r, column=4, value=round(float(d_ck[D, a:bnd].sum()), 4))
51             if b == 0:
52                 ws.cell(row=r, column=5, value="0:00")
53                 ws.cell(row=r, column=6, value=round(float(soc_0[D]), 2))
54             elif b == 1:
55                 ws.cell(row=r, column=5, value="24:00")
56                 ws.cell(row=r, column=6, value=round(float(soc_24[D]), 2))
57             r += 1
58
59
60 def write_emergency_sheet(ws, emg, dates=REP_DATES):
61     emg_ck = clock_day_view(emg)
62     ws.delete_rows(2, ws.max_row)
63     r = 2
64     n_days = 0
65     for D in range(31, 365):
66         ivs = emergency_intervals(emg_ck, D)
67         if not ivs:
68             continue
69         n_days += 1
70         for j, (lab, q) in enumerate(ivs):
71             ws.cell(row=r, column=1, value=dates[D - 31].to_pydatetime() if j == 0 else None)
72             ws.cell(row=r, column=2, value=lab)
73             ws.cell(row=r, column=3, value=round(q, 2))
74             r += 1
75     return n_days
76
77
78 def spec_date_detail(g_plan, a_final, C, D_, EM, soc_start, cost_by_day):
79     """四指定日明细（论文表用）。cost_by_day: dict(日期索引 -> 费用分解)"""
80     c_ck = clock_day_view(C); d_ck = clock_day_view(D_); emg_ck = clock_day_view(EM)
81     soc_0, soc_24 = soc_boundaries(C, D_, soc_start)
82     detail = {}
83     for s in ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]:
84         D = (pd.Timestamp(s) - pd.Timestamp("2025-01-01")).days
85         t1 = {f"{(k + 1) * 10 // 60:02d}::{(k + 1) * 10 % 60:02d}":
86             round(float((g_plan if a_final is None else a_final)[D, k]), 2)
87             for k in [59, 71, 83, 95, 107, 119]}
88         t2 = {lab: [round(float(c_ck[D, a:bnd].sum()), 2), round(float(d_ck[D, a:bnd].sum()), 2)]
89             for a, bnd, lab in BLOCKS}
90         detail[s] = dict(day_index=D, t1_slots=t1, t2_blocks=t2,

```

```

91         t3_emergency=emergency_intervals(emg_ck, D),
92         soc_0=round(float(soc_0[D]), 2), soc_24=round(float(soc_24[D]), 2),
93         **cost_by_day(D))
94     return detail

```

Listing 12 灵敏度分析 sens_analysis.py

```

1  # -*- coding: utf-8 -*-
2  """全局灵敏度分析：储能效率、紧急购电倍数、SOC 安全区间。
3
4  - P1: 确定性日循环 LP 对 / SOC 区间的敏感性（毫秒级）
5  - P2: 主策略（贪心执行）对 / SOC 区间 / 紧急倍数的一阶敏感性
6      紧急倍数 m 下同时重选报童分位数 q=(m-1)/m（理论最优），报告"固定策略"与"重选q"两种口径
7  """
8  import sys, json
9  from pathlib import Path
10 sys.path.insert(0, str(Path(__file__).parent))
11 import numpy as np
12 import common
13 import q2_simulate
14 from q2_simulate import simulate, REP, PRICE_A1
15 from common import key_numbers, log, RESULTS, FIGURES, N, SOC_INIT
16
17 A1, LOAD, PV, PRICE4, FC3 = common.load_all()
18 price_c = np.roll(A1["price"], 1); load_c = np.roll(A1["load"], 1); pv_c = np.roll(A1["pv"], 1)
19 fc = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")
20 lf, pf = fc["load_fc"], fc["pv_fc"]
21
22 # ---- P1: 效率与 SOC 区间扫描 ----
23 def p1_cost(eta=None, smin=None, smax=None):
24     if eta is not None: common.ETA = eta
25     if smin is not None: common.SOC_MIN = smin
26     if smax is not None: common.SOC_MAX = smax
27     sol = common.solve_plan(price_c, load_c, pv_c, soc0=SOC_INIT, cyclic=True, validate=False)
28     common.ETA, common.SOC_MIN, common.SOC_MAX = 0.9, 1200.0, 10800.0
29     assert sol["status"] == 0
30     return sol["cost_plan"]
31
32 p1_eta = {}
33 for eta in [0.85, 0.875, 0.9, 0.925, 0.95]:
34     p1_eta[eta] = p1_cost(eta=eta)
35 p1_soc = {}
36 for smin, smax in [(1200, 10800), (960, 10800), (1440, 10800), (1200, 8640), (1200, 12000)]:
37     p1_soc[(smin, smax)] = p1_cost(smin=smin, smax=smax)
38 print("P1 效率敏感性:", {k: round(v, 2) for k, v in p1_eta.items()})
39 print("P1 SOC区间敏感性:", {str(k): round(v, 2) for k, v in p1_soc.items()})
40
41 # ---- P2: 效率与 SOC 区间（贪心执行，策略结构不变） ----
42 def p2_total(eta=None, smin=None, smax=None, q=0.7):
43     saved = (common.ETA, common.SOC_MIN, common.SOC_MAX,
44             q2_simulate.ETA, q2_simulate.SOC_MIN, q2_simulate.SOC_MAX)
45     if eta is not None: common.ETA = q2_simulate.ETA = eta
46     if smin is not None: common.SOC_MIN = q2_simulate.SOC_MIN = smin
47     if smax is not None: common.SOC_MAX = q2_simulate.SOC_MAX = smax
48     s = simulate(lf, pf, PRICE_A1[None, :], buffer_q=q, exec_mode="greedy")
49     (common.ETA, common.SOC_MIN, common.SOC_MAX,
50     q2_simulate.ETA, q2_simulate.SOC_MIN, q2_simulate.SOC_MAX) = saved

```

```

51     return float(s["cost_total"][REP].sum())
52
53 p2_eta = {eta: p2_total(eta=eta) for eta in [0.85, 0.9, 0.95]}
54 p2_soc = {(960, 10800): p2_total(smin=960), (1440, 10800): p2_total(smin=1440),
55           (1200, 8640): p2_total(smax=8640), (1200, 12000): p2_total(smax=12000)}
56 print("P2 效率敏感性:", {k: round(v / 1e4, 1) for k, v in p2_eta.items()}, "万元")
57 print("P2 SOC区间敏感性:", {str(k): round(v / 1e4, 1) for k, v in p2_soc.items()}, "万元")
58
59 # ---- P2: 紧急倍数 m (固定策略 vs 重选 q=(m-1)/m) ----
60 p2_emerg = {}
61 for m in [3, 4, 5, 6, 8]:
62     q_opt = (m - 1) / m
63     s_fix = simulate(lf, pf, PRICE_A1[None, :], buffer_q=0.7, exec_mode="greedy")
64     fix_total = float((s_fix["cost_plan"][REP] + s_fix["cost_emg"][REP] / 5.0 * m).sum())
65     s_re = simulate(lf, pf, PRICE_A1[None, :], buffer_q=q_opt, exec_mode="greedy")
66     re_total = float((s_re["cost_plan"][REP] + s_re["cost_emg"][REP] / 5.0 * m).sum())
67     p2_emerg[m] = (fix_total, re_total)
68     print(f" 紧急倍数 m={m}: 固定q=0.7 {fix_total/1e4:.1f} 万, 重选q={q_opt:.2f} {re_total/1e4:.1f} 万")
69
70 key_numbers(
71     sens_p1_eta={str(k): round(v, 2) for k, v in p1_eta.items()},
72     sens_p1_soc={str(k): round(v, 2) for k, v in p1_soc.items()},
73     sens_p2_eta={str(k): round(v, 0) for k, v in p2_eta.items()},
74     sens_p2_soc={str(k): round(v, 0) for k, v in p2_soc.items()},
75     sens_p2_emerg={str(m): [round(a, 0), round(b, 0)] for m, (a, b) in p2_emerg.items()},
76 )
77
78 # ---- 图: 三联灵敏度 ----
79 import matplotlib
80 matplotlib.use("Agg")
81 import matplotlib.pyplot as plt
82 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
83 plt.rcParams["axes.unicode_minus"] = False
84 fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.9))
85 ax = axes[0]
86 ks = sorted(p1_eta); ax.plot([k * 100 for k in ks], [p1_eta[k] for k in ks], "o-", color="#2c3e50")
87 ax.axvline(90, color="#c0392b", ls="--", lw=1)
88 ax.set_xlabel("充放电效率 (%)"); ax.set_ylabel("问题一日电购电费 (元)")
89 ax.set_title("(a) 效率敏感性 (问题一)")
90 ax = axes[1]
91 ks = sorted(p2_eta); ax.plot([k * 100 for k in ks], [p2_eta[k] / 1e4 for k in ks], "o-", color="#2c3e50")
92 ax.axvline(90, color="#c0392b", ls="--", lw=1)
93 ax.set_xlabel("充放电效率 (%)"); ax.set_ylabel("问题二总费用 (万元)")
94 ax.set_title("(b) 效率敏感性 (问题二)")
95 ax = axes[2]
96 ms = sorted(p2_emerg)
97 ax.plot(ms, [p2_emerg[m][0] / 1e4 for m in ms], "s--", color="#7f8c8d", label="固定 q=0.7")
98 ax.plot(ms, [p2_emerg[m][1] / 1e4 for m in ms], "o-", color="#c0392b", label="按理论重选 q")
99 ax.axvline(5, color="#888", ls=":", lw=1)
100 ax.set_xlabel("紧急购电价格倍数 m"); ax.set_ylabel("问题二总费用 (万元)")
101 ax.set_title("(c) 紧急倍数敏感性 (问题二)"); ax.legend(fontsize=8.5)
102 fig.tight_layout(); fig.savefig(FIGURES / "fig_sens.png", dpi=300); plt.close(fig)
103 log("灵敏度分析与图已输出")

```

Listing 13 图表风格模块 figstyle.py

```

1 # -*- coding: utf-8 -*-

```

```

2  """论文图表统一风格 v4 (Okabe-Ito 色板, 按版心实际尺寸出图, 程序自检闭环)。
3
4  规范要点:
5  - 无图内总标题 (LaTeX 图注即标题); 子图编号不烧入图片, 由图注按阅读顺序描述
6  - 禁用双 Y 轴; 柱状图纵轴一律从零开始; 不放 "+x%" 式简陋标注
7  - 图例优先放面板上方外侧 (legend_above), 杜绝压数据
8  - 每张图过 visual_qa.audit_layout 自检 (缺字=FAIL 即中止; 裁切/重叠=WARN 打印)
9  - 宽度 = 论文 \textwidth(16cm=6.299in) * includegraphics 的 width 比例, 印刷 1:1 不缩放
10 """
11 import sys
12 from pathlib import Path
13
14 import matplotlib
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17
18 sys.path.insert(0, r"C:\Users\26517\Documents\CUMCM\skill\scipilot-figure-skill\scripts")
19 from visual_qa import audit_layout, print_report # noqa: E402
20
21 TEXTW = 6.299 # \textwidth = 21cm - 2*2.5cm = 16cm = 6.299in
22
23 C = { # Okabe-Ito 色盲安全色板
24     "red": "#D55E00", "blue": "#0072B2", "green": "#009E73", "orange": "#E69F00",
25     "purple": "#CC79A7", "sky": "#56B4E9", "yellow": "#F0E442", "black": "#000000",
26     "grey": "#7F7F7F", "dark": "#2C3E50",
27 }
28
29
30 def tw(frac):
31     """includegraphics width 比例 → 设计宽度 (英寸), 字体大小即印刷大小。"""
32     return TEXTW * frac
33
34
35 def setup(small=False):
36     """统一 rcParams。small=True 用于 0.48 版心以下的小图。"""
37     base = 9.0 if small else 9.5
38     plt.rcParams.update({
39         "font.sans-serif": ["SimHei", "Microsoft YaHei"],
40         "axes.unicode_minus": False,
41         "figure.dpi": 110, "savefig.dpi": 300,
42         "font.size": base,
43         "axes.labelsize": base + 0.5, "axes.titlesize": base + 0.5,
44         "xtick.labelsize": base - 1.0, "ytick.labelsize": base - 1.0,
45         "legend.fontsize": base - 1.0, "legend.frameon": False,
46         "axes.spines.top": False, "axes.spines.right": False,
47         "axes.grid": True, "axes.grid.axis": "y",
48         "grid.color": "#BBBBBB", "grid.alpha": 0.3, "grid.linewidth": 0.5,
49         "axes.axisbelow": True, "lines.linewidth": 1.6,
50         "figure.constrained_layout.use": False,
51     })
52
53
54 def panel_label(ax, s):
55     """子图标签不画在图上——由 LaTeX 图注按阅读顺序描述各面板。"""
56     pass
57
58

```

```

59 def legend_above(ax, ncol=None, fontsize=None, x=0.0):
60     """图例放面板上方外侧（无子图编号，从面板左缘开始），杜绝压数据。"""
61     handles, labels = ax.get_legend_handles_labels()
62     if not labels:
63         return
64     if ncol is None:
65         ncol = len(labels)
66     ax.legend(loc="lower left", bbox_to_anchor=(x, 1.02, 1.0 - x, 0.12),
67             ncol=ncol, frameon=False, fontsize=fontsize,
68             columnspacing=1.1, handlelength=1.7, borderaxespad=0.0)
69
70
71 def hour_axis(ax):
72     ax.set_xticks(range(0, 25, 4))
73     ax.set_xlim(0, 24)
74     ax.set_xlabel("时刻 (h) ")
75
76
77 def audit_save(fig, name, figures_dir):
78     """程序自检 → 保存 300dpi PNG。FAIL 即中止（不静默跳过），WARN 打印留痕。"""
79     issues = audit_layout(fig)
80     fails = [i for i in issues if i[0] == "FAIL"]
81     if issues:
82         print(f" [audit] {name}: {len(issues)} 项 (FAIL×{len(fails)}) ")
83         print_report(issues)
84     if fails:
85         plt.close(fig)
86         raise SystemExit(f"{name} 自检 FAIL, 中止")
87     out = Path(figures_dir) / name
88     fig.savefig(out, dpi=300, bbox_inches="tight", pad_inches=0.02)
89     plt.close(fig)
90     kb = out.stat().st_size // 1024
91     print(f" {name} ({kb} KB)")

```

Listing 14 图表生成程序 `restyle_figures.py`

```

1  # -*- coding: utf-8 -*-
2  """论文图表管线 v3 —— 遵循 scipilot-figure-skill 规范:
3  - 无图内总标题（LaTeX 图注即标题，避免双重图注）
4  - 禁用双 Y 轴：电价与电量拆为共享 x 轴的上下子图
5  - 按版心实际尺寸出图（全宽 6.3in），正文/刻度 8-9pt，不二次缩放
6  - Okabe-Ito 色盲安全色板 + 线型冗余编码
7  - 每图过 visual_qa.audit_layout 程序自检（缺字/裁切/刻度重叠），FAIL 即退出
8  """
9  import sys, json
10 from pathlib import Path
11 sys.path.insert(0, str(Path(__file__).parent))
12 sys.path.insert(0, r"C:\Users\26517\Documents\CUMCM\skill\scipilot-figure-skill\scripts")
13 import numpy as np
14 import pandas as pd
15 import matplotlib
16 matplotlib.use("Agg")
17 import matplotlib.pyplot as plt
18 from common import load_all, solve_plan, RESULTS, FIGURES, N, DT, ETA, SOC_INIT
19 from visual_qa import audit_layout, print_report
20
21 A1, LOAD, PV, PRICE4, FC3 = load_all()

```

```

22 KN = json.loads((RESULTS / "key_numbers.json").read_text(encoding="utf-8"))
23 T = np.arange(N) * 10 / 60.0          # 槽起点 (小时)
24 W = 10 / 60.0                        # 槽宽 (小时)
25
26 # Okabe-Ito 色盲安全色板
27 C = {"red": "#D55E00", "blue": "#0072B2", "green": "#009E73", "orange": "#E69F00",
28      "purple": "#CC79A7", "sky": "#56B4E9", "yellow": "#F0E442", "grey": "#7F7F7F",
29      "dark": "#2C3E50", "black": "#000000"}
30
31 plt.rcParams.update({
32     "font.sans-serif": ["SimHei", "Microsoft YaHei"],
33     "axes.unicode_minus": False,
34     "figure.dpi": 110, "savefig.dpi": 300,
35     "font.size": 9, "axes.labelsize": 9.5, "axes.titlesize": 10,
36     "xtick.labelsize": 8, "ytick.labelsize": 8,
37     "legend.fontsize": 8, "legend.frameon": False,
38     "axes.spines.top": False, "axes.spines.right": False,
39     "axes.grid": True, "axes.grid.axis": "y",
40     "grid.color": "#BBBBBB", "grid.alpha": 0.3, "grid.linewidth": 0.5,
41     "axes.axisbelow": True, "lines.linewidth": 1.5,
42 })
43 FW = 6.3          # 全宽 (0.98\textwidth)
44
45
46 def panel(ax, s):
47     ax.text(0.0, 1.03, s, transform=ax.transAxes, fontsize=9.5,
48            fontweight="bold", va="bottom")
49
50
51 def audit(fig, name):
52     issues = audit_layout(fig)
53     fails = [i for i in issues if i[0] == "FAIL"]
54     print(f" [audit] {name}: {len(issues)} 项"
55           + (" if not issues else f" (FAIL×{len(fails)}) ") )
56     if issues:
57         print_report(issues)
58     assert not fails, f"{name} 自检 FAIL, 中止"
59     fig.savefig(FIGURES / name, dpi=300, bbox_inches="tight")
60     plt.close(fig)
61
62
63 def hour_axis(ax):
64     ax.set_xticks(range(0, 25, 4))
65     ax.set_xlim(0, 24)
66     ax.set_xlabel("时刻 (h) ")
67
68
69 # ===== 1. 问题一: 三面板 (电价/边际成本 → 调度 → SOC) =====
70 def fig_q1():
71     price_c = np.roll(A1["price"], 1)
72     load_c = np.roll(A1["load"], 1)
73     pv_c = np.roll(A1["pv"], 1)
74     sol = solve_plan(price_c, load_c, pv_c, soc0=SOC_INIT, cyclic=True)
75     g, c, d, s = sol["g"], sol["c"], sol["d"], sol["s"]
76     mc = sol["res"].eqlin.marginals[N + 1: 2 * N + 1]
77
78     fig, axes = plt.subplots(3, 1, figsize=(FW, 6.4), sharex=True,

```

```

79         gridspec_kw={"height_ratios": [1.0, 1.75, 1.05],
80                        "hspace": 0.14})
81
82     ax = axes[0]
83     ax.step(T, price_c, where="post", color=C["grey"], lw=1.4, label="电价")
84     ax.step(T, mc, where="post", color=C["dark"], lw=1.6, ls="--",
85            label="边际供电成本（对偶）")
86     ax.set_ylabel("元/kWh")
87     ax.set_ylim(0, 1.55)
88     ax.legend(loc="upper left", ncol=2, columnspacing=1.2)
89     panel(ax, "(a)")
90
91     ax = axes[1]
92     ax.step(T, g, where="post", color=C["red"], lw=1.5, label="计划购电")
93     ax.bar(T, c, width=W, align="edge", color=C["blue"], alpha=0.8, label="充电")
94     ax.bar(T, -d, width=W, align="edge", color=C["green"], alpha=0.8, label="放电")
95     ax.axhline(0, color="k", lw=0.6)
96     ax.set_ylabel("kWh/10min")
97     ym = max(g.max(), c.max(), d.max()) * 1.18 + 50      # 动态上限防截断，留图例空间
98     ax.set_ylim(-ym, ym)
99     ax.legend(loc="upper left", ncol=3, columnspacing=1.2)
100    panel(ax, "(b)")
101
102    ax = axes[2]
103    ax.plot(T, s, color=C["purple"], lw=1.9)
104    ax.axhline(1200, color=C["red"], ls=":", lw=0.9)
105    ax.axhline(10800, color=C["red"], ls=":", lw=0.9)
106    ax.text(23.8, 1180, "电量下限", ha="right", va="top", fontsize=7.5, color=C["red"])
107    ax.text(23.8, 10830, "电量上限", ha="right", va="bottom", fontsize=7.5, color=C["red"])
108    ax.set_ylabel("储电量 (kWh)")
109    ax.set_ylim(0, 12200)
110    hour_axis(ax)
111    panel(ax, "(c)")
112    audit(fig, "fig_q1_strategy.png")
113
114    # ===== 2. 问题二：策略费用构成（堆叠柱） =====
115    def fig_q2cmp():
116        from q2_simulate import simulate, PRICE_A1, REP
117        fc = np.load(Path(__file__).parent / "cache" / "forecast_p2.npz")
118        lf, pf = fc["load_fc"], fc["pv_fc"]
119        s_naive = simulate(lf, pf, PRICE_A1[None, :], buffer_q=None, exec_mode="greedy")
120        s_greedy = simulate(lf, pf, PRICE_A1[None, :], buffer_q=0.7, exec_mode="greedy")
121        sim = np.load(RESULTS / "q2_main_sim.npz")
122        entries = [
123            ("无储能\n基准", KN["q2_cost_baseline"], 0.0),
124            ("朴素预测\n+贪心", float(s_naive["cost_plan"][REP].sum()),
125             float(s_naive["cost_emg"][REP].sum())),
126            ("报童缓冲\n+贪心", float(s_greedy["cost_plan"][REP].sum()),
127             float(s_greedy["cost_emg"][REP].sum())),
128            ("报童缓冲\n+滚动", 13103589.0, KN["q2_cost_buffer_rolling"] - 13103589.0),
129            ("光伏分位数\n裕量+滚动", KN["q2_e5_plan"], KN["q2_e5_emg"]),
130            ("随机规划\n+滚动（主）", float(sim["cost_plan"][REP].sum()),
131             float(sim["cost_emg"][REP].sum())),
132            ("理想先知", KN["q2_cost_oracle"], 0.0),
133        ]
134        names = [e[0] for e in entries]
135        plans = np.array([e[1] for e in entries]) / 1e4

```

```

136 emgs = np.array([e[2] for e in entries]) / 1e4
137 x = np.arange(len(names))
138 fig, ax = plt.subplots(figsize=(FW, 3.7))
139 ax.bar(x, plans, 0.62, color=C["blue"], label="计划购电费")
140 ax.bar(x, emgs, 0.62, bottom=plans, color=C["red"], label="紧急购电费 (5×)")
141 top = (plans + emgs)
142 for xi, p, e in zip(x, plans, emgs):
143     ax.text(xi, p + e + 2.2, f"{p + e:.1f}", ha="center", fontsize=8.5,
144             fontweight="bold")
145     if e > 0.05 * (plans.max() + emgs.max()):
146         ax.text(xi, p + e / 2, f"{e / (p + e) * 100:.0f}%", ha="center",
147                 fontsize=7.5, color="white", fontweight="bold")
148 ax.set_xticks(x)
149 ax.set_xticklabels(names, fontsize=8)
150 ax.set_ylabel("回测期总费用 (万元)")
151 ax.set_ylim(0, top.max() * 1.16)
152 ax.legend(loc="upper right")
153 audit(fig, "fig_q2_strategy_cmp.png")
154
155
156 # ===== 3. 问题二: 缓冲分位数敏感性 =====
157 def fig_q2sens():
158     sens = KN["q2_sens_q"]
159     qs = sorted(float(k) for k in sens)
160     vals = np.array([sens[str(q)] for q in qs]) / 1e4
161     fig, ax = plt.subplots(figsize=(4.9, 3.3))
162     ax.plot([q * 100 for q in qs], vals, "o-", color=C["dark"], markersize=5)
163     ax.axvspan(70, 80, color=C["sky"], alpha=0.22, label="平坦最优区 $[0.7,0.8]$" )
164     ax.axvline(80, color=C["red"], ls="--", lw=1.0, label="报童理论值 80%")
165     i = int(vals.argmin())
166     ax.plot(qs[i] * 100, vals[i], "o", color=C["red"], markersize=6, zorder=5)
167     ax.annotate(f"最低 {vals[i]:.1f} 万元", xy=(qs[i] * 100, vals[i]),
168               xytext=(qs[i] * 100 - 13, vals[i] + 2.4),
169               arrowprops=dict(arrowstyle="→", color=C["grey"], lw=0.9),
170               fontsize=8)
171     ax.set_xlabel("缓冲分位数 $q$ (%)")
172     ax.set_ylabel("回测期总费用 (万元)")
173     ax.set_ylim(vals.min() - 4, vals.max() + 6)
174     ax.legend(loc="upper right")
175     audit(fig, "fig_q2_sens_q.png")
176
177
178 # ===== 4. 问题二: 最严重日三面板 =====
179 def fig_q2day():
180     sim = np.load(RESULTS / "q2_main_sim.npz")
181     g, Cc, Dd, EM = sim["g_plan"], sim["c"], sim["d"], sim["emg"]
182     soc_st = sim["soc_start"]
183     emg_cost = sim["cost_emg"].copy(); emg_cost[:31] = 0
184     Dm = int(np.argmax(emg_cost))
185     dstr = (pd.Timestamp("2025-01-01") + pd.Timedelta(days=Dm)).strftime("%Y-%m-%d")
186     price = A1["price"]
187
188     fig, axes = plt.subplots(3, 1, figsize=(FW, 6.4), sharex=True,
189                             gridspec_kw={"height_ratios": [0.75, 1.75, 1.05],
190                                           "hspace": 0.14})
191     ax = axes[0]
192     ax.step(T, price, where="post", color=C["grey"], lw=1.4)

```

```

193 ax.set_ylabel("电价\n (元/kWh) ")
194 ax.set_ylim(0, 1.55)
195 panel(ax, "(a)")
196
197 ax = axes[1]
198 ax.step(T, g[Dm], where="post", color=C["red"], lw=1.5, label="计划购电")
199 ax.bar(T, Cc[Dm], width=W, align="edge", color=C["blue"], alpha=0.8, label="实际充电")
200 ax.bar(T, -Dd[Dm], width=W, align="edge", color=C["green"], alpha=0.8, label="实际放电")
201 m = EM[Dm] > 1e-6
202 ax.bar(T[m], EM[Dm][m], width=W, align="edge", color=C["purple"],
203        label="紧急购电")
204 ax.axhline(0, color="k", lw=0.6)
205 ax.set_ylabel("kWh/10min")
206 ym = max(g[Dm].max(), Cc[Dm].max(), Dd[Dm].max()) * 1.18 + 50
207 ax.set_ylim(-ym, ym)
208 ax.legend(loc="upper left", ncol=4, columnspacing=1.0)
209 panel(ax, "(b)")
210
211 ax = axes[2]
212 soc = soc_st[Dm] + np.cumsum(ETA * Cc[Dm] - Dd[Dm] / ETA)
213 ax.plot(T, soc, color=C["purple"], lw=1.9)
214 ax.axhline(1200, color=C["red"], ls=":", lw=0.9)
215 ax.axhline(10800, color=C["red"], ls=":", lw=0.9)
216 ax.set_ylabel("储电量 (kWh) ")
217 ax.set_ylim(0, 12200)
218 hour_axis(ax)
219 panel(ax, "(c)")
220 audit(fig, "fig_q2_worst_day.png")
221
222
223 # ===== 5. 问题三: 调整策略对比 =====
224 def fig_q3cmp():
225     names = ["不调整", "仅 6 时", "仅 12 时", "仅 18 时", "全调整\n (主策略)"]
226     vals = [KN["q3_cost_noAdj"], KN["q3_cost_adj6"], KN["q3_cost_adj12"],
227            KN["q3_cost_adj18"], KN["q3_cost_main"]]
228     base = vals[0]
229     x = np.arange(5)
230     colors = [C["grey"]] + [C["sky"]] * 3 + [C["red"]]
231     fig, ax = plt.subplots(figsize=(5.4, 3.5))
232     ax.bar(x, np.array(vals) / 1e4, 0.58, color=colors)
233     ax.axhline(base / 1e4, color=C["grey"], ls="--", lw=0.9)
234     for xi, v in zip(x, vals):
235         lab = f"{v/1e4:.1f}" if v >= base else f"{v/1e4:.1f}\n(-{(base-v)/1e4:.1f})"
236         ax.text(xi, v / 1e4 + 0.9, lab, ha="center", va="bottom", fontsize=8.5,
237                fontweight="bold" if xi == 4 else "normal",
238                color=C["dark"] if v >= base else C["green"])
239     ax.set_xticks(x)
240     ax.set_xticklabels(names, fontsize=8.5)
241     ax.set_ylabel("回测期总费用 (万元) ")
242     ax.set_ylim(min(vals) / 1e4 - 5, max(vals) / 1e4 + 5)
243     audit(fig, "fig_q3_adj_cmp.png")
244
245
246 # ===== 6. 问题三: 调整示例日 =====
247 def fig_q3day():
248     sim = np.load(RESULTS / "q3_main_sim.npz")
249     G, A, Cc, Dd = sim["g_plan"], sim["a_final"], sim["c"], sim["d"]

```

```

250 soc_st = sim["soc_start"]
251 adj = np.abs(A - G).sum(axis=1); adj[:31] = 0
252 Dm = int(np.argmax(adj))
253 dstr = (pd.Timestamp("2025-01-01") + pd.Timedelta(days=Dm)).strftime("%Y-%m-%d")
254
255 fig, axes = plt.subplots(2, 1, figsize=(FW, 5.2), sharex=True,
256                          gridspec_kw={"height_ratios": [1.9, 1.0], "hspace": 0.13})
257 ax = axes[0]
258 ax.step(T, G[Dm], where="post", color=C["red"], lw=1.5, label="0:00 计划购电")
259 ax.step(T, A[Dm], where="post", color=C["purple"], lw=1.6, ls="--",
260        label="调整后购电")
261 ax.plot(T, PV[Dm] * DT, color=C["orange"], lw=1.3, label="光伏实际 (xΔt)")
262 ax.axhline(0, color="k", lw=0.5)
263 ym = max(G[Dm].max(), A[Dm].max(), (PV[Dm] * DT).max()) * 1.20 + 50
264 ax.set_ylim(-0.08 * ym, ym)
265 for h, lab in [(6, "6 时"), (12, "12 时"), (18, "18 时")]:
266     ax.axvline(h, color="#555555", ls=":", lw=0.8)
267     ax.text(h + 0.15, ym * 0.93, lab, fontsize=7.5, color="#555555", va="top")
268 ax.set_ylabel("kWh/10min")
269 ax.legend(loc="upper left", ncol=3, columnspacing=1.0)
270 panel(ax, "(a)")
271
272 ax = axes[1]
273 soc = soc_st[Dm] + np.cumsum(ETA * Cc[Dm] - Dd[Dm] / ETA)
274 ax.plot(T, soc, color=C["purple"], lw=1.9)
275 ax.axhline(1200, color=C["red"], ls=":", lw=0.9)
276 ax.axhline(10800, color=C["red"], ls=":", lw=0.9)
277 ax.set_ylabel("储电量 (kWh)")
278 ax.set_ylim(0, 12200)
279 hour_axis(ax)
280 panel(ax, "(b)")
281 audit(fig, "fig_q3_adj_day.png")
282
283
284 # ===== 7. 问题四: 固定 vs 波动 =====
285 def fig_q4():
286     fixed = [KN["q2_cost_baseline"], KN["q2_cost_main"], KN["q3_cost_noAdj"],
287             KN["q3_cost_main"], KN["q2_cost_oracle"]]
288     fluct = [KN["q4_cost_baseline"], KN["q4_2_cost_main"], KN["q4_3_cost_noAdj"],
289            KN["q4_3_cost_main"], KN["q4_2_cost_oracle"]]
290     labels = ["无储能\n基准", "问题二\n模型", "问题三模型\n不调整",
291             "问题三模型\n全调整 (主)", "理想先知"]
292     x = np.arange(5)
293     w = 0.37
294     fig, ax = plt.subplots(figsize=(FW, 3.9))
295     ax.bar(x - w / 2, np.array(fixed) / 1e4, w, color=C["grey"],
296          label="固定电价 (附件1)")
297     ax.bar(x + w / 2, np.array(fluct) / 1e4, w, color=C["red"],
298          label="波动电价 (附件4)")
299     mx = max(max(fixed), max(fluct)) / 1e4
300     for xi, f, v in zip(x, fixed, fluct):
301         ax.text(xi - w / 2, f / 1e4 + 1.4, f"{f/1e4:.0f}", ha="center", fontsize=7.8)
302         ax.text(xi + w / 2, v / 1e4 + 1.4, f"{v/1e4:.0f}", ha="center", fontsize=7.8)
303         ax.text(xi, max(f, v) / 1e4 + 7.2, f"+{(v/f-1)*100:.1f}%", ha="center",
304              fontsize=8, color=C["dark"], fontweight="bold")
305     ax.set_xticks(x)
306     ax.set_xticklabels(labels, fontsize=8.5)

```

```

307     ax.set_ylabel("回溯期总费用 (万元)")
308     ax.set_ylim(0, mx * 1.20)
309     ax.legend(loc="upper right")
310     audit(fig, "fig_q4_cmp.png")
311
312
313 # ===== 8. 灵敏度三联 =====
314 def fig_sens():
315     p1_eta = KN["sens_p1_eta"]; p2_eta = KN["sens_p2_eta"]; p2e = KN["sens_p2_emerg"]
316     fig, axes = plt.subplots(1, 3, figsize=(FW, 2.55))
317     ax = axes[0]
318     k = sorted(float(v) for v in p1_eta)
319     ax.plot([v * 100 for v in k], [p1_eta[str(v)] for v in k], "o-", color=C["dark"])
320     ax.axvline(90, color=C["red"], ls="--", lw=1.0)
321     ax.set_xlabel("充放电效率  $\eta$  (%)")
322     ax.set_ylabel("问题一购电费 (元)")
323     panel(ax, "(a)")
324     ax = axes[1]
325     k = sorted(float(v) for v in p2_eta)
326     ax.plot([v * 100 for v in k], [p2_eta[str(v)] / 1e4 for v in k], "o-",
327            color=C["dark"])
328     ax.axvline(90, color=C["red"], ls="--", lw=1.0)
329     ax.set_xlabel("充放电效率  $\eta$  (%)")
330     ax.set_ylabel("问题二总费用 (万元)")
331     panel(ax, "(b)")
332     ax = axes[2]
333     ms = sorted(int(m) for m in p2e)
334     fix = [p2e[str(m)][0] / 1e4 for m in ms]
335     re_q = [p2e[str(m)][1] / 1e4 for m in ms]
336     ax.plot(ms, fix, "s--", color=C["grey"], label="固定  $q_{f}=0.7$ ")
337     ax.plot(ms, re_q, "o-", color=C["red"], label="按理论重选  $q$ ")
338     ax.axvline(5, color="#888888", ls=":", lw=1.0)
339     ax.set_xlabel("紧急购电倍数  $m$ ")
340     ax.set_ylabel("问题二总费用 (万元)")
341     ax.legend(loc="upper left")
342     panel(ax, "(c)")
343     fig.subplots_adjust(wspace=0.42, bottom=0.16)
344     audit(fig, "fig_sens.png")
345
346
347 if __name__ == "__main__":
348     print("重做全部图表 (v3, 规范管线): ")
349     fig_q1(); fig_q2cmp(); fig_q2sens(); fig_q2day(); fig_q3cmp(); fig_q3day(); fig_q4(); fig_sens()
350     # 灰度预览 (色盲可分性检查用)
351     from PIL import Image
352     for p in FIGURES.glob("fig_*.png"):
353         if p.stem.startswith("fig_"):
354             Image.open(p).convert("L").save(FIGURES / (p.stem + "_gray.png"))
355     print("全部图表已按规范重做并生成灰度预览")

```